# Applications of Deep Reinforcement Learning in Communications and Networking: A Survey

Nguyen Cong Luong, Dinh Thai Hoang, *Member, IEEE*, Shimin Gong, *Member, IEEE*, Dusit Niyato, *Fellow, IEEE*, Ping Wang, *Senior Member, IEEE*, Ying-Chang Liang, *Fellow, IEEE*, and Dong In Kim, *Fellow, IEEE*

***Abstract*—This paper presents a comprehensive literature review on applications of deep reinforcement learning (DRL) in communications and networking. Modern networks, e.g., Internet of Things (IoT) and unmanned aerial vehicle (UAV) networks, become more decentralized and autonomous. In such networks, network entities need to make decisions locally to maximize the network performance under uncertainty of network environment. Reinforcement learning has been efficiently used to enable the network entities to obtain the optimal policy including, e.g., decisions or actions, given their states when the state and action spaces are small. However, in complex and large-scale networks, the state and action spaces are usually large, and the reinforcement learning may not be able to find the optimal policy in reasonable time. Therefore, DRL, a combination of reinforcement learning with deep learning, has been developed to overcome the shortcomings. In this survey, we first give a tutorial of DRL from fundamental concepts to advanced models. Then, we review DRL approaches proposed to address emerging issues in communications and networking. The issues include dynamic network access, data rate control, wireless caching, data offloading, network security, and connectivity preservation which are all important to next generation networks, such as 5G and beyond. Furthermore, we present applications of DRL for traffic routing, resource sharing, and data collection. Finally, we highlight important challenges, open issues, and future research directions of applying DRL.**

***Index Terms*—Deep reinforcement learning, deep Q-learning, networking, communications, spectrum access, rate control, security, caching, data offloading, data collection.**

Manuscript received October 11, 2018; revised January 27, 2019 and March 25, 2019; accepted May 7, 2019. Date of publication May 14, 2019; date of current version November 25, 2019. This work was supported in part by the A*STAR-NTU-SUTD Joint Research Grant Call on Artificial Intelligence for the Future of Manufacturing under Grant RGANS1906, in part by WASP/NTU under Grant M4082187 (4080), in part by the Singapore MOE Tier 1 under Grant 2017-T1-002-007 RG122/17, in part by MOE Tier 2 under Grant MOE2014-T2-2-015 ARC4/15, in part by the Singapore NRF2015-NRF-ISF001-2277, in part by the Singapore EMA Energy Resilience under Grant NRF2017EWT-EP003-041, in part by the National Research Foundation of Korea grant funded by the Korean Government under Grant 2014R1A5A1011478 and Grant 2017R1A2B2003953, in part by the National Natural Science Foundation of China under Grant 61631005, Grant U1801261, and Grant 61571100, in part by NSFC under Grant 61601449, and in part by the Shenzhen Talent Peacock Plan Program under Grant KQTD2015071715073798. *(Corresponding author: Dong In Kim.)*

N. C. Luong and D. Niyato are with the School of Computer Science and Engineering, Nanyang Technological University, Singapore (e-mail: clnguyen@ntu.edu.sg; dniyato@ntu.edu.sg).

D. T. Hoang is with the Faculty of Engineering and Information Technology, University of Technology Sydney, Sydney, NSW 2007, Australia (e-mail: hoang.dinh@uts.edu.au).

S. Gong is with the School of Intelligent Systems Engineering, Sun Yat-sen University, Guangzhou 510275, China (e-mail: gong0012@e.ntu.edu.sg).

P. Wang is with the Department of Electrical Engineering and Computer Science, York University, Toronto, ON M3J 1P3, Canada (e-mail: pingw@yorku.ca).

Y.-C. Liang is with the Center for Intelligent Networking and Communications, University of Electronic Science and Technology of China, Chengdu 610054, China (e-mail: liangyc@ieee.org).

D. I. Kim is with the Department of Electrical and Computer Engineering, Sungkyunkwan University, Suwon 16419, South Korea (e-mail: dikim@skku.ac.kr).

Digital Object Identifier 10.1109/COMST.2019.2916583

## I. Introduction

REINFORCEMENT learning [1] is one of the most important research directions of machine learning which has significant impacts to the development of Artificial Intelligence (AI) over the last 20 years. Reinforcement learning is a learning process in which an agent can periodically make decisions, observe the results, and then automatically adjust its strategy to achieve the optimal policy. However, this learning process, even though proved to converge, takes a lot of time to reach the best policy as it has to explore and gain knowledge of an entire system, making it unsuitable and inapplicable to large-scale networks. Consequently, applications of reinforcement learning are very limited in practice. Recently, deep learning [2] has been introduced as a new breakthrough technique. It can overcome the limitations of reinforcement learning, and thus open a new era for the development of reinforcement learning, namely *Deep Reinforcement Learning* (DRL). DRL embraces the advantage of Deep Neural Networks (DNNs) to train the learning process, thereby improving the learning speed and the performance of reinforcement learning algorithms. As a result, DRL has been adopted in numerous applications of reinforcement learning in practice such as robotics, computer vision, speech recognition, and natural language processing [2]. One of the most famous applications of DRL is AlphaGo [3], the first computer program which can beat a human professional without handicaps on a full-sized 19×19 board.

In the areas of communications and networking, DRL has been recently used as an emerging tool to effectively address various problems and challenges. In particular, modern networks such as Internet of Things (IoT), Heterogeneous Networks (HetNets), and Unmanned Aerial Vehicle (UAV) network become more decentralized, ad-hoc, and autonomous in nature. Network entities such as IoT devices, mobile users, and UAVs need to make local and autonomous decisions, e.g., spectrum access, data rate selection, transmit power control, and base station association, to achieve the goals of different networks including, e.g., throughput maximization

1553-877X © 2019 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:35:20 UTC from IEEE Xplore. Restrictions apply.

and energy consumption minimization. Under uncertain and stochastic environments, most of the decision-making problems can be modeled by a so-called *Markov Decision Process* (MDP) [4]. Dynamic programming [5], [6] and other algorithms such as value iteration, as well as reinforcement learning techniques can be adopted to solve the MDP. However, the modern networks are large-scale and complicated, and thus the computational complexity of the techniques rapidly becomes unmanageable. As a result, DRL has been developing to be an alternative solution to overcome the challenge. In general, the DRL approaches provide the following advantages:

- DRL can obtain the solution of sophisticated network optimizations. Thus, it enables network controllers, e.g., base stations, in modern networks to solve non-convex and complex problems, e.g., joint user association, computation, and transmission schedule, to achieve the optimal solutions without complete and accurate network information.
- DRL allows network entities to learn and build knowledge about the communication and networking environment. Thus, by using DRL, the network entities, e.g., a mobile user, can learn optimal policies, e.g., base station selection, channel selection, handover decision, caching and offloading decisions, without knowing channel model and mobility pattern.
- DRL provides autonomous decision-making. With the DRL approaches, network entities can make observation and obtain the best policy locally with minimum or without information exchange among each other. This not only reduces communication overheads but also improves security and robustness of the networks.
- DRL improves significantly the learning speed, especially in the problems with large state and action spaces. Thus, in large-scale networks, e.g., IoT systems with thousands of devices, DRL allows network controller or IoT gateways to control dynamically user association, spectrum access, and transmit power for a massive number of IoT devices and mobile users.
- Several other problems in communications and networking such as cyber-physical attacks, interference management, and data offloading can be modeled as games, e.g., the non-cooperative game. DRL has been recently used as an efficient tool to solve the games, e.g., finding the Nash equilibrium, without the complete information.

Although there are some surveys related to machine learning, they do not discuss the applications of DRL in communications and networking. Specifically, there are surveys that discuss the applications of DRL such as [7] and [8], but they are specifically for computer vision and natural language processing. Also, there are surveys discussing the applications of machine learning for networking such as [9], [10], [11], [12], and [13]. However, they mostly focus on deep learning approaches. In particular, the survey in [9] discusses deep learning approaches for network cybersecurity, the survey in [10] reviews deep learning approaches for network traffic control, the survey in [11] presents deep learning approaches for physical layer modulation, network access/resource allocation, and network routing, and the survey in [12] presents deep learning approaches for emerging issues including edge caching and computing, multiple radio access and interference management. In summary, the existing surveys either consider applications of DRL for computer vision and natural language processing or discuss applications of deep learning for networking. There is no survey specifically discussing the applications of DRL for communications and networking. This motivates us to deliver the survey with the tutorial of DRL and the comprehensive literature review on the applications of DRL to address issues in communications and networking. For convenience, the related works in this survey are classified based on issues in communications and networking as shown in Fig. 2. The major issues include network access, data rate control, wireless caching, data offloading, network security, connectivity preservation, traffic routing, and data collection. Also, the percentages of DRL related works for different networks and different issues in the networks are shown in Figs. 1(a) and 1(b), respectively. From the figures, we observe that the majority of the related works are for the cellular networks. Also, the related works to the wireless caching and offloading have received more attention than the other issues.

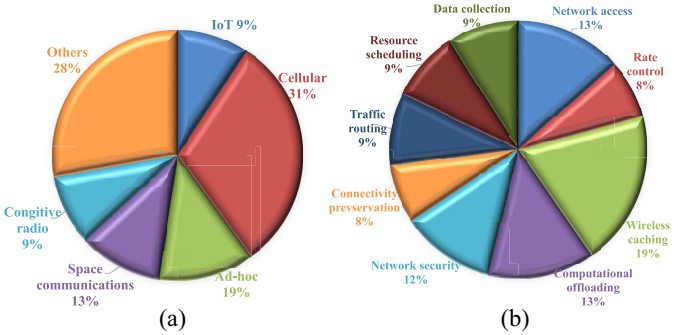



Fig. 1. Percentages of related work for (a) different networks and (b) different issues in the networks.

The rest of this paper is organized as follows. Section II presents the introduction of reinforcement learning and discusses DRL techniques as well as their extensions. Section III reviews the applications of DRL for dynamic network access and adaptive data rate control. Section IV discusses the applications of DRL for wireless caching and data offloading. Section V presents DRL related works for network security and connectivity preservation. Section VI considers how to use DRL to deal with other issues in communications and networking. Important challenges, open issues, and future research directions are outlined in Section VII. Section VIII concludes the paper. The list of abbreviations commonly appeared in this paper is given in Table I. Note that DRL consists of two different algorithms which are Deep Q-Learning (DQL) and policy gradients [14]. In particular, DQL is mostly used for the DRL related works. Therefore, in the rest of the paper, we use "DRL" and "DQL" interchangeably to refer to the DRL algorithms.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:35:20 UTC from IEEE Xplore. Restrictions apply.

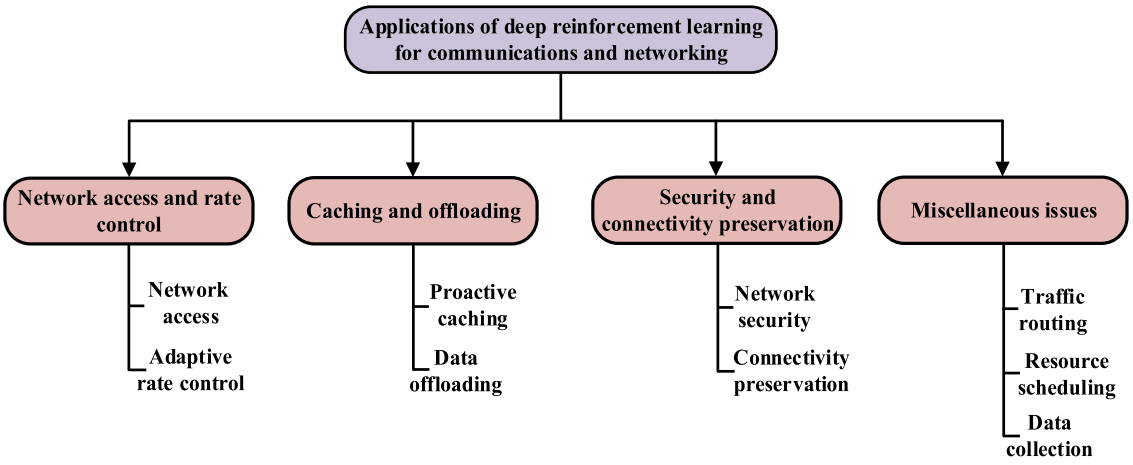



Fig. 2. A taxonomy of the applications of deep reinforcement learning for communications and networking.

## II. Deep Reinforcement Learning: An Overview

In this section, we first present fundamental knowledge of Markov decision processes, reinforcement learning, and deep learning techniques which are important branches of machine learning theory. We then discuss DRL technique that can capitalize on the capability of the deep learning to improve efficiency and performance in terms of the learning rate for reinforcement learning algorithms. Afterward, advanced DRL models and their extensions are reviewed.

### A. Markov Decision Processes

MDP [4] is a discrete time stochastic control process. MDP provides a mathematical framework for modeling decision-making problems in which outcomes are partly random and under control of a decision maker or an agent. MDPs are useful for studying optimization problems which can be solved by dynamic programming and reinforcement learning techniques. Typically, an MDP is defined by a tuple ($\mathcal{S}$, $\mathcal{A}$, $p$, $r$) where $\mathcal{S}$ is a finite set of states, $\mathcal{A}$ is a finite set of actions, $p$ is a transition probability from state $s$ to state $s'$ after action $a$ is executed, and $r$ is the immediate reward obtained after action $a$ is performed. We denote $\pi$ as a "policy" which is a mapping from a state to an action. The goal of an MDP is to find an optimal policy to maximize the reward function. An MDP can be finite or infinite time horizon. For the finite time horizon MDP, an optimal policy $\pi^*$ to maximize the expected total reward is defined by $\max_\pi \mathbb{E}[\sum_{t=0}^{T} r_t(s_t, \pi(s_t))]$, where $a_t = \pi(s_t)$. For the infinite time horizon MDP, the objective can be to maximize the expected discounted total reward or to maximize the average reward. The former is defined by $\max_\pi \mathbb{E}[\sum_{t=0}^{T} \gamma r_t(s_t, \pi(s_t))]$, while the latter is expressed by $\liminf_{T\to\infty} \max_\pi \mathbb{E}[\sum_{t=0}^{T} r_t(s_t, \pi(s_t))]$, where $\gamma \in [0, 1]$ is the discount factor. The discount factor $\gamma$ determines the importance of future rewards compared with the current reward. If $\gamma = 0$, the agent is "myopic", i.e., it only considers to maximize its current reward, i.e., immediate reward. In contrast, if $\gamma$ approaches one, the agent will strive for a long-term higher reward.

TABLE I
List of Abbreviations

| Abbreviation | Description |
|---|---|
| ANN/APF | Artificial Neural Network/Artificial Potential Field |
| A3C | Asynchronous Advantage Actor- Critic |
| CRN | Cognitive Radio Network |
| CNN | Convolutional Neural Network |
| DRL/DQL | Deep Reinforcement Learning/Deep Q-Learning |
| DNN | Deep Neural Network |
| DQN/DDQN/DRQN | Deep Q-Network/Double DQN/Deep Recurrent Q-Learning |
| DASH | Dynamic Adaptive Streaming over HTTP |
| DoS | Denial-of-Service |
| ESN | Echo State Network |
| FNN/RNN | Feedforward Neural Network/Recurrent Neural Network |
| FSMC | Finite-State Markov Channel |
| HVFT | High Volume Flexible Time |
| ITS | Intelligent Transportation System |
| LSM/LSTM | Liquid State Machine/Long Short-Term Memory |
| MEC | Mobile Edge Computing |
| MDP/POMDP | Markov Decision Process/Partially Observable MDP |
| NFSP | Neural Fictitious Self-Play |
| NFV | Network Function Virtualization |
| RDPG | Recurrent Deterministic Policy Gradient |
| RCNN | Recursive Convolutional Neural Network |
| RRH/BBU | Remote Radio Head/BaseBand Unit |
| RSSI | Received Signal Strength Indicators |
| SPD | Sequential Prisoner's Dilemma |
| SBS/BS | Small Base Station/Base Station |
| SDN | Software-Defined Network |
| SU/PU | Secondary User/Primary User |
| UDN/UAN | Ultra-Density Network/Underwater Acoustic Network |
| UAV | Unmanned Aerial Vehicle |
| VANET/V2V | Vehicular Ad hoc Network/Vehicle-to-Vehicle |

*1) Partially Observable Markov Decision Process:* In MDPs, we assume that the system state is fully observable by the agent. However, in many cases, the agent only

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:35:20 UTC from IEEE Xplore. Restrictions apply.

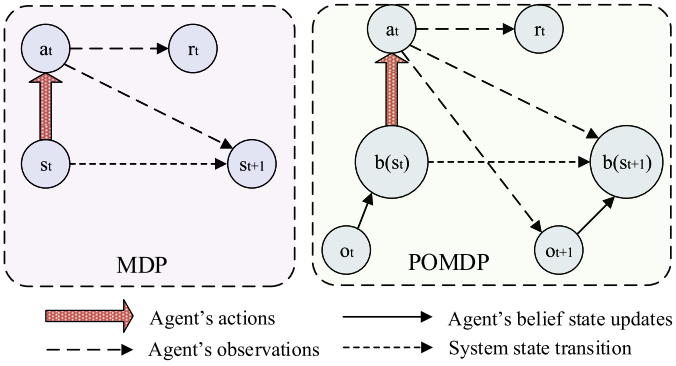



Fig. 3. An illustration of MDP and POMDP.

can observe a part of the system state, and thus Partially Observable Markov Decision Processes (POMDPs) [15] can be used to model the decision-making problems. A typical POMDP model is defined by a 6-tuple $(\mathcal{S}, \mathcal{A}, p, r, \Omega, \mathcal{O})$, where $\mathcal{S}$, $\mathcal{A}$, $p$, $r$ are defined the same as in the MDP model, $\Omega$ and $\mathcal{O}$ are defined as the set of observations and observation probabilities, respectively. At each time epoch, the agent is at state $s$, selects an action $a$ based on its belief about the current state $s$, i.e., $b(s)$, and observes the immediate reward $r$ and current observation $o$. Based on the observation $o$ and its belief about the current state $b(s)$, the agent then updates its belief about the new state $s'$, i.e., $b(s')$, as follows [15]:

$$b(s') = \frac{\mathcal{O}(o|s,a,s') \sum_{s\in\mathcal{S}} p(s'|s,a) b(s)}{\sum_{s'\in\mathcal{S}} \mathcal{O}(o|s,a,s') \sum_{s\in\mathcal{S}} p(s'|s,a) b(s)}, \quad (1)$$

where $\mathcal{O}(o|s,a,s')$ is the probability that the agent obtains observation $o$ after action $a$ is taken at state $s$ and the agent moves to state $s'$. $p(s'|s,a)$ is defined the same as that of the MDP model, i.e., the transition probability from state $s$ to state $s'$ after action $a$ is performed at state $s$. Finally, the agent receives an immediate reward $r$ that is equal to $r(s, a)$ in the MDP. Similar to the MDP model, the agent in POMDP also aims to find the optimal policy $\pi^*$ in order to maximize its expected long-term discounted reward $\sum_{t=0}^{\infty} \gamma r_t(s_t, \pi^*(s_t))$. Fig. 3 illustrates and compares an MDP and a POMDP.

*2) Markov Games:* In game theory, a Markov game, or a stochastic game [16], is a dynamic game with probabilistic transitions played by multiple players, i.e., agents. A typical Markov game model is defined by a tuple $(\mathcal{I}, \mathcal{S}, \{\mathcal{A}^i\}_{i\in\mathcal{I}}, p, \{r^i\}_{i\in\mathcal{I}})$, where

- $\mathcal{I} \triangleq \{1, \ldots, i, \ldots, I\}$ is a set of agents,
- $\mathcal{S} \triangleq \{\mathcal{S}^1, \ldots, \mathcal{S}^i, \ldots, \mathcal{S}^I\}$ is the global state space of all agents with $\mathcal{S}^i$ being the state space of agent $i$,
- $\{\mathcal{A}^i\}_{i\in\mathcal{I}}$ are sets of action spaces of the agents with $\mathcal{A}^i$ being the action space of agent $i$,
- $p \triangleq \mathcal{S} \times \mathcal{A}^1 \times \cdots \times \mathcal{A}^I \to [0, 1]$ is the transition probability function of the system.
- $\{r^i\}_{i\in\mathcal{I}}$ are payoff functions of the agents with $r^i \triangleq \mathcal{S} \times \mathcal{A}^1 \times \cdots \times \mathcal{A}^I \to \mathbb{R}$, i.e., the payoff of agent $i$ obtained after all actions of the agents are executed.

In a Markov game, the agents start at some initial state $s_0 \in \mathcal{S}$. After observing the current state, all the agents simultaneously select their actions $a = \{a^1, \ldots, a^I\}$ and they will receive their corresponding rewards together with their own new observations. At the same time, the system will transit to a new state $s' \in \mathcal{S}$ with probability $p(s'|s, a)$. The procedure is repeated at the new state and continues for a finite or infinite number of stages. In this game, all the agents try to find their optimal policies to maximize their own expected long-term average rewards, i.e., $\sum_{t=0}^{\infty} \gamma_i r_t^i(s_t, \pi_i^*(s_t))$, $\forall i$. The set of all optimal policies of this game, i.e., $\{\pi_1^*, \ldots, \pi_I^*\}$ is known to be the equilibrium of this game. If there is a finite number of players and the sets of states and actions are finite, then the Markov game always has a Nash equilibrium [17] under a finite number of stages. The same is true for Markov games with infinite stages, but the total payoff of agents is the discounted sum [17].

### B. Reinforcement Learning

Reinforcement learning, an important branch of machine learning, is an effective tool and widely used in the literature to address MDPs [1]. In a reinforcement learning process, an agent can learn its optimal policy through interaction with its environment. In particular, the agent first observes its current state, and then takes an action, and receives its immediate reward together with its new state as illustrated in Fig. 4(a). The observed information, i.e., the immediate reward and new state, is used to adjust the agent's policy, and this process will be repeated until the agent's policy approaches to the optimal policy. In reinforcement learning, $Q$-learning is the most effective method and widely used in the literature. In the following, we will discuss the $Q$-learning algorithm and its extensions for advanced MDP models.

*1) Q-Learning Algorithm:* In an MDP, we aim to find an optimal policy $\pi^* : \mathcal{S} \to \mathcal{A}$ for the agent to maximize the expected long-term reward function for the system. Accordingly, we first define value function $\mathcal{V}^\pi : \mathcal{S} \to \mathbb{R}$ that represents the expected value obtained by following policy $\pi$ from each state $s \in \mathcal{S}$. The value function $\mathcal{V}$ for policy $\pi$ quantifies the goodness of the policy through an infinite horizon and discounted MDP that can be expressed as follows:

$$\begin{aligned}\mathcal{V}^\pi(s) &= \mathbb{E}_\pi\left[\sum_{t=0}^{\infty} \gamma r_t(s_t, a_t) | s_0 = s\right] \\ &= \mathbb{E}_\pi[r_t(s_t, a_t) + \gamma \mathcal{V}^\pi(s_{t+1}) | s_0 = s]. \end{aligned} \quad (2)$$

Since we aim to find the optimal policy $\pi^*$, an optimal action at each state can be found through the optimal value function expressed by $\mathcal{V}^*(s) = \max_{a_t}\{\mathbb{E}_\pi[r_t(s_t, a_t) + \gamma \mathcal{V}^\pi(s_{t+1})]\}$.

If we denote $\mathcal{Q}^*(s, a) \triangleq r_t(s_t, a_t) + \gamma \mathbb{E}_\pi[\mathcal{V}^\pi(s_{t+1})]$ as the optimal $Q$-function for all state-action pairs, then the optimal value function can be written by $\mathcal{V}^*(s) = \max_a\{\mathcal{Q}^*(s, a)\}$. Now, the problem is reduced to find optimal values of $Q$-function, i.e., $\mathcal{Q}^*(s, a)$, for all state-action pairs, and this can be done through iterative processes. In particular, the $Q$-function is updated according to the following rule:

$$\begin{aligned}\mathcal{Q}_{t+1}(s, a) &= \mathcal{Q}_t(s, a) \\ &\quad + \alpha_t\left[r_t(s, a) + \gamma \max_{a'} \mathcal{Q}_t(s, a') - \mathcal{Q}_t(s, a)\right]. \end{aligned} \quad (3)$$

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:35:20 UTC from IEEE Xplore. Restrictions apply.

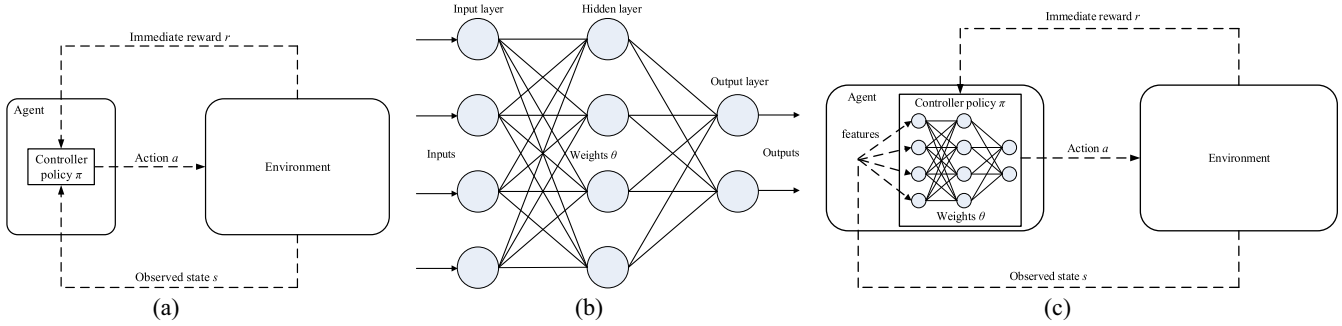



Fig. 4. (a) Reinforcement learning, (b) Artificial neural network, and (c) Deep Q-learning.

**Algorithm 1** The $Q$-Learning Algorithm

**Input:** For each state-action pair ($s$, $a$), initialize the table entry $\mathcal{Q}(s, a)$ arbitrarily, e.g., to zero. Observe the current state $s$, initialize a value for the learning rate $\alpha$ and the discount factor $\gamma$.

**for** $t := 1$ to $T$ **do**

From the current state-action pair ($s$, $a$), execute action $a$ and obtain the immediate reward $r$ and a new state $s'$.

Select an action $a'$ based on the state $s'$ and then update the table entry for $\mathcal{Q}(s, a)$ as follows:

$$\mathcal{Q}_{t+1}(s,a) \leftarrow \mathcal{Q}_t(s,a) + \alpha_t \Big[ r_t(s,a) + \gamma \max_{a'} \mathcal{Q}_t(s',a') - \mathcal{Q}_t(s,a) \Big] \tag{4}$$

Replace $s \leftarrow s'$.

**end for**

**Output:** $\pi^*(s) = \arg\max_a \mathcal{Q}^*(s,a)$.

The core idea behind this update is to find the Temporal Difference (TD) between the predicted $\mathcal{Q}$-value, i.e., $r_t(s,a) + \gamma \max_{a'} \mathcal{Q}_t(s,a')$ and its current value, i.e., $\mathcal{Q}_t(s,a)$. In (3), the learning rate $\alpha_t$ is used to determine the impact of new information to the existing $\mathcal{Q}$-value. The learning rate can be chosen to be a constant, or it can be adjusted dynamically during the learning process. However, it must satisfy Assumption 1 to guarantee the convergence for the $Q$-learning algorithm.

*Assumption 1:* The step size $\alpha_t$ is deterministic, nonnegative and satisfies the following conditions: $\alpha_t \in [0,1]$, $\sum_{t=0}^{\infty} \alpha_t = \infty$, and $\sum_{t=0}^{\infty} (\alpha_t)^2 < \infty$.

The step size adaptation $\alpha_t = \frac{1}{t}$ is one of the most common examples used in reinforcement learning. More discussions for selecting an appropriate step size can be found in [18]. The details of the $Q$-learning algorithm are then provided in Algorithm 1.

Once either all $\mathcal{Q}$-values converge or a certain number of iterations is reached, the algorithm will terminate. The algorithm then yields the optimal policy indicating an action to be taken at each state such that $\mathcal{Q}^*(s,a)$ is maximized for all states in the state space, i.e., $\pi^*(s) = \arg\max_a \mathcal{Q}^*(s,a)$. Under the assumption of the step size (i.e., Assumption 1), it is proved in [19] that the $Q$-learning algorithm converges to the optimum action-values with probability one.

It is worth noting that unlike value function $\mathcal{V}^\pi$, Q-function is an example of model-free learning algorithm that does not require the agent to know the system model parameters, e.g., the state-transition and reward models, in advance to estimate the pairs of state-action values. Specifically, the core idea of Q-function is to approximate the values of state-action pairs through samples obtained during the interactions with the environment. Furthermore, while the value function takes the expectation of all actions according to the policy $\pi$, the Q-function only focuses on a particular action at a particular state. As a result, learning algorithms using Q-function are less complex than those of using the value function. However, from the sampling perspective, the dimension of Q-function is higher than that of value function, and thus Q-function might be more difficult to get enough samples, i.e., state-action pairs, to learn. Therefore, if the system model is available in advance, the value function is usually preferable.

*2) SARSA (An Online Q-Learning Algorithm):* Although the $Q$-learning algorithm can find the optimal policy for the agent without requiring knowledge about the environment, this algorithm works in an offline fashion. In particular, Algorithm 1 can obtain the optimal policy only after all $\mathcal{Q}$-values converge. Therefore, this section presents an alternative online learning algorithm, i.e., the SARSA algorithm, which allows the agent to approach the optimal policy in an online fashion.

Different from the $Q$-learning algorithm, the SARSA algorithm is an online algorithm which allows the agent to choose optimal actions at each time step in a real-time fashion without waiting until the algorithm converges. In the $Q$-learning algorithm, the policy is updated according to the maximum reward of available actions regardless of which policy is applied, i.e., an off-policy method. In contrast, the SARSA algorithm interacts with the environment and updates the policy directly from the actions taken, i.e., an on-policy method. Note that the SARSA algorithm updates $\mathcal{Q}$-values from the quintuple $\mathcal{Q}(s,a,r,s',a')$.

*3) Q-Learning for Markov Games:* To apply Q-learning algorithm to the Markov game context, we first define the $Q$-function for agent $i$ by $\mathcal{Q}_i(s, a^i, \mathbf{a}^{-i})$, where $\mathbf{a}^{-i} \triangleq \{a^1, \ldots, a^{i-1}, a^{i+1}, \ldots, a^I\}$ denotes the set of actions of all agents except $i$. Then, the Nash $Q$-function of

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:35:20 UTC from IEEE Xplore. Restrictions apply.

agent $i$ is defined by:

$$\mathcal{Q}_i^*\left(s, a^i, \mathbf{a}^{-i}\right) = r^i\left(s, a^i, \mathbf{a}^{-i}\right) + \beta \sum_{s' \in \mathcal{S}} p\left(s' | s, a^i, \mathbf{a}^{-i}\right) \times \mathcal{V}^i\left(s', \pi_1^*, \ldots, \pi_I^*\right), \quad (5)$$

where $(\pi_1^*, \ldots, \pi_I^*)$ is the joint Nash equilibrium strategy, $r^i(s, a^i, \mathbf{a}^{-i})$ is agent $i$'s immediate reward in state $s$ under the joint action $(a^i, \mathbf{a}^{-i})$, and $\mathcal{V}^i(s', \pi_1^*, \ldots, \pi_I^*)$ is the total discounted reward over an infinite time horizon starting from state $s'$ given that all the agents follow the equilibrium strategies.

In [17], the authors propose a multi-agent Q-learning algorithm for general-sum Markov games which allows the agents to perform updates based on assuming Nash equilibrium behavior over the current Q-values. In particular, agent $i$ will learn its $Q$-values by forming an arbitrary guess from starting time of the game. At each time step $t$, agent $i$ observes the current state and takes an action $a^i$. Then, it observes its immediate reward $r^i$, actions taken by others $\mathbf{a}^{-i}$, others' immediate rewards, and the new system state $s'$. After that, agent $i$ calculates a Nash equilibrium $(\pi_1(s'), \ldots, \pi_I(s'))$ for the state game $(\mathcal{Q}_1^t(s'), \ldots, \mathcal{Q}_I^t(s'))$, and updates its $Q$-values according to:

$$\mathcal{Q}_i^{t+1}\left(s, a^i, \mathbf{a}^{-i}\right) = (1 - \alpha_t)\mathcal{Q}_i^t\left(s, a^i, \mathbf{a}^{-i}\right) + \alpha_t\left[r_t^i + \gamma \mathscr{N}_t{}^i(s')\right], \quad (6)$$

where $\alpha_t \in (0, 1)$ is the learning rate and $\mathscr{N}_t{}^i(s') \triangleq \mathcal{Q}_i^t(s') \times \pi_1(s') \times \cdots \times \pi_I(s')$.

In order to calculate the Nash equilibrium, agent $i$ needs to know $(\mathcal{Q}_1^t(s'), \ldots, \mathcal{Q}_I^t(s'))$. However, the information about other agents' $\mathcal{Q}$-values is not given, and thus agent $i$ must learn this information too. To do so, agent $i$ will set estimations about others' $\mathcal{Q}$-values at the beginning of the game, e.g., $\mathcal{Q}_0^j(s, a^i, \mathbf{a}^{-i}) = 0, \forall j, s$. As the game proceeds, agent $i$ observes other agents' immediate rewards and previous actions. That information can then be used to update agent $i$'s conjectures on other agents' $Q$-functions. Agent $i$ updates its beliefs about agent $j$'s $Q$-function, according to the same updating rule in (6). Then, the authors prove that under some highly restrictive assumptions on the form of the state games during learning, the proposed multi-agent $Q$-learning algorithm is guaranteed to be converged.

### C. Deep Learning

Deep learning [2] is composed of a set of algorithms and techniques that attempt to find important features of data and to model its high-level abstractions. The main goal of deep learning is to avoid manual description of a data structure (like hand-written features) by automatic learning from the data. Its name refers to the fact that typically any neural network with two or more hidden layers is called DNN. Most deep learning models are based on an Artificial Neural Network (ANN), even though they can also include propositional formulas or latent variables organized layer-wise in deep generative models such as the nodes in Deep Belief Networks and Deep Boltzmann Machines.

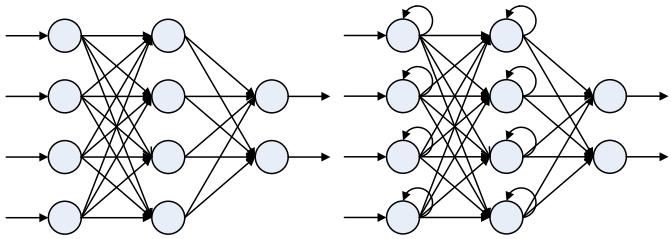



Fig. 5. FNN and RNN.

An ANN is a computational nonlinear model based on the neural structure of the brain that is able to learn to perform tasks such as classification, prediction, decision-making, and visualization. An ANN consists of artificial neurons and is organized into three interconnected layers: input, hidden, and output as illustrated in Fig. 4(b). The input layer contains input neurons that send information to the hidden layer. The hidden layer sends data to the output layer. Every neuron has weighted inputs (synapses), an activation function, and one output. Synapses are the adjustable parameters that convert a neural network to a parameterized system. The activation function of a node defines the outputs of that node given the inputs. In particular, the activation function will map the input values into target ranges depending on the selected activation function. For example, the logistic activation function will map all inputs in the real number domain into the range of 0 to 1.

During the training phase, ANNs use backpropagation as an effective learning algorithm to compute quickly a gradient descent with respect to the weights. Backpropagation is a special case of automatic differentiation. In the context of learning, backpropagation is commonly used by the gradient descent optimization algorithm to adjust the weights of neurons by calculating the gradient of the loss function. This technique is also sometimes called backward propagation of errors, because the error is calculated at the output and distributed back through the network layers.

A DNN is defined as an ANN with multiple hidden layers. There are two typical DNN models, i.e., Feedforward Neural Network (FNN) and Recurrent Neural Network (RNN). In the FNN, the information moves in only one direction, i.e., from the input nodes, through the hidden nodes and to the output nodes, and there are no cycles or loops in the network as shown in Fig. 5. In FNNs, Convolutional Neural Network (CNN) is the most well known model with a wide range of applications especially in image and speech recognition. The CNN contains one or more convolutional layers, pooling or fully connected, and uses a variation of multilayer perceptrons discussed above. In general, CNNs have two main components, i.e., feature extraction and classification, as illustrated in Fig. 6. The feature extraction component is placed at the hidden layers with the aim of performing a series of convolutions and pooling operations during which the features are being detected. After that, the classification component, placed at the fully connected layers, will assign a probability for the object, e.g., on the image, to what we need to predict.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:35:20 UTC from IEEE Xplore. Restrictions apply.

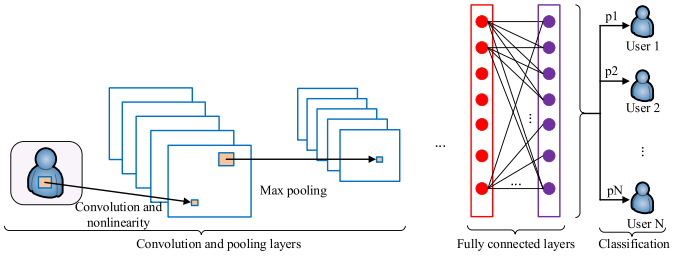



Fig. 6. An illustration of CNN.

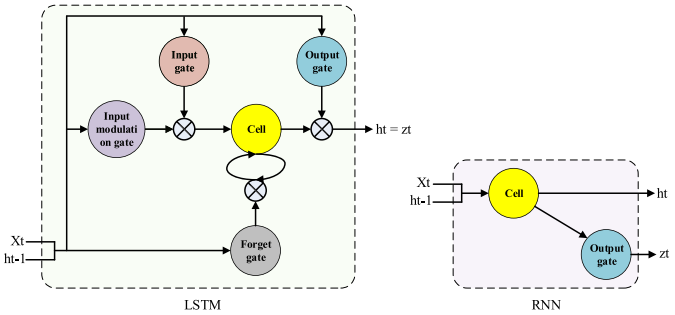



Fig. 7. An illustration of a basic (a) LSTM and (b) RNN cell.

Unlike FNNs, the RNN is a variant of a recursive artificial neural network in which connections between neurons make directed cycles. It means that an output depends not only on its immediate inputs, but also on the previous further step's neuron state. The RNNs are designed to utilize sequential data, when the current step has some relation with the previous steps. This makes the RNNs ideal for applications with a time component, e.g., time-series data, and natural language processing. However, all RNNs have feedback loops in the recurrent layer. This lets RNNs maintain information in memory over time. Nevertheless, it can be difficult to train standard RNNs to solve problems that require learning long-term temporal dependencies. The reason is that the gradient of the loss function decays exponentially with time, which is called the vanishing gradient problem. Thus, Long Short-Term Memory (LSTM) is often used in RNNs to address this issue. LSTMs are designed to model temporal sequences and their long-range dependencies are more accurate than conventional RNNs. In particular, LSTMs provide a solution by incorporating memory units that allow the network to learn when to forget previous hidden states and when to update hidden states given new information. Usually, LSTM units are implemented in "blocks" which have three or four "gates", e.g., input gate, forget gate, output gate and input modulation gate, as illustrated in Fig. 7 [20] to control information flow drawing on the logistic function. Unlike the RNN, the LSTM memory cell is composed of three components, i.e., the previous memory cell $c_t$, current input $x_t$ and previous hidden state $h_{t-1}$. Here, the input gate and forget gate will be used to selectively forget its previous memory or consider its current input. Likewise, the output gate learns how much of the memory cell to transfer to the hidden state. These additional blocks enable the LSTM to learn extremely complex and long-term temporal dynamics that the RNN is unable to do.

### *D. Deep Q-Learning*

The $Q$-learning algorithm can efficiently obtain an optimal policy when the state space and action space are small. However, in practice, with complicated system models, these spaces are usually large. As a result, the $Q$-learning algorithm may not be able to find the optimal policy. Thus, Deep $Q$-Learning (DQL) algorithm is introduced to overcome this shortcoming. Intuitively, the DQL algorithm implements a Deep $Q$-Network (DQN), i.e., a DNN, instead of the $Q$-table to derive an approximate value of $Q^*(s, a)$ as shown in Fig. 4(c).

As stated in [21], the average reward obtained by reinforcement learning algorithms may not be stable or even diverge when a nonlinear function approximator is used. This stems from the fact that a small change of $\mathcal{Q}$-values may greatly affect the policy. Thus, the data distribution and the correlations between the $\mathcal{Q}$-values and the target values $R + \gamma \max_{a'} \mathcal{Q}(s', a')$ are varied. To address this issue, two mechanisms, i.e., experience replay and target $Q$-network, can be used.

- *Experience replay mechanism:* The algorithm first initializes a replay memory **D**, i.e., the memory pool, with transitions $(s_t, a_t, r_t, s_{t+1})$, i.e., experiences, generated randomly, e.g., through using $\epsilon$-greedy policy. Then, the algorithm randomly selects samples, i.e., minibatches, of transitions from **D** to train the DNN. The Q-values obtained by the trained DNN will be used to obtain new experiences, i.e., transitions, and these experiences will then be stored in the memory pool **D**. This mechanism allows the DNN trained more efficiently by using both old and new experiences. In addition, by using the experience replay, the transitions are more independent and identically distributed, and thus the correlations between observations can be removed.
- *Fixed target Q-network:* In the training process, the $\mathcal{Q}$-value will be shifted. Thus, the value estimations can be out of control if a constantly shifting set of values is used to update the $Q$-network. This leads to the destabilization of the algorithm. To address this issue, the target $Q$-network is used to update frequently but slowly the primary $Q$-networks' values. In this way, the correlations between the target and estimated $\mathcal{Q}$-values are significantly reduced, thereby stabilizing the algorithm.

The DQL algorithm with experience replay and fixed target $Q$-network is presented in Algorithm 2. DQL inherits and promotes advantages of both reinforcement and deep learning techniques, and thus it has a wide range of applications in practice such as game development [3], transportation [22], and robotics [23]. Table II summarizes how different the DQL approach and other techniques, i.e., reinforcement learning, deep learning, and traditional combinatorial optimization methods, solve optimization problems.

### *E. Advanced Deep Q-Learning Models*

*1) Double Deep Q-Learning:* In some stochastic environments, the $Q$-learning algorithm performs poorly due to the large over-estimations of action values [24]. These over-estimations result from a positive bias that is introduced

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:35:20 UTC from IEEE Xplore. Restrictions apply.

TABLE II
COMPARISONS AMONG OPTIMIZATION TECHNIQUES

| Techniques | Problem-solving |
|---|---|
| Non-linear programming | Used to address static optimization problems, i.e., optimizes the objective function for one time instant only. To address this optimization, we can analyze the objective function and adopt suitable techniques. For example, if the objective function is quadratic and the constraints are linear, quadratic programming techniques can be used. |
| Dynamic programming | Used to address complex problem by breaking it down into a set of simpler subproblems over multiple steps, solving each of those subproblems just once at at a time, and storing their solutions in a memory. Thus, in the future, if the same subproblem occurs, instead of recomputing its solution, one simply looks up the previously computed solution, thereby saving computation time. |
| Reinforcement Learning (RL) | This is a branch of machine learning used to help an agent to learn the optimal policy when the agent has no information about the surrounding environment. Specifically, the agent first observes its current state, and then takes an action, and receives its immediate reward together with its new state. The observed information, i.e., the immediate reward and new state, is used to adjust the agent's policy, and this process is repeated until the agent's policy approaches the optimal policy. |
| Deep Learning (DL) | This is a branch of machine learning used to help an agent to learn the optimal policy when the agent has some information about surrounding environment in advance. In particular, based on the obtained information (i.e., stored in a database), the agent will train the neural network and find the optimal parameters for the network. The trained neural network will be then implemented on the agent to help the agent make decisions in an online fashion. |
| Deep Reinforcement Learning (DRL) | This is an advanced model of reinforcement learning technique in which deep learning is utilized as an effective tool to improve learning rate for reinforcement learning algorithms. In particular, during real-time learning process, the obtained experiences will be stored and used as the data to train the neural network. The trained neural network will be then used to help the agent make optimal decisions in a real-time manner. Note that unlike deep learning technique, the neural network in the DRL will be trained frequently based on new experiences obtained during the real-time interactions with surrounding environment. |

**Algorithm 2** The DQL Algorithm With Experience Replay and Fixed Target $Q$-Network

1: Initialize replay memory $\mathbf{D}$.
2: Initialize the $Q$-network $\mathbf{Q}$ with random weights $\boldsymbol{\theta}$.
3: Initialize the target $Q$-network $\hat{\mathbf{Q}}$ with random weights $\boldsymbol{\theta}'$.
4: **for** *episode=1 to T* **do**
5: With probability $\epsilon$ select a random action $a_t$, otherwise select $a_t = \arg\max \mathcal{Q}^*(s_t, a_t, \boldsymbol{\theta})$.
6: Perform action $a_t$ and observe immediate reward $r_t$ and next state $s_{t+1}$.
7: Store transition $(s_t, a_t, r_t, s_{t+1})$ in $\mathbf{D}$.
8: Select randomly samples c$(s_j, a_j, r_j, s_{j+1})$ from $\mathbf{D}$.
9: The weights of the neural network then are optimized by using stochastic gradient descent with respect to the network parameter $\boldsymbol{\theta}$ to minimize the loss:

$$\left[r_j + \gamma \max_{a_{j+1}} \hat{\mathcal{Q}}(s_{j+1}, a_{j+1}; \boldsymbol{\theta}') - \mathcal{Q}(s_j, a_j; \boldsymbol{\theta})\right]^2. \quad (7)$$

10: Reset $\hat{\mathbf{Q}} = \mathbf{Q}$ after every a fixed number of steps.
11: **end for**

because $Q$-learning uses the maximum action value as an approximation for the maximum expected action value as shown in Eq. (4). The reason is that the same samples are used to decide which action is the best, i.e., with highest expected reward, and the same samples are also used to estimate that action-value. Thus, to overcome the over-estimation problem of the $Q$-learning algorithm, the authors in [25] introduce a solution using two $Q$-value functions, i.e., $\mathcal{Q}_1$ and $\mathcal{Q}_2$, to simultaneously select and evaluate action values. In particular, the selection of an action is still due to the online weights $\boldsymbol{\theta}_1$. This means that, as in $Q$-learning, we are still estimating the value of the greedy policy according to the current values, as defined by $\boldsymbol{\theta}_1$. However, the second set of weights $\boldsymbol{\theta}_2$ is used to evaluate fairly the value of this policy. This second set of weights can be updated symmetrically by switching the roles of $\boldsymbol{\theta}_1$ and $\boldsymbol{\theta}_2$. Inspired by this idea, the authors in [25] then develop Double Deep $Q$-Learning (DDQL) model [26] using a Double Deep Q-Network (DDQN) with the loss function updated as follows:

$$\left[r_j + \gamma \hat{\mathcal{Q}}\left(s_{j+1}, \arg\max_{a_{j+1}} \mathcal{Q}(s_{j+1}, a_{j+1}; \boldsymbol{\theta}); \boldsymbol{\theta}'\right) - \mathcal{Q}(s_j, a_j; \boldsymbol{\theta})\right]^2. \quad (8)$$

Unlike double $Q$-learning, the weights of the second network $\boldsymbol{\theta_2}$ are replaced with the weights of the target networks $\boldsymbol{\theta}'$ for the evaluation of the current greedy policy as shown in Eq. (8). The update to the target network stays unchanged from DQN, and remains a periodic copy of the online network. Due to the effectiveness of DDQL, there are some applications of DDQL introduced recently to address dynamic spectrum access problems in multichannel wireless networks [27] and resource allocation in heterogeneous networks [28].

*2) Deep Q-Learning With Prioritized Experience Replay:* Experience replay mechanism allows the reinforcement learning agent to remember and reuse experiences, i.e., transitions, from the past. In particular, transitions are uniformly sampled from the replay memory $\mathbf{D}$. However, this approach simply replays transitions at the same frequency as that the agent was originally experienced, regardless of their significance.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:35:20 UTC from IEEE Xplore. Restrictions apply.

Therefore, the authors in [29] develop a framework for prioritizing experiences, so as to replay important transitions more frequently, and therefore learn more efficiently. Ideally, we want to sample more frequently those transitions from which there is much to learn. In general, the DQL with the Prioritized Experience Replay (PER) samples transitions with a probability related to the last encountered absolute error [29]. New transitions are inserted into the replay buffer with maximum priority, providing a bias towards recent transitions. Note that stochastic transitions may also be favoured, even when there is little left to learn about them. Through real experiments on many Atari games, the authors demonstrate that DQL with PER outperforms DQL with uniform replay on 41 out of 49 games. However, this solution is only appropriate to implement when we can find and define the important experiences in the replay memory **D**.

*3) Dueling Deep Q-Learning:* The *Q*-values, i.e., $\mathcal{Q}(s,a)$, used in the *Q*-learning algorithm, i.e., Algorithm 1, are to express how good it is to take a certain action at a given state. The value of an action *a* at a given state *s* can actually be decomposed into two fundamental values. The first value is the state-value function, i.e., $\mathscr{V}(s)$, to estimate the importance of being in a particular state $s$. The second value is the action-value function, i.e., $\mathscr{A}(a)$, to estimate the importance of selecting an action *a* compared with other actions. As a result, the *Q*-value function can be expressed by two fundamental value functions as follows: $\mathcal{Q}(s,a) = \mathscr{V}(s) + \mathscr{A}(a)$.

Stemming from the fact that in many MDPs, it is unnecessary to estimate both values, i.e., action and state values of Q-function $\mathcal{Q}(s,a)$, at the same time. For example, in many racing games, moving left or right matters if and only if the agent meets the obstacles or enemies. Inspired by this idea, the authors in [30] introduce an idea of using two streams, i.e., two sequences, of fully connected layers instead of using a single sequence with fully connected layers for the DQN. The two streams are constructed such that they are able to provide separate estimations on the action and state value functions, i.e., $\mathscr{V}(s)$ and $\mathscr{A}(a)$. Finally, the two streams are combined to generate a single output $\mathcal{Q}(s,a)$ as follows:

$$\mathcal{Q}(s,a;\boldsymbol{\alpha},\boldsymbol{\beta}) = \mathscr{V}(s;\boldsymbol{\beta}) + \left(\mathscr{A}(s,a;\boldsymbol{\alpha}) - \frac{\sum_{a'}\mathscr{A}(s,a';\boldsymbol{\alpha})}{|\mathcal{A}|}\right), \tag{9}$$

where $\boldsymbol{\beta}$ and $\boldsymbol{\alpha}$ are the parameters of the two streams $\mathscr{V}(s;\boldsymbol{\beta})$ and $\mathscr{A}(s,a';\boldsymbol{\alpha})$, respectively. Here, $|\mathcal{A}|$ is the total number of actions in the action space $\mathcal{A}$. Then, the loss function is derived in the similar way to (7). Through the simulation, the authors show that the proposed dueling DQN can outperform DDQN [26] in 50 out of 57 learned Atari games. However, the proposed dueling architecture only clearly benefits for MDPs with large action spaces. For small state spaces, the performance of dueling DQL is even not as good as that of double DQL as shown in simulation results in [30].

*4) Asynchronous Multi-Step Deep Q-Learning:* Most of the *Q*-learning methods such as DQL and dueling DQL rely on the experience replay method. However, such kind of method has several drawbacks. For example, it uses more memory and computation resources per real interaction, and it requires off-policy learning algorithms that can update from data generated by an older policy. This limits the applications of DQL. Therefore, the authors in [31] introduce a method using multiple agents to train the DNN in parallel. In particular, the authors propose a training procedure which utilizes asynchronous gradient decent updates from multiple agents at once. Instead of training one single agent that interacts with its environment, multiple agents are interacting with their own version of the environment simultaneously. After a certain amount of timesteps, accumulated gradient updates from an agent are applied to a global model, i.e., the DNN. These updates are asynchronous and lock free. In addition, to tradeoff between bias and variance in the policy gradient, the authors adopt *n*-step updates method [1] to update the reward function. In particular, the truncated *n*-step reward function can be defined by $r_t^{(n)} = \sum_{k=0}^{n-1}\gamma^{(k)} r_{t+k+1}$. Thus, the alternative loss for each agent will be derived by:

$$\left[r_j^{(n)} + \gamma_j^{(n)} \max_{a'} \hat{\mathcal{Q}}\big(s_{j+n}, a'; \boldsymbol{\theta}'\big) - \mathcal{Q}\big(s_j, a_j; \boldsymbol{\theta}\big)\right]^2. \tag{10}$$

The effects of training speed and quality of the proposed asynchronous DQL with multi-step learning are analyzed for various reinforcement learning methods, e.g., 1-step *Q*-learning, 1-step SARSA, and n-step *Q*-learning. They show that asynchronous updates have a stabilizing effect on policy and value updates. Also, the proposed method outperforms the current state-of-the-art algorithms on the Atari games while training for half of the time on a single multi-core CPU instead of a GPU. As a result, some recent applications of asynchronous DQL have been developed for handover control problems in wireless systems [32]

*5) Distributional Deep Q-Learning:* All aforementioned methods use the Bellman equation to approximate the expected value of future rewards. However, if the environment is stochastic in nature and the future rewards follow multimodal distribution, choosing actions based on expected value may not lead to the optimal outcome. For example, we know that the expected transmission time of a packet in a wireless network is 20 minutes. However, this information may not be so meaningful because it may overestimate the transmission time most of the time. For example, the expected transmission time is calculated based on the normal transmissions (without collisions) and the interference transmissions (with collisions). Although the interference transmissions are very rare to happen, but it takes a lot of time. Then, the estimation about the expected transmission is overestimated most of the time. This makes estimations not useful for the DQL algorithms.

Thus, the authors in [33] introduce a solution using distributional reinforcement learning to update *Q*-value function based on its distribution rather than its expectation. In particular, let $\mathcal{Z}(s,a)$ be the return obtained by starting from state $s$, executing action *a*, and following the current policy, then $\mathcal{Q}(s,a) = \mathbb{E}[\mathcal{Z}(s,a)]$. Here, $\mathcal{Z}$ represents the distribution of future rewards, which is no longer a scalar quantity like *Q*-values. Then we obtain the distributional version of Bellman equation as follows: $\mathcal{Z}(s,a) = r + \gamma\mathcal{Z}(s',a')$. Although the proposed distributional deep Q-learning is demonstrated to

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:35:20 UTC from IEEE Xplore. Restrictions apply.

outperform the conventional DQL [21] on many Atari 2600 Games (45 out of 57 games), its performance relies much on the distribution function $\mathcal{Z}$. If $\mathcal{Z}$ is well defined, the performance of distributional deep $Q$-learning is much more significant than that of the DQL. Otherwise, its performance is even worse than that of the DQL.

*6) Deep Q-Learning With Noisy Nets:* In [34], the authors introduce Noisy Net, a type of neural network whose bias and weights are iteratively perturbed during training by a parametric function of the noise. This network basically adds the Gaussian noise to the last (fully-connected) layers of the network. The parameters of this noise can be adjusted by the model during training, which allows the agent to decide when and in what proportion it wants to introduce the uncertainty to its weights. In particular, to implement the noisy network, we first replace the $\epsilon$-greedy policy by a randomized action-value function. Then, the fully connected layers of the value network are parameterized as a noisy network, where the parameters are drawn from the noisy network parameter distribution after every replay step. For replay, the current noisy network parameter sample is held fixed across the batch. Since the DQL takes one step of optimization for every action step, the noisy network parameters are re-sampled before every action.

Through experimental results, the authors demonstrate that by adding the Gaussian noise layer to the DNN, the performance of conventional DQL [21], dueling DQL [30], and asynchronous DQL [31] can be significantly improved for a wide range of Atari games. However, the impact of noise to the performance of the deep DQL algorithms is still under debating in the literature, and thus analysis on the impact of noise layer requires further investigations.

*7) Rainbow Deep Q-Learning:* In [35], the authors propose a solution which integrates all advantages of seven aforementioned solutions (including DQL) into a single learning agent, called Rainbow DQL. In particular, this algorithm first defines the loss function based on the asynchronous multi-step and distributional DQL. Then, the authors combine the multi-step distributional loss with double $Q$-learning by using the greedy action in $s_{t+n}$ selected according to the $Q$-network as the bootstrap action $a^*_{t+n}$, and evaluate the action by using the target network.

In standard proportional prioritized replay [29] technique, the absolute TD-error is used to prioritize the transitions. Here, TD-error at a time slot is the error in the estimate made at the time slot. However, in the proposed Rainbow DQL algorithm, all distributional Rainbow variants prioritize transitions by the Kullbeck-Leibler (KL) loss because this loss may be more robust to noisy stochastic environment. Alternatively, the dueling architecture of DNNs is presented in [30]. Finally, the Noisy Net layer [35] is used to replace all linear layers in order to reduce the number of independent noise variables. Through simulation, the authors show that this is the most advanced technique which outperforms almost all current DQL algorithms in the literature over 57 Atari 2600 games.

In Table III, we summarize the DQL algorithms and their performance under the parameter settings used in [35]. As observed in Table III, all of the DQL algorithms have been developed by Google DeepMind based on the original work in [21]. So far, through experimental results on Atari 2600 games, the Rainbow DQL presents very impressive results over all other DQL algorithms. However, more experiments need to be further conducted in different domains to confirm the real efficiency of the Rainbow DQL algorithm.

### F. Deep Q-Learning for Extensions of MDPs

*1) Deep Deterministic Policy Gradient Q-Learning for Continuous Action:* Although DQL algorithm can solve problems with high-dimensional state spaces, it can only handle discrete and low-dimensional action spaces. However, systems in many applications have continuous, i.e., real values, and high dimensional action spaces. The DQL algorithms cannot be straightforwardly applied to continuous actions since they rely on choosing the best action that maximizes the $Q$-value function. In particular, a full search in a continuous action space to find the optimal action is often infeasible.

In [36], the authors introduce a model-free off-policy actor-critic algorithm using deep function approximators that can learn policies in high-dimensional, continuous action spaces. The key idea is based on the deterministic policy gradient (DPG) algorithm proposed in [37]. In particular, the DPG algorithm maintains a parameterized actor function $\mu(s;\boldsymbol{\theta}^{\mu})$ with parameter vector $\boldsymbol{\theta}$ which specifies the current policy by deterministically mapping states to a specific action. The critic $Q(s, a)$ is learned by using the Bellman equation as in $Q$-learning. The actor is updated by applying the chain rule to the expected return from the start distribution with respect to the actor parameters.

Based on this update rule, the authors then introduce Deep DPG (DDPG) algorithm which can learn competitive policies by using low-dimensional observations, e.g., cartesian coordinates or joint angles, under the same hyper-parameters and network structure. The algorithm makes a copy of the actor and critic networks $Q'(s, a;\boldsymbol{\theta}^{Q'})$ and $\mu'(s;\boldsymbol{\theta}^{\mu'})$, respectively, to calculate the target values. The weights of these target networks are then updated with slowly tracking on the learned networks, i.e., $\boldsymbol{\theta'} \rightarrow \tau\boldsymbol{\theta} + (1-\tau)\boldsymbol{\theta'}$ with $\tau \ll 1$. This means that the target values are constrained to change slowly, greatly improving the stability of learning. Note that the major challenge of learning in continuous action spaces is exploration. Therefore, in the proposed algorithm, the exploration policy $\mu'$ is constructed by adding noise sampled from a noise process $\mathcal{N}$ to the actor policy.

*2) Deep Recurrent Q-Learning for POMDPs:* To tackle problems with partially observable environments by deep reinforcement learning, the authors in [38] propose a framework called Deep Recurrent Q-Learning (DRQN) in which an LSTM layer was used to replace the first post-convolutional fully-connected layer of the conventional DQN. The recurrent structure is able to integrate an arbitrarily long history to better estimate the current state instead of utilizing a fixed-length history as in DQNs. Thus, DRQNs estimate the function $\mathcal{Q}(o_t, h_{t-1};\boldsymbol{\theta})$ instead of $\mathcal{Q}(s_t, a_t);\boldsymbol{\theta})$, where $\boldsymbol{\theta}$ denotes the parameters of entire network, $h_{t-1}$ denotes the output of the LSTM layer at the previous step, i.e., $h_t = LSTM(h_{t-1}, o_t)$. DRQN matches DQN's performance

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:35:20 UTC from IEEE Xplore. Restrictions apply.

TABLE III
PERFORMANCE COMPARISON AMONG DQL ALGORITHMS BY GOOGLE DEEPMIND

| DQL algorithms | Key features | Pros | Cons | Apps |
|---|---|---|---|---|
| DQL [21] | Utilize the DNN to train Q-value function | Fast convergence and easy to implement | Over estimation on action values | Suitable to implement on MDPs with small number of actions |
| DDQL [26] | Use 2 Q-value functions to simultaneously select and evaluate action values | Easy to implement and faster convergence than DQL | Without considering specially features of MDPs | Applicable to almost MDPs |
| Prioritized DDQL [29] | Prioritize experiences in replay memory | Faster convergence than DQL and DDQL | Require information about important experiences in replay memory | Especially effective for MDPs with prioritized experiences |
| Dueling DDQL [30] | Using 2 DNNs to simultaneously estimate the action and state value functions | Much faster than DQL, DDQL and prioritized DDQL | High complexity and less efficient on MDPs with small action and state spaces | Especially effective to deal with MDPs with large action and state spaces |
| Asynchronous DQL [31] | Using multiple agents to train the DNN in parallel | Learning speed is extremely faster by using multiple agents for training | High complexity with intensive requirements on hardware devices for training | Especially effective for MDPs with very large state and action spaces |
| Distributional DQL [33] | Use a distribution function (instead of expectation) to update Q-value function | More accuracy in evaluating Q-value function | Need to know the distribution of reward function over state and action spaces | Suitable to implement on MDPs with available distribution reward function |
| Noisy Nets DQL [34] | Adding the Gaussian noise layer to the DNN for training | Improve efficiency in exploring the environment | Efficiency of adding Gaussian noise layer is still debating | Especially effective for MDPs with very large state and action spaces |
| Rainbow [35] | Combine features of all aforementioned algorithms | Inherit all advantages of aforementioned algorithms | Extremely high complexity with many requirements of MDPs in advance | Only suitable for MDPs with large state/action spaces and some properties known in advance |

on standard MDP problems and outperforms DQN in partially observable domains. Regarding the training process, DRQN only considers the convolutional features of the observation history instead of explicitly incorporating the actions. Through the experiments, the authors demonstrate that DRQN is capable of handling partial observability, and recurrency confers benefits when the quality of observations changes during evaluation time.

*3) Deep SARSA Learning:* In [39], the authors introduce a DQL technique based on SARSA learning to help the agent determine optimal policies in an online fashion. In this algorithm, given the current state $s$, the CNN is used to obtain the current state-action value $\mathcal{Q}(s, a)$. Then, the current action $a$ is selected by the $\epsilon$-greedy algorithm. After that, the immediate reward $r$ and the next state $s'$ can be observed. To estimate the current $\mathcal{Q}(s, a)$, the next state-action value $\mathcal{Q}(s', a')$ is obtained. Here, when the next state $s'$ is used as the input of the CNN, $\mathcal{Q}(s', a')$ can be obtained as the output. Then, a label vector related to $\mathcal{Q}(s, a)$ is defined as $\mathcal{Q}(s', a')$ which represents the target vector. The two vectors only have one different component, i.e., $r + \gamma\mathcal{Q}(s', a') \rightarrow \mathcal{Q}(s, a)$. It should be noted that during the training phase, the next action $a'$ for estimating the current state-action value is never greedy. On the contrary, there is a small probability that a random action is chosen for exploration.

*4) Deep Q-Learning for Markov Games:* In [40], the authors introduce the general notion of sequential prisoner's dilemma (SPD) to model real world prisoner's dilemma (PD) problems. Since SPD is more complicated than PD, existing approaches addressing learning in matrix PD games cannot be directly applied in SPD. Thus, the authors propose a multi-agent DRL approach for mutual cooperation in SDP games. The deep multi-agent reinforcement learning towards mutual cooperation consists of two phases, i.e., offline and online phases. The offline phase generates policies with varying cooperation degrees. Since the number of policies with different cooperation degrees is infinite, it is computationally infeasible to train all the policies from scratch. To address this issue, the algorithm first trains representative policies using actor-critic until it converges, i.e., cooperation and defection baseline policy. Second, the algorithm synthesizes the full range of policies from the above baseline policies. Another task is to detect effectively the cooperation degree of the opponent. The algorithm divides this task into two steps. First, the algorithm trains an LSTM-based cooperation degree detection network offline, which will be then used for real-time detection during the

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:35:20 UTC from IEEE Xplore. Restrictions apply.

online phase. In the online phase, the agent plays against the opponents by reciprocating with a policy of a slightly higher cooperation degree than that of the opponent. On one hand, intuitively the algorithm is cooperation-oriented and seeks for mutual cooperation whenever possible. On the other hand, the algorithm is also robust against selfish exploitation and resorts to defection strategy to avoid being exploited whenever necessary.

Unlike [40] which considers a repeated normal form game with complete information, in [41], the authors introduce an application of DRL for extensive form games with imperfect information. In particular, the authors in [41] introduce Neural Fictitious Self-Play (NFSP), a DRL method for learning approximate Nash equilibria of imperfect-information games. NFSP combines FSP with neural network function approximation. An NFSP agent has two neural networks. The first network is trained by reinforcement learning from memorized experience of play against fellow agents. This network learns an approximate best response to the historical behaviour of other agents. The second network is trained by supervised learning from memorized experience of the agent's own behaviour. This network learns a model that averages over the agent's own historical strategies. The agent behaves according to a mixture of its average strategy and best response strategy.

In the NSFP, all players of the game are controlled by separate NFSP agents that learn from simultaneous play against each other, i.e., self-play. An NFSP agent interacts with its fellow agents and memorizes its experience of game transitions and its own best response behaviour in two memories, $\mathcal{M}_{RL}$ and $\mathcal{M}_{SL}$. NFSP treats these memories as two distinct datasets suitable for DRL and supervised classification, respectively. The agent trains a neural network, $Q(s, a; \boldsymbol{\theta}^Q)$, to predict action values from data in $\mathcal{M}_{RL}$ using off-policy reinforcement learning. The resulting network defines the agent's approximate best response strategy, $\beta = \epsilon$-greedy($Q$), which selects a random action with probability $\epsilon$ and otherwise chooses the action that maximizes the predicted action values. The agent trains a separate neural network $\Pi(s, a; \boldsymbol{\theta}^{\Pi})$ to imitate its own past best response behavior by using supervised classification on the data in $\mathcal{M}_{SL}$. NFSP also makes use of two technical innovations in order to ensure the stability of the resulting algorithm as well as to enable simultaneous self-play learning. Through experimental results, the authors show that the NFSP can converge to approximate Nash equilibria in a small poker game.

*Summary:* In this section, we have presented the basics of reinforcement learning, deep learning, and DQL. Furthermore, we have discussed various advanced DQL techniques and their extensions. Different DQL techniques can be used to solve different problems in different network scenarios. In the next sections, we review DQL related works for various problems in communications and networking.

## III. Network Access and Rate Control

Modern networks such as IoT become more decentralized and ad-hoc in nature. In such networks, entities such as sensors and mobile users need to make independent decisions, e.g., channel and base station selections, to achieve their own goals, e.g., throughput maximization. However, this is challenging due to the dynamic and the uncertainty of network status. Learning algorithms such as DQL allow to learn and build knowledge about the networks that are used to enable the network entities to make their optimal decisions. In this section, we review the applications of DQL for the following issues.

- *Dynamic spectrum access:* Dynamic spectrum access allows users to locally select channels to maximize their throughput. However, the users may not have full observations of the system, e.g., channel states. Thus, DQL can be used as an effective tool for dynamic spectrum access.
- *Joint user association and spectrum access:* User association is implemented to determine which user to be assigned to which Base Station (BS). The joint user association and spectrum access problems are studied in [42] and [43]. However, the problems are typically combinatorial and non-convex which require nearly complete and accurate network information to obtain the optimal strategy. DQL is able to provide distributed solutions which can be effectively used for the problems.
- *Adaptive rate control:* This refers to bitrate/data rate control in dynamic and unpredictable environments such as Dynamic Adaptive Streaming over HTTP (DASH). Such a system allows clients or users to independently choose video segments with different bitrates to download. The client's objective is to maximize its Quality of Experience (QoE). DQL can be adopted to effectively solve the problem instead of dynamic programming which has high complexity and demands complete information.

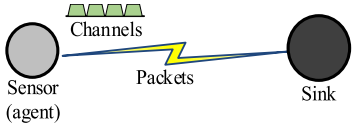



Fig. 8. Channel selection based on DQL in IoT.

### A. Network Access

This section discusses how to use DQL to solve the spectrum access and user association in networks.

*1) Dynamic Spectrum Access:* The authors in [44] propose a dynamic channel access scheme of a sensor based on the DQL for IoT. The model is shown in Fig. 8. At each time slot, the sensor selects one of $M$ channels for transmitting its packet. The channel state is either in low interference, i.e., successful transmission, or in high interference, i.e., transmission failure. Since the sensor only knows the channel state after selecting the channel, the sensor's optimization decision problem can be formulated as a POMDP. In particular, the action of sensor is to select one of $M$ channels. The sensor receives a positive reward "$+1$" if the selected channel is in low interference, and a negative reward "$-1$" otherwise. The objective is to find an optimal policy which maximizes the sensor's expected accumulated discounted reward over time

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:35:20 UTC from IEEE Xplore. Restrictions apply.

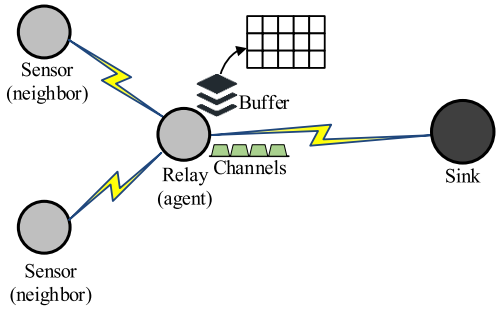



Fig. 9. Joint channel selection and packet forwarding in IoT.

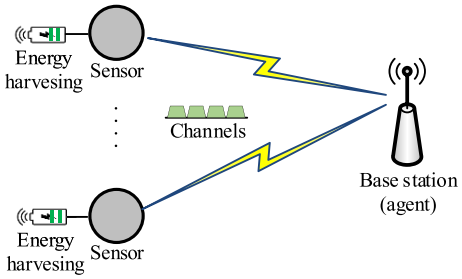



Fig. 10. Channel access in energy harvesting-enabled IoT systems.

slots. In fact, the objective can be obtained by the myopic policy [45]. However, the myopic policy requires the prior knowledge of the system transition matrix which is hard to obtain. DQL allows the sensor to find the optimal policy from its experiences, and thus it can be adopted to solve the sensor's problem. In particular, the DQL uses a DQN[1] with experience replay [46]. The input of the DQN is a state of the sensor which is the combination of actions and observations, i.e., the rewards, in the past time slots. The output includes Q-values corresponding to the actions. To balance the exploration of the current best Q-value with the exploration of the better one, the $\epsilon$-greedy policy is adopted for the action selection mechanism. The simulation results based on the real data from [47] show that the proposed scheme achieves an average reward of 4.4 that is close to the myopic policy [45] with an average reward of 4.5. Note that the myopic policy requires the knowledge of the system transition matrix.

Reference [44] can be considered to be a pioneer work using the DQL for the channel access. However, the DQL keeps following the learned policy over time slots and stops learning a suitable policy. Actual IoT environments are dynamic, and the DQN in the DQL needs to be re-trained. An adaptive DQL scheme is proposed in [48] which evaluates the accumulated reward of the current policy for every period. When the reward is reduced by a given threshold, the DQN is re-trained to find a new good policy. The simulation results [48] show that when the states of the channels change, the adaptive DQL scheme can detect the change and start re-learning to obtain the high reward.

The models in [44] and [48] are constrained to only one sensor. Consider a multi-sensor scenario, the authors in [49] address the joint channel selection and packet forwarding using the DQL. The model is shown in Fig. 9 in which one sensor as a relay forwards packets received from its neighboring sensors to the sink. The sensor is equipped with a buffer to store the received packets. At each time slot, the sensor selects a set of channels for the packet forwarding so as to maximize its utility, i.e., the ratio of the number of transmitted packets to the transmit power. Similar to [44], the sensor's problem can be formulated as an MDP. The action is to select a set of channels, the number of packets transmitted on the channels, and a modulation mode. To avoid packet loss, the state is defined as the combination of the buffer state and channel state. The MDP is then solved by the DQL in which the input is the state and the output is the action selection. The DQL uses the stacked autoencoder to reduce the massive calculation and storage in the Q-learning phase. The sensor's utility function is proved to be bounded which can guarantee the convergence of the algorithm. The analysis shows that the computational complexity of the proposed algorithm is $O(KM(J+1))$that is lower than that of the strategy iteration algorithm [50] with the computational complexity of $O(KM(J+1))(L+1)^K C^M)$, where $K$ is the number of buffers, $L$ is the buffer length, $M$ and $C$ respectively are the numbers of channels and channel states, and $J$ is the number of possible transmission modes. The simulation results show that the proposed scheme significantly improves the system utility compared with the random action selection scheme. In particular, the average system utility of the proposed scheme is 0.63, while that obtained by the random policy is 0.37. However, as the packet arrival rate increases, the system utility of the proposed scheme decreases since the sensor needs to consume more power to transmit all packets.

Consuming more power leads to poor sensor's performance due to its energy constraint, i.e., a shorter IoT system lifetime. The channel access problem in the energy harvesting-enabled IoT system is investigated in [51]. The model consists of one BS and energy harvesting-based sensors (see Fig. 10). The BS as a controller allocates channels to the sensors. However, the uncertainty of ambient energy availability at the sensors may make the channel allocation inefficient. For example, the channel allocated to the sensor with low available energy may not be fully utilized since the sensor cannot communicate later.

Therefore, the BS's problem is to predict the sensors' battery states and select sensors for the channel access so as to maximize the total rate. To solve the BS's problem, the optimal approaches such as the uplink resource allocation scheme [52] can be adopted. However, the scheme requires the BS to have perfect non-causal knowledge of all the random processes. The perfect knowledge may not be available since the sensors are distributed randomly over a geographical area. Thus, the DQL is used to solve the problem of the BS, i.e., the agent. The DQL uses a DQN consisting of two LSTM-based neural network layers. The first layer generates the predicted battery states of sensors, and the second layer uses the predicted states along with Channel State Information (CSI) to determine the channel access policy. The state space consists of (i) channel access scheduling history, (ii) the history of predicted battery information, (iii) the history of the true battery information,

[1]Remind that DQN is the core of the DQL algorithms.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:35:20 UTC from IEEE Xplore. Restrictions apply.

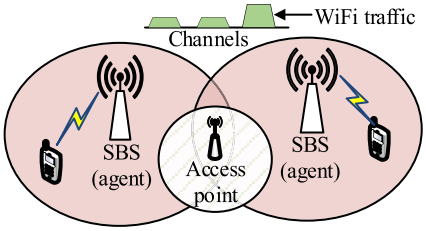



Fig. 11. Unlicensed channel access in LTE networks.

and (iv) the current CSI of the sensors. The action space contains all sets of sensors to be selected for the channel access, and the reward is the difference between the total rate and the prediction error. As shown in the simulation results, the proposed scheme is close to the optimal approach [52] and outperforms the myopic policy [45] in terms of total rate. In particular, the total rates obtained by the proposed scheme, the myopic policy, and the optimal scheme are 6.8, 6.5, and 7.0 kbps, respectively. Moreover, the battery prediction error obtained from the proposed scheme is close to zero.

The above schemes, e.g., [44] and [51], focus on the rate maximization. In IoT systems such as Vehicle-to-Vehicle (V2V) communications, latency also needs to be considered due to the mobility of V2V transmitters/receivers and vital applications in the traffic safety. One of the problems of each V2V transmitter is to select a channel and a transmit power level to maximize its capacity under a latency constraint. Given the decentralized network, a DQN is adopted to make optimal decisions as proposed in [53]. The model consists of V2V transmitters, i.e., agents, which share a set of channels. The actions of each V2V transmitter include choosing channels and transmit power levels. The reward is a function of the V2V transmitter's capacity and latency. The state observed by the V2V transmitter consists of (i) the instantaneous CSI of the corresponding V2V link, (ii) the interference to the V2V link in the previous time slot, (iii) the channels selected by the V2V transmitter' neighbors in the previous time slot, and (iv) the remaining time to meet the latency constraint. The state is also an input of the DQN. The output includes Q-values corresponding to the actions. As shown in the simulation results, by dynamically adjusting the power and channel selection when V2V links are likely to violate the latency constraint, the proposed scheme has more V2V transmitters meeting the latency constraint compared with the random channel allocation.

To reduce spectrum cost, the above IoT systems often use unlicensed channels. However, this may cause the interference to existing networks, e.g., WLANs. The authors in [54] propose to use the DQN to jointly address the dynamic channel access and interference management. The model consists of Small Base Stations (SBSs) which share unlicensed channels in an LTE network (see Fig. 11). At each time slot, the SBS selects one of channels for transmitting its packet. However, there may be WLAN traffics on the selected channel, and thus the SBS accesses the selected channel with a probability. The actions of the SBS include pairs of channel selection and channel access probability. The problem of the SBS is to determine an action vector so as to maximize its total throughput, i.e., its utility, over all channels and time slots. The resource allocation problem can be formulated as a non-cooperative game, and the DQN using LSTM can be adopted to solve the game. The input of the DQN is the history traffic of the SBSs and the WLAN on the channels. The output includes predicted action vectors of the SBSs. The utility function of each SBS is proved to be convex, and thus the DQN-based algorithm converges to a Nash equilibrium of the game. The analysis shows that the computational complexity per time step of the proposed scheme is $O(n_c^2 + n_c n_i + n_c n_0 + n_c)$, where $n_c$, $n_i$, and $n_o$ are the numbers of memory cells, input units, and output units, respectively. The simulation results based on real traffic data from [55] show that the proposed scheme can improve the average throughput up to 28% compared with the standard Q-learning. Moreover, deploying more SBSs in the LTE network does not allow more airtime fraction for the network. This implies that the proposed scheme can avoid causing performance degradation to the WLAN. However, the proposed scheme requires synchronization between the SBSs and the WLAN which is challenging in real networks.

In the same cellular network context, the authors in [27] address the dynamic spectrum access problem for multiple users sharing $K$ channels. At a time slot, the user selects a channel with a certain attempt probability or chooses not to transmit at all. The state is the history of the user's actions and its local observations, and the user's strategy is mapping from the history to an attempt probability. The problem of the user is to find a vector of the strategies, i.e., the policy, over time slots to maximize its expected accumulated discounted data rate of the user.

The above problem is solved by training a DQN. The input of the DQN includes past actions and the corresponding observations. The output includes estimated Q-values of the actions. To avoid the overestimation in the Q-learning, the DDQN [25] is used. Moreover, the dueling DQN [30] is employed to improve the estimated Q-value. The DQN is then offline trained at a base station. Similar to [54], the multichannel random access is modeled as a non-cooperative game. As proved in [27], the game has a subgame perfect equilibrium. Note that some users can keep increasing their attempt probability to increase their rates. This makes the equilibrium point inefficient, and thus the strategy space of the users is restricted to avoid the situation. The simulation results show that the proposed scheme can achieve twice the channel throughput compared with the slotted-Aloha [56]. The reason is that in the proposed scheme, each user only learns from its local observation without an online coordination or carrier sensing. However, the proposed scheme requires the central unit which may raise the message exchanges as the training is frequently updated.

In the aforementioned models, the number of users is fixed in all time slots, and the arrival of new users is not considered. The authors in [57] address the channel allocation to new arrival users in a multibeam satellite system. The multibeam satellite system generates a geographical footprint subdivided into multiple beams which provide services to ground User Terminals (UTs). The system has a set of channels. If there

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:35:20 UTC from IEEE Xplore. Restrictions apply.

exist available channels, the system allocates a channel to the new arrived UT, i.e., the new service is satisfied. Otherwise, the service is blocked. The system's problem is to find a channel allocation decision to minimize the total service blocking probability of the new UT over time slots without causing the interference to the current UTs.

The system's problem can be viewed as a temporal correlated sequential decision-making optimization problem which is effectively solved by the DQN. Here, the satellite system is the agent. The action is an index indicating which channel is allocated to the new arrived UT. The reward is positive when the new service is satisfied and is negative when the service is blocked. The state includes the set of current UTs, the current channel allocation matrix, and the new arrived UT. Note that the state has the spatial correlation feature due to the co-channel interference, and thus it can be represented in an image-like fashion, i.e., an image tensor. Therefore, the DQN adopts the CNN to extract useful features of the state. The simulation results show that the proposed DQN algorithm converges after a certain number of training steps. Also, by allocating available channels to the new arrived UTs, the proposed scheme can improve the system traffic up to 24.4% compared with the fixed channel allocation scheme. However, as the number of current UTs increases, the number of available channels is low or even zero. Therefore, the dynamic channel allocation decisions of the proposed scheme become meaningless, and the performance difference between the two schemes becomes insignificant. For the future work, a joint channel and power allocation algorithm based on the DQL can be investigated.

*2) Joint User Association and Spectrum Access:* Joint user association and spectrum access problems are typically non-convex. To solve the problems, traditional approaches such as linear programming [58] are developed to obtain the optimal solution. However, the approaches require nearly complete and accurate network information that is usually not available. Learning techniques such as Q-learning can be used. However, it is challenging to obtain an optimal solution due to the large state and action spaces of the joint optimization problems. By combining DNN with Q-learning, DQL is effectively used to solve the joint optimization problems as proposed in [28] and [59].

The authors in [28] consider a HetNet which consists of multiple users and BSs including macro base stations and femto base stations (see Fig. 12). The BSs share a set of orthogonal channels, and the users are randomly located in the network. The problem of each user is to select one BS and a channel to maximize its data rate while guaranteeing that the Signal-to-Interference-plus-Noise Ratio (SINR) of the user is higher than a minimum Qualtiy of Service (QoS) requirement. The DQL is adopted to solve the problem in which each user is an agent, and its state is a vector including QoS states of all users, i.e., the global state. Here, the QoS state of the user refers to whether its SINR exceeds the minimum QoS requirement or not. At each time slot, the user takes an action. If the QoS is satisfied, the user receives utility as its immediate reward. Otherwise, it receives a negative reward, i.e., an action selection cost. Note that the cumulative reward of one

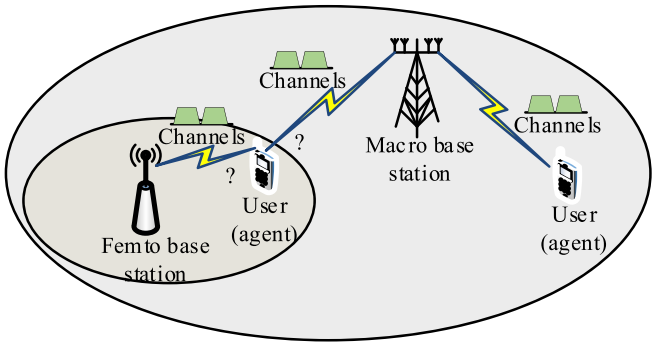



Fig. 12. Joint user association and channel selection based on DQL in HetNets.

user depends on actions of other users, then the user's problem can be defined as an MDP. Similar to [27], the DDQN and the dueling DQN are used to learn the optimal policy, i.e., the joint BS and channel selections, for the user to maximize its cumulative reward. The simulation results from [28] show that the proposed scheme outperforms the Q-learning implemented in [19] in terms of convergence speed and system capacity. The simulation comparisons imply that the DQN can be effectively used to solve the complex problems, e.g., joint optimization problems, in the large-scale systems such as HetNets and IoT.

The scheme proposed in [28] is considered to be the first work using the DQL for the joint user association and spectrum access problem. Inspired by this work, the authors in [59] propose to use the DQL for a joint user association, spectrum access, and content caching problem. The network model is an LTE network which consists of UAVs serving ground users. The UAVs are equipped with storage units and can act as cached-enabled LTE-BSs. The UAVs are able to access both licensed and unlicensed bands in the network. The UAVs are controlled by a cloud-based server, and the transmissions from the cloud to the UAVs are implemented by using the licensed cellular band. The problem of each UAV is to determine (i) its optimal user association, (ii) the bandwidth allocation indicators on the licensed band, (iii) the time slot indicators on the unlicensed band, and (iv) a set of popular contents that the users can request to maximize the number of users with stable queue, i.e., users satisfied with content transmission delay.

The UAV's problem is combinatorial and non-convex, and the DQL can be used to solve it. The UAVs do not know the users' content requests, and thus the Liquid State Machine approach (LSM) [60] is adopted to predict the content request distribution of the users and to perform resource allocation. In particular, predicting the content request distribution is implemented at the cloud based on an LSM-based prediction algorithm. Then, given the request distributions, each UAV as an agent uses an LSM-based learning algorithm to find its optimal users association. Specifically, the input of the LSM-based learning algorithm consists of actions, i.e., UAV-user association schemes, that other UAVs take, and the output includes the expected numbers of users with stable queues corresponding to actions that the UAV can take. After the user association is done, the optimal content caching is determined based on the results of [61, Th. 2], and the optimal spectrum

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:35:20 UTC from IEEE Xplore. Restrictions apply.

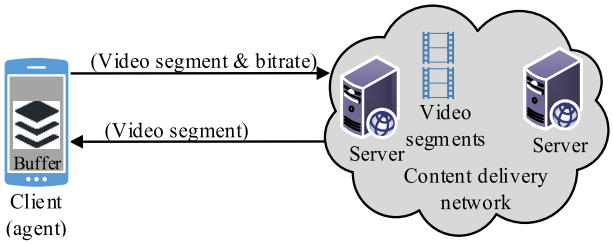



Fig. 13. A dynamic adaptive streaming system based on HTTP standard.

allocation is done by using linear programming. Based on the Gordon's Theorem [62], the proposed DQL is proved to converge with probability one. The simulation results using the content request data from [63] show that the proposed DQL can converge within 400 iterations. Compared with the Q-learning, the proposed DQN improves the convergence time up to 33%. Moreover, the proposed DQL significantly improves the number of users with stable queues up to 50% compared with the Q-learning without cache. In fact, energy efficiency is also important for the UAVs, and thus applying the DQL for a joint user association, spectrum access, and power allocation problem needs to be investigated.

### B. Adaptive Rate Control

Dynamic Adaptive Streaming over HTTP (DASH) becomes the dominant standard for video streaming [64]. DASH is able to leverage existing content delivery network infrastructure and is compatible with a multitude of client-side applications. A general DASH system is shown in Fig. 13 in which the videos are stored in servers as multiple segments, i.e., chunks. Each segment is encoded at different compression levels to generate *representations* with different bitrates, i.e., different video visual quality. At each time slot, the client chooses a representation, i.e., a segment with a certain bitrate, to download. The client's problem is to find an optimal policy which maximizes its QoE such as maximizing average bitrate and minimizing rebuffering, i.e., the time which the video playout freezes.

As presented in [65], the above problem can be modeled as an MDP in which the agent is the client and the action is choosing a representation to download. To maximize the QoE, the reward is defined as a function of (i) visual quality of the video, (ii) video quality stability, (iii) rebuffering event, and (iv) buffer state. Given the reward formulation, the state of the client should include (i) the video quality of the last downloaded segment, (ii) the current buffer state, (iii) the rebuffering time, and (iv) the channel capacities experienced during downloading of segments in the past time slots. The MDP can be solved by using dynamic programming, but the computational complexity rapidly becomes unmanageable as the size of the problem increases. Thus, the authors in [65] adopt the DQL to solve the problem. Similar to [51], the LSTM networks are used in which the input is the state of the client, and the output includes Q-values corresponding to the client's possible actions. To improve the performance of the standard LSTM, peephole connections are added into the LSTM networks. The simulation results based on the dataset from [66] show that the proposed DQL algorithm can converge much faster than Q-learning. In particular, the DQL algorithm converges in around 3 episodes, while the Q-learning converges in around 180 episodes. The fast convergence suggests that DQL can provide efficient solution to problems in real-time applications. Moreover, the proposed DQL improves the video quality and reduces the rebuffering since it is able to dynamically manage the buffer by considering the buffer state and channel capacity.

The network model and the optimization problem in [65] are also found in [67]. However, different from [65], the authors in [67] adopt the Asynchronous Advantage Actor-Critic (A3C) method [31] for the DQL to further enhance and speed up the training. As presented in Section II-F1, A3C includes two neural networks, namely, *actor network* and *critic network*. The actor network is to choose bitrates for the client, and the critic network helps train the actor network. For the actor network, the input is the client's state, and the output is a policy, i.e., a probability distribution over possible actions given states that the client can take. Here, the action is choosing the next representation, i.e., the next segment with a certain bitrate, to download. For the critic network, the input is the client's state, and the output is the expected total reward when following the policy obtained from the actor network. The simulation results based on the mobile dataset from [68] show that the proposed DQL can improve the average QoE up to 25% compared with the bitrate control scheme [69]. Also, by having sufficient buffer to handle the network's throughput fluctuations, the proposed DQL reduces the rebuffering around 32.8% compared with the baseline scheme.

In practice, the DQL algorithm proposed in [67] can be easily deployed in a multi-client network since A3C is able to support parallel training for multiple agents. Accordingly, each client, i.e., an agent, is configured to observe its reward. Then, the client sends a tuple including its state, action, and reward to a server. The server uses the actor-critic algorithm to update its actor network model. The server then pushes the newest model to the agent. This update process can happen asynchronously among all agents which improves quality and speeds up the training. Although the parallel training scheme may incur a Round-Trip Time (RTT) between the clients and the server, the simulation results in [67] show that the RTT between the clients and the server reduces the average QoE by only 3.5%. The performance degradation is small, and thus the proposed DQL can be implemented in real network systems.

In [65] and [67], the input of the DQL, i.e., the client's state, includes the video quality of the last downloaded video segment. The video segment is raw which may cause "state explosion" to the state space [70]. To reduce the state space and to improve the QoE, the authors in [70] propose to use a video quality prediction network. The prediction network extracts useful features from the raw video segments using CNN and RNN. Then, the output of the prediction network, i.e., the predicted video quality, is used as one of the inputs of the DQL which is proposed in [67]. Simulation results based on the broadband dataset from [71] show that the proposed DQL can improve the average QoE up to 25% compared with the Google Hangout, i.e., a communication platform developed

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:35:20 UTC from IEEE Xplore. Restrictions apply.

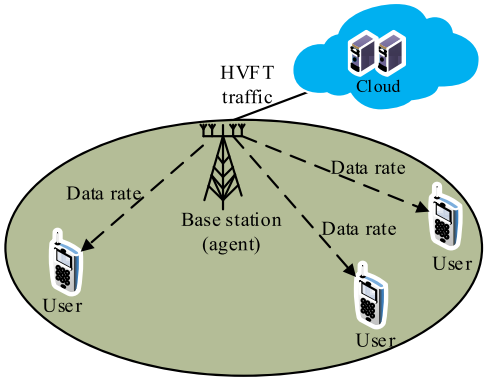



Fig. 14. Data rate control based on DQL in HVFT applications.

by Google. Moreover, the proposed DQL can reduce the average latency of video transmission around 45% due to the small state space. This means that in the scenarios that the state space is large, the CNN should be used to improve the user QoE and the convergence time.

Apart from the DASH systems, the DQL can be effectively used for the rate control in High Volume Flexible Time (HVFT) applications. HVFT applications use cellular networks to deliver IoT traffic as shown in Fig. 14. The HVFT applications have a large volume of traffic, and the traffic scheduling, e.g., data rate control, in the HVFT applications is necessary. One common approach is to assign static priority classes per traffic type, and then traffic scheduling is based on its priority class. However, such an approach does not evolve to accommodate new traffic classes. Thus, learning methods such as DQL should be used to provide adaptive rate control mechanisms as proposed in [72]. The network model is a single cell including one BS as a central controller and multiple mobile users. The problem at the BS is to find a proper policy, i.e., data rate for the users, to maximize the amount of transmitted HVFT traffic while minimizing performance degradation to existing data traffics. It is shown in [72] that the problem can be formulated as an MDP. The agent is the BS, and the state includes the current network state and the useful features extracted from network states in the past time slots. The network state at a time slot includes (i) the congestion metric, i.e., the cell's traffic load, at the time slot, (ii) the total number of network connections, and (iii) the cell efficiency, i.e., the cell quality. The action that the BS takes is a combination of the traffic rate for the users. To achieve the BS' objective, the reward is defined as a function of (i) the sum of HVFT traffic, (ii) traffic loss to existing applications due to the presence of the HVFT traffic, and (iii) the amount of bytes served below desired minimum throughput. The DQL using the actor and critic networks with LSTM is then adopted. By using the real network data collected in Melbourne, the simulation results show that the proposed DQL scheme increases the HVFT traffic up to 2 times compared with the heuristic control scheme. The proposed DQL is thus expected to be applied in modern networks in large-scale cities with a large population growth.

In the aforementioned approaches, the maximum number of objectives is constrained, e.g., to 3 in [73]. The authors in [74] show that the DQL can be used for the rate control to achieve multiple objectives in complex communication systems. The network model is a future space communication system which is expected to operate in unpredictable environments, e.g., orbital dynamics, atmospheric and space weather, and dynamic channels. In the system, the transmitter needs to be configured with several transmit parameters, e.g., symbol rate and encoding rate, to achieve multiple conflict objectives, e.g., low Bit Error Rate (BER), throughput improvement, power and spectral efficiency. The adaptive coding and modulation schemes, i.e., [75], can be used. However, the methods allow to achieve only limited number objectives. Learning algorithms such as the DQL can be thus used. The agent is the transmitter in the system. The action is a combination of (i) symbol rate, (ii) energy per symbol, (iii) modulation mode, (iv) number of bits per symbol, and (v) encoding rate. The objective is to maximize the system performance. Thus, the reward is defined as a *fitness* function of performance parameters including (i) BER estimated at the receiver, (ii) throughput, (iii) spectral efficiency, (iv) power consumption, and (v) transmit power efficiency. The state is the system performance measured by the transmitter, and thus the state is the reward. To achieve multiple objectives, the DQL is implemented by using a set of multiple neural networks in parallel. The input of the DQL is the current state and the channel conditions, and the output is the predicted action. The neural networks are trained by using the Levenberg-Marquardt backpropagation algorithm [76]. The simulation results show that the proposed DQL can achieve the fitness score, i.e., the weighted sum of different objectives, close to the ideal, i.e., the exhaustive search approach. This implies that the DQL is able to select near-optimal actions and learn the relationship between rewards and actions given dynamic channel conditions.

*Summary:* This section reviews applications of DQL for the dynamic network access and adaptive rate control. The reviewed approaches are summarized along with the references in Table IV. We observe that the problems are mostly modeled as an MDP. Moreover, DQL approaches for the IoT and DASH systems receive more attentions than other networks. Future networks, e.g., 5G networks, involve multiple network entities with multiple conflicting objectives, e.g., provider's revenue versus users' utility maximization. This poses a number of challenges to the traditional resource management mechanisms that deserve in-depth investigation. In the next section, we review the adoption of DQL for the emerging services, i.e., offloading and caching.

## IV. Caching and Offloading

As one of the key features of information-centric networking, in-network caching can efficiently reduce duplicated content transmissions. The studies on wireless caching has shown that access delays, energy consumption, and the total amount of traffic can be reduced significantly by caching contents in wireless devices. Big data analytics [77] also demonstrate that with limited cache size, proactive caching at network edge nodes can achieve 100% user satisfaction while offloading 98% of the backhaul traffic. Joint content

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:35:20 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
A SUMMARY OF APPROACHES USING DQL FOR NETWORK ACCESS AND ADAPTIVE RATE CONTROL

| ISSUES | REF. | MODEL | LEARNING ALGORITHMS | AGENT | STATES | ACTIONS | REWARDS | NETWORKS |
|---|---|---|---|---|---|---|---|---|
| Network access | [44] | POMDP | DQN using FNN | Sensor | Past channel selections and observations | Channel selection | Score +1 or -1 | IoT |
| | [49] | MDP | DQN using FNN | Sensor | Current buffer state and channel state | Channel, packets, and modulation mode selection | Ratio of number of transmitted packets to transmit power | IoT |
| | [51] | MDP | DQN with LSTM | Base station | Channel access history, predicted and true battery information history, and current CSI | Sensor selection for channel access | Total rate and prediction error | IoT |
| | [53] | MDP | DQN with LSTM | V2V transmitter | Current CSI, past interference, past channel selections, and remaining time to meet the latency constraints | Channel and transmit power selection | Capacity and latency | IoT |
| | [54] | Game | DQN with LSTM | Small base station | Traffic history of small base stations and the WLAN | Channel selection and channel access probability | Throughput | LTE network |
| | [27] | Game | DDQN and dueling DQN | Mobile user | Past channel selections and observations | Channel selection | Data rate | CRN |
| | [57] | MDP | DQN with CNN | Satellite system | Current user terminals, channel allocation matrix, and the new arrival user | Channel selection | Score +1 or -1 | Satellite system |
| | [28] | MDP | DDQN and dueling DQN | Mobile user | QoS states | Base station and channel selection | Utility | HetNet |
| | [59] | Game | DQN with LSM | UAV | Content request distribution | Base station selection | Users with stable queues | LTE network |
| Rate control | [65] | MDP | DQN with LSTM and peephole connections | Client | Last segment quality, current buffer state, rebuffering time, and channel capacities | Bitrate selection for segment | Video quality, rebuffering even, and buffer state | DASH system |
| | [67] | MDP | DQN with A3C | Client | Last segment quality, current buffer state, rebuffering time, and channel capacities | Bitrate selection for segment | Video quality, rebuffering even, and buffer state | DASH system |
| | [70] | MDP | DQN with CNN and RNN | Client | Predicted video quality, current buffer state, rebuffering time, and channel capacities | Bitrate selection for segment | Video quality, rebuffering even, and buffer state | DASH system |
| | [72] | MDP | DQN using A3C and LSTM | Base station | Congestion metric, current network connections, and cell efficiency | Traffic rate decisions for mobile users | HVFT traffic, traffic loss to existing applications, and the amount of served bytes | HVFT application |
| | [74] | MDP | DQN using FNN | Base station | Measurements of BER, throughput, spectral efficiency, power consumption, and transmit power efficiency | Symbol rate, energy per symbol, modulation mode, number of bits per symbol, and encoding rate | Same as the state | Space communication system |

caching and offloading can address the gap between the mobile users' large data demands and the limited capacities in data storage and processing. This motivates the study on Mobile Edge Computing (MEC). By deploying both computational resources and caching capabilities close to end users, MEC significantly improves energy efficiency and QoS for applications that require intensive computations and low latency. A unified study on caching, offloading, networking, and transmission control in MEC scenarios involves very complicated system analysis because of strong couplings among mobile users with heterogeneities in application demand, QoS provisioning, mobility pattern, radio access interface, and wireless resources. A learning-based and model-free approach becomes a promising candidate to manage huge state space and optimization variables, especially by using DNNs. In this section, we review the modeling and optimization of caching and offloading policies in wireless networks by leveraging the DRL framework.

### A. Wireless Proactive Caching

Wireless proactive caching has attracted great attentions from both academia and industry. Statistically, a few popular contents are usually requested by many users during a short time span, which accounts for most of the traffic load. Therefore, proactively caching popular contents can avoid the heavy traffic burden of the backhaul links. In particular, this technique aims at pre-caching the contents from the remote content servers at the edge devices or BSs that are close to the end users. If the requested contents are already cached locally, the BS can directly serve the end users with small delay. Otherwise, the BS requests these contents from the original content server and updates the local cache based on the caching policy, which is one of the main design problem for wireless proactive caching.

*1) QoS-Aware Caching:* Content popularity is the key factor used to solve the content caching problem. With a large number of contents and their time-varying popularities, DQL

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:35:20 UTC from IEEE Xplore. Restrictions apply.

is an attractive strategy to tackle this problem with high-dimensional state and action spaces. The authors in [77] present a DQL scheme to improve the caching performance. The system model consists of a single BS with a fixed cache size. For each request, the BS as an agent makes a decision on whether or not to store the currently requested content in the cache. If the new content is kept, the BS determines which local content will be replaced. The state is the feature space of the cached contents and the currently requested content. The feature space consists of the total number of requests for each content in a specific short-, medium-, and long-term. There are two types of actions: (i) to find a pair of contents and exchange the cache states of the two contents and (ii) to keep the cache states of the contents unchanged. The aim of the BS is to maximize the long-term cache hit rate, i.e., reward.

The DQL scheme in [77] trains the policy by using the DDPG method [78] and employs Wolpertinger architecture [79] to reduce the size of the action space and avoid missing an optimal policy. The Wolpertinger architecture consists of three main parts: an actor network, K-Nearest Neighbors (K-NN), and a critic network. The actor network is to avoid a large action space. The critic network is to correct the decision made by the actor network. The DDPG method is applied to update both critic and actor networks. K-NN can help to explore a set of actions to avoid poor decisions. The actor and critic networks are then implemented by using FNNs. The simulation results show that the proposed DQL scheme outperforms the first-in first-out scheme in terms of long-term cache hit rate. In particular, the cache hit rate obtained by the DQL scheme is 0.5, while that obtained by the first-in first-out scheme is 0.4. The performance comparisons demonstrate that the proposed DQL scheme can achieve competitive cache hit rates while effectively reducing the run-time. This makes the proposed framework efficient and suitable for handling large-scale data.

Maximizing the long-term cache hit rate in [77] implies that the cache stores the most popular contents. In a dynamic environment, contents stored in a cache have to be replaced according to the users' dynamic requests. An optimization of the placement or replacement of cached contents is studied in [80] by a deep learning method. The optimization algorithm is trained by a DNN in advance and then used for real-time caching or scheduling with minimum delay. The authors in [81] propose an optimal caching policy to learn the cache expiration times, i.e., Time-To-Live (TTL), for dynamically changing requests in content delivery networks. The system includes a cloud database server and multiple mobile devices that can issue queries and update entries in a single database. The query results can be cached for a specified time interval at server-controlled caches. All cached queries will become invalid if one of the cached records has been updated. A large TTL will strain cache capacities while a small TTL increases latencies significantly if the database server is physically remote.

Unlike the DDPG approach used in [77], the authors in [81] propose to utilize Normalized Advantage Functions (NAFs) for continuous DQL scheme to learn optimal cache expiration duration. The key problem in continuous DQL is to select an action maximizing the Q-function, while avoiding performing a costly numerical optimization at each step. The use of NAFs obviates a second actor network that needs to be trained separately. Instead, a single neural network is used to output both a value function and an advantage term. The DQL agent at the cloud database uses an encoding of a query itself and the query miss rates as the system states, which allows for an easier generalization. The system reward is linearly proportional to the current load, i.e., the number of cached queries divided by the total capacity. This reward function can encourage longer TTLs when fewer queries are cached, and shorter TTLs when the load is close to the system capacity. Considering incomplete measurements for rewards and next-states at run-time, the authors introduce the Delayed Experience Injection (DEI) approach that allows the DQL agent to keep track of incomplete transitions when measurements are not immediately available. The authors evaluate the learning algorithm by Yahoo! cloud serving benchmark with customized Web workloads [82]. The simulation results verify that the learning approach based on NAFs and DEI outperforms a statistical estimator.

*2) Joint Caching and Transmission Control:* The caching policies determine where to store and retrieve the requested content efficiently, e.g., by learning the contents' popularities [77] and cache expiration time [81]. Another important aspect of caching design is the transmission control of the content delivery from caches to end users, especially for wireless systems with dynamic channel conditions. To avoid mutual interference in multi-user wireless networks, the transmission control decides which cached contents can be transmitted concurrently as well as the most appropriate control parameters, e.g., transmit power, precoding, data rate, and channel allocation. Hence, the joint design of caching and transmission control is required to enable efficient content delivery in multi-user wireless networks.

Recently, some approaches, e.g., [83], are proposed for the joint caching and interference alignment in wireless systems. However, most of the approaches assume that the channel state information is invariant that may not be the case in dynamic wireless systems. The authors in [84]–[86] propose to use the DQL framework to address the joint caching and interference alignment to tackle mutual interference in multi-user wireless networks. The authors consider an MIMO system with limited backhaul capacity and the caches at the transmitter. The precoding design for interference alignment requires the global CSI at each transmitter. A central scheduler is responsible for collecting CSI and cache status from each user via the backhaul, scheduling the users' transmission, and optimizing the resource allocation. By enabling content caching at individual transmitters, we can decrease the demand for data transfer and thus save more backhaul capacity for real-time CSI update and sharing. Using the DQL-based approach at the central scheduler can reduce the explicit demand for CSI and the computational complexity in matrix optimization, especially with time-varying channel conditions. The DQL agent implements the DNN to approximate the Q-function with experience replay in training. To make the learning process more stable, the target Q-network parameter is updated

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:35:20 UTC from IEEE Xplore. Restrictions apply.

by the Q-network for every a few time instants. The collected information is assembled into a system state and sent to the DQL agent, which feeds back an optimal action for the current time instant. The action indicates which users to be active, and the resource allocation among active users. The system reward represents the total throughput of multiple users. An extended work of [84] and [85] with a similar DQL framework is presented in [86], in which a CNN-based DQN is adopted and evaluated in a more practical conditions with imperfect or delayed CSI. Simulation results show that the performance of the MIMO system is significantly improved compared with the baseline scheme [83] in terms of the total throughput and energy efficiency. In particular, at $SNR = 15$ dB, the total rate obtained by the proposed DQN scheme is 240 Mbps, while that obtained by the baseline scheme is 200 Mbps.

Interference management is an important requirement of wireless systems. The application-related QoS or user experience is also an essential metric. Different from [84]–[86], the authors in [87] propose a DQL approach to maximize Quality of Experience (QoE) of IoT devices by jointly optimizing the cache allocation and transmission rate in content-centric wireless networks. The system state is specified by the nodes' caching conditions, e.g., the service information and cached contents, as well as the transmission rates of the cached contents. The aim of the DQL agent is to minimize continuously the network cost or maximize the QoE. The proposed DQL framework is further enhanced with the use of PER and DDQN. PER replays important transitions more frequently so that DQN can learn from samples more efficiently. The use of DDQN can stabilize the learning by providing two value functions in separated neural networks. This avoids an overestimation of the DQN with the increasing number of actions. These two neural networks are not completely decoupled as the target network is a periodic copy of estimation network. A discrete simulator ccnSim [88] is used to model the caching behavior in various graph structures. The output data trace of the simulator is then imported to MATLAB and used to evaluate the learning algorithm. As shown in the simulation results, the proposed DQL framework can achieve a QoE value of 4 that is double to that of the standard penetration test scheme. Moreover, the computational complexity of the DDQN is $O(\lg n)$ that is lower than that of the standard penetration test scheme with the computational complexity of $O(lsn^3)$, where $n$ is the number of content-centric computing nodes, $s$ is the number of service nodes, and $l$ is the number of transmission rate values.

The QoE can be used to characterize the users' perception of Virtual Reality (VR) services. The authors in [89] address the joint content caching and transmission strategy in a wireless VR network, where UAVs capture videos on live games and transmit them to small-cell BSs servicing the VR users. Millimeter wave (mmWave) downlink backhaul links are used for VR content transmission from the UAVs to BSs. The BSs can also cache the popular contents that may be requested frequently by end users. The joint content caching and transmission problem is formulated as an optimization to maximize the users' reliability, i.e., the probability that the content transmission delay satisfies the instantaneous delay target. The maximization involves the control of transmission format, users' association, the set and format of cached contents. A DQL framework combining the Liquid State Machine (LSM) and Echo State Network (ESN) is proposed for each BS to find the optimal transmission and caching strategies. As a randomly generated spiking neural network [90], LSM can store information about the network environment over time and adjust the users' association policy, cached contents and formats according to the users' content requests. It has been used in [59] to predict the users' content request distribution while having only limited information regarding the network and different users. Conventional LSM uses FNNs as the output function, which demands high complexity in training due to the computation of gradients for all of the neurons. Conversely, the proposed DQL framework uses an ESN as the output function, which uses historical information to find the relationship between the users' reliability, caching, and content transmission. It also has a lower complexity in training and a better memory for network information. Simulation results show that the proposed DQL framework can yield 25.4% gain in terms of users' reliability compared to the baseline Q-learning.

*3) Joint Caching, Networking, and Computation:* Caching and transmission control will become more involved in a HetNet that integrates different communication technologies, e.g., cellular system, device-to-device network, vehicular network, and networked UAVs, to support various application demands. The network heterogeneity raises the problem of complicated system design that needs to address challenging issues such as mutual interference, differentiated QoS provisioning, and resource allocation, hopefully in a unified framework. Obviously this demands a joint optimization far beyond the extent of joint caching and transmission control.

Accordingly, the authors in [91] propose a DQL framework for energy-efficient resource allocation in green wireless networks, jointly considering the couplings among networking, in-network caching and computation. The system consists of a Software-Defined Network (SDN) with multiple virtual networks and mobile users requesting for video on-demand files that require a certain amount of computational resource at either the content server or at local devices. In each virtual network, an authorized user issues a request to download files from a set of available SBSs in its neighborhood area. The wireless channels between each mobile user and the SBSs are characterized as Finite-State Markov Channels (FSMC). The states are the available cache capacity at the SBSs, the channel conditions between mobile users and SBSs, the computational capability of the content servers and mobile users. The DQL agent at each SBS decides an association between each mobile user and SBS, where to perform the computational task, and how to schedule the transmissions of SBSs to deliver the required data. The objective is to minimize the total energy consumption of the system from data caching, wireless transmission, and computation. Simulation results show that the total energy consumption in different testing scenarios is very high at the beginning of the learning process and gradually decreases a stable value when the learning converges. Moreover, the energy consumption of the unified DRL

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:35:20 UTC from IEEE Xplore. Restrictions apply.

framework considering caching, networking, and computing is significantly lower than those of other DRL frameworks that only focus on part of the control variables.

The DQL scheme proposed in [91] has been applied to improve the performance of Vehicular Ad doc NETworks (VANETs) in [92]–[94]. The network model includes multiple BSs, Road Side Units (RSUs), MEC servers, and content servers. All devices are controlled by a mobile virtual network operator. The vehicles request for video contents that can be cached at the BSs or retrieved from remote content servers. The authors in [92] formulate the resource allocation problem as a joint optimization of caching, networking, and computing, e.g., compressing and encoding operations of the video contents. The system states include the CSI from each BS, the computational capability, and cache size of each MEC/content server. The network operator feeds the system state to the FNN-based DQN and gets the optimal policy that determines the resource allocation for each vehicle. To exploit spatial correlations in learning, the authors in [93] enhance Q-learning by using CNNs in DQN. This makes it possible to extract high-level features from raw input data. Two schemes have been introduced in [94] to improve stability and performance of the ordinary DQN method. Firstly, DDQN is designed to avoid over-estimation of Q-value in ordinary DQN. Hence, the action can be decoupled from the target Q-value generation. This makes the training process faster and more reliable. Secondly, the dueling DQN approach is also integrated in the design with the intuition that it is not always necessary to estimate the reward by taking some action. The state-action Q-value in dueling DQN is decomposed into one value function representing the reward in the current state, and the advantage function that measures the relative importance of a certain action compared with other actions. Simulation results show that the proposed DQL scheme outperforms the existing static scheme in terms of total utility. In particular, the total utility obtained by the DQL scheme is 8000, while that obtained by the existing static scheme is 5000.

Considering the huge action space and high complexity with the vehicle's mobility and service delay deadline $T_d$, a multi-time scale DQN framework is proposed in [95] to minimize the system cost by the joint design of communication, caching and computing in VANET. The policy design accounts for limited storage capacities and computational resources at the vehicles and the RSUs. The small timescale DQN is for every time slot and aims to maximize the exact immediate reward. Additionally, the large timescale DQN is designed for every $T_d$ time slots within the service delay deadline, and used to estimate the reward considering the vehicle's mobility in a large timescale. Simulation results show that the proposed framework can reduce the cost up to 30% compared with the random resource allocation scheme.

The aforementioned DQL framework for VANETs, e.g., [92]–[94], has also been generalized to smart city applications in [96], which necessitates dynamic orchestration of networking, caching, and computation to meet different servicing requirements. Through Network Function Virtualization (NFV) [97], the physical wireless network in smart cities can be divided logically into several virtual ones by the network operator, which is responsible for network slicing and resource scheduling, as well as allocation of caching and computing capacities. The use cases in smart cities are presented in [98], [99], which apply the generalized DQL framework to improve the security and efficiency for trust-based data exchange, sharing, and delivery in mobile social networks through the resource allocation and optimization of MEC allocation, caching, and D2D (Device-to-Device) networking.

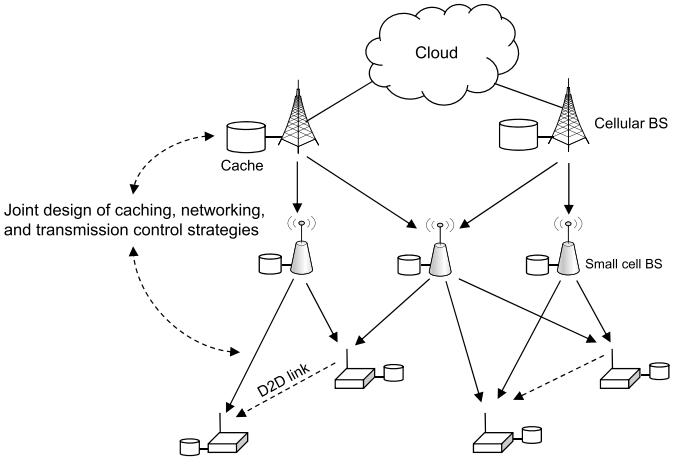



Fig. 15. Joint caching, networking, and transmission control to optimize cache hit rate [77], cache expiration time [81], interference alignment [84]–[86], Quality of Experience [87], [89], energy efficiency [91], resource allocation [92]–[94], traffic latency, or redundancy [96], [98].

### *B. Data and Computation Offloading*

With limited computation, memory and power supplies, IoT devices such as sensors, wearable devices, and handheld devices become the bottleneck to support advanced applications such as interactive online gaming and face recognition. To address such a challenge, IoT devices can offload the computational tasks to nearby MEC servers, integrated with the BSs, Access Points (APs), and even neighboring Mobile Users (MUs). As a result, data and computation offloading can potentially reduce the processing delay, save the battery energy, and even enhance security for computation-intensive IoT applications. However, the critical problem in the computation offloading is to determine the offloading rate, i.e., the amount of computational workload, and choose the MEC server from all available servers. If the chosen MEC server experiences heavy workloads and degraded channel conditions, it may take even longer time for the IoT devices to offload data and receive the results from the MEC server. Hence, the design of an offloading policy has to take into account the time-varying channel conditions, user mobility, energy supply, computation workload and the computational capabilities of different MEC servers. Some optimal offloading approaches, e.g., the dynamic programming based offloading algorithm and heuristic offloading algorithm [100], are proposed. However, the approaches assume that the mobility patterns of the mobile users are given in advance. Without knowing the mobility pattern in advance, the DQL can be

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:35:20 UTC from IEEE Xplore. Restrictions apply.

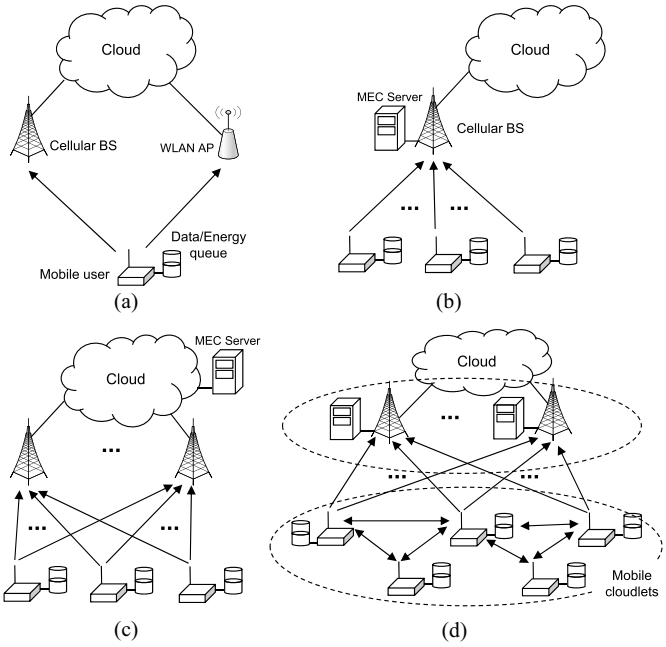



Fig. 16. Data/computation offloading models in cellular networks: (a) Offloading cellular traffic to WLAN [101], (b) Offloading to a single MEC-enabled BS [102], (c) Offloading to one shared MEC server via multiple BSs [103], [104], [108], (d) Offloading to multiple MEC-enabled BSs [112], [113] and mobile cloudlets [114], [115].

used for each mobile user to learn the optimal offloading policy from past experiences such as the approaches proposed in [101] and [102]. The authors in [101] focus on minimizing the mobile user's cost and energy consumption by offloading cellular traffic to WLAN. Each mobile user can either access the cellular network, or the complimentary WLAN as illustrated in Fig. 16(a), but with different monetary costs. The mobile user also has to pay a penalty if the data transmission does not finish before the deadline. The mobile user's data offloading decision can be modeled as an MDP. The system state includes the mobile user's location and the remaining file size of all data flows. The mobile user will choose to transmit data through either WLAN or cellular network, and decide how to allocate channel capacities to concurrent flows. CNNs are employed in the DQL to predict a continuous value of the mobile user's remaining data. Simulation results reveal that the DQN-based scheme generally outperforms the dynamic programming algorithm for the MDP in terms of cost and energy consumption. In particular, the DQL-based scheme can reduce the energy consumption up to 500 Joules compared with that of the dynamic programming algorithm. The reason is that the DQN can learn from experience while the dynamic programming algorithm cannot obtain the optimal policy with incorrect transition probability.

The allocation of limited computational resources at the MEC server is critical for cost and energy minimization. The authors in [102] consider an MEC-enabled cellular system, in which multiple mobile users can offload their computational tasks via wireless channels to one MEC server, co-located with the cellular BS as shown in Fig. 16(b). Each mobile user has a computational-intensive task, characterized by the required computational resources, CPU cycles, and the maximum tolerable delay. The capacity of the MEC server is limited to accommodate all mobile users' task loads. The bandwidth sharing between different mobile users' offloading also affects the overall delay performance and energy consumptions. The DQL is used to minimize the cost of delay and power consumptions for all mobile users, by jointly optimizing the offloading decision and computational resource allocation. The system states include the sum of cost of the entire system and the available computational capacity of the MEC server. The action of BS is to determine the resource allocation and offloading decision for each mobile user. To limit the size of action space, a pre-classification step is proposed to check the mobile users' feasible set of actions. Simulation results show that the proposed scheme can reduce the sum cost up to 55% compared with the static allocation strategies.

In contrast to [102], multiple BSs in an ultra-dense network is considered in [103] and [104], as shown in Fig. 16(c), with the objective of minimizing the long-term cost of delay in computation offloading. All computational tasks are offloaded to the shared MEC server via different BSs. Besides the allocation of computational resources and transmission control, the offloading policy also has to optimize the association between mobile users and the BSs. With dynamic network conditions, the mobile users' decision-making can be formulated as an MDP. The system states are the channel conditions between the mobile user and the BSs, the states of energy and task queues. The cost function is defined as a weighted sum of the execution delay, the handover delay and the computational task dropping cost. The authors in [104] firstly propose a DDQN-based DQL algorithm to learn the optimal offloading policy without knowing the network dynamics. By leveraging the additive structure of the utility function, the Q-function decomposition combined with the DDQN further leads to a novel online SARSA-based DRL algorithm. Numerical experiments show that the new algorithm achieves a significant improvement in computation offloading performance compared with the baseline policies, e.g., the DQN-based DQL algorithm and some heuristic offloading strategies without learning. The high density of SBSs can relieve the data offloading pressure in peak traffic hours but consume a large amount of energy in off-peak time. Therefore, the authors in [105], [106], and [107] propose a DQL-based strategy for controlling the (de)activation of different SBSs to minimize the energy consumption without compromising the quality of provisioning. In particular, in [105], the on/off decision framework uses a DQL scheme to approximate both the policy and value functions in an actor-critic method. The reward of the DQL agent is defined as a cost function relating to energy consumption, QoS degradation, and the switching cost of SBSs. The DDPG approach is also employed together with an action refinement scheme to expedite the training process. Through extensive numerical simulations, the proposed scheme is shown to greatly outperform other baseline methods in terms of both energy and computational efficiency.

With a similar model to that in [104], computation offloading finds a proper application for cloud-based malware detection in [108]. A review of the threat models and the RL-based solutions for security and privacy protection in mobile offloading and caching are discussed in [109]. With limited energy

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:35:20 UTC from IEEE Xplore. Restrictions apply.

supply, computational resources, and channel capacity, mobile users cannot always update the local malware database and process all application data in time and thus are vulnerable to zero-day attacks [110]. By leveraging the remote MEC server, all mobile users can offload their application data and detection tasks via different BSs to the MEC/security server with larger and more sophisticated malware database, more computational capabilities, and powerful security services. This can be modeled by a dynamic malware detection game in which multiple mobile users interact with each other in resource competition, e.g., the allocation of wireless channel capacities and the computational capabilities of the MEC/security server. A DQL scheme is proposed for each mobile user to learn its offloading data rate to the MEC/security server. The system states include the channel state and the size of application traces. The objective is to optimize the detection accuracy of the security server, which is defined as a concave function in the total amount of malware samples. The Q-value is estimated by using a CNN in the DQL framework. The authors also propose the hotbooting Q-learning technique that provides a better initialization for Q-learning by exploiting the offloading experiences in similar scenarios. It can save exploration time at the initial stage and accelerate the learning speed compared with a standard Q-learning algorithm with all-zero initialization of the Q-value [111]. The proposed DQL scheme not only improves the detection speed and accuracy, but also increases the mobile users' battery life. The simulation results reveal that compared with the hotbooting Q-learning and standard Q-learning schemes, the DQL-based malware detection has faster learning rate, higher accuracy, and lower detection delay. For example, the detection delay of the proposed DQL scheme reduces by 24.6% and 35.3% at time slot 2000, respectively, compared with those of the hotbooting Q-learning and the standard Q-learning schemes.

Multiple MEC servers have been considered in [112], [113], as illustrated in Fig. 16(d). The authors in [112] aim to design optimal offloading policy for IoT devices with energy harvesting capabilities. The system consists of multiple MEC servers, such as BSs and APs, with different capabilities in computation and communications. The IoT devices are equipped with energy storage and energy harvesters. They can execute computational tasks locally and offload the tasks to the MEC servers. The IoT device's offloading decision can be formulated as an MDP. The system states include the battery status, the channel capacity, and the predicted amount of harvested energy in the future. The IoT device evaluates the reward based on the overall delay, energy consumption, the task drop loss and the data sharing gains in each time slot. Similar to [108], the authors in [112] enhance Q-learning by the hotbooting technique to save the random exploration time at the beginning of learning. The authors also propose a fast DQL offloading scheme that uses hotbooting to initialize the CNN and accelerates the learning speed. The authors in [113] view the MEC-enabled BSs as different physical machines constituting a part of the cloud resources. The cloud optimizes the MUs' computation offloading to different virtual machines residing on the physical machines. A two-layered DQL algorithm is proposed for the offloading problem to maximize the utilization of cloud resources. The system state relates to the waiting time of each computational task and the number of virtual machines. The first layer is implemented by a CNN-based DQL framework to estimate an optimal cluster for each computational task. Different clusters of physical machines are generated based on the K-NN algorithm. The second layer determines the optimal serving physical machine within the cluster by Q-learning method.

The aforementioned works all focus on data or computation offloading in cellular system via BSs to remote MEC servers, e.g., [101]–[104], [108], [112], [113]. In [115] and [114], the authors study QoS-aware computation offloading in an ad-hoc mobile network. By making a certain payment, the mobile user can offload its computational tasks to nearby mobile users constituting a mobile cloudlet, as shown in Fig. 16(d). Each mobile user has a first-in-first-out queue with limited buffer size to store the arriving tasks arriving as a Poisson process. The mobile user selects nearby cloudlets within D2D communication range for task offloading. The offloading decision depends on the states including the number of remaining tasks, the quality of the links between mobile users and the cloudlet, and the availability of the cloudlet's resources. The objective is to maximize a composite utility function, subject to the mobile user's QoS requirements, e.g., energy consumption and processing delay. The utility function is firstly an increasing function of the total number of tasks that have been processed either locally or remotely by the cloudlets. It is also related to the user's benefit such as energy efficiency and payment for task offloading. This problem is formulated as an MDP and solved by linear programming and Q-learning approaches, depending on the availability of information about the state transition probabilities. This work is further enhanced by leveraging DNN or DQN to learn the decision strategy more efficiently. A similar model is studied in [116], where the computation offloading is formulated as an MDP to minimize the cost of computation offloading. The solution to the MDP can be used to train a DNN by supervised learning. The well-trained DNN is then applied to unseen network conditions for real-time decision-making. Simulation results show that the use of deep supervised learning achieves significant performance gain in offloading accuracy and cost saving.

Data and computation offloading is also used in fog computing. The mobile application demanding a set of data and computational resources can be hosted in a container, e.g., virtual machine of a fog node. With user's mobility, the container has to be migrated or offloaded to other nodes and dynamically consolidated. With the container migration, some nodes with low resource utilization can be switched off to reduce power consumption. The authors in [117] model the container migration as a multi-dimensional MDP, which is solved by the DQL. The system states consist of the delay, the power consumption and the migration cost. The action includes the selection policy that selects the containers to be emigrated from each source node, and the allocation policy that determines the destination node of each container. The action space can be optimized for more efficient exploration by dividing fog nodes into under-utilization, normal-utilization, and over-utilization groups. By powering off under-utilization nodes, all their containers will

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:35:20 UTC from IEEE Xplore. Restrictions apply.

be migrated to other nodes to reduce power consumption. The training process is also optimized by using DDQN and PER which assigns different priorities to the transitions in experience memory. This helps the DQL agent at each fog node to perform better in terms of faster learning speed and more stability. The analysis presents that the proposed scheme can be executed in polynomial time. Simulation results reveal that the DQL scheme achieves fast decision-making and outperforms the existing baseline approaches significantly in terms of delay, power consumption, and migration cost.

*Summary:* This section reviews the applications of the DQL for wireless caching and data/computation offloading, which are inherently coupled with networking and allocation of channel capacity, computational resources, and caching capabilities, etc. We observe that the DQL framework for caching is typically centralized and mostly implemented at the network controller, e.g., the BS, service provider, and central scheduler, which is more powerful in information collection and cross-layer policy design. On the contrary, the end users have more control over their offloading decisions, and hence we observe more popular implementation of the DQL agent at local devices, e.g., mobile users, IoT devices, and fog nodes. Though an orchestration of networking, caching, data and computation offloading in one unified DQL framework is promising for network performance maximization, we face many challenges in designing highly-stable and fast-convergent learning algorithms, due to excessive delay and unsynchronized information collection from different network entities.

## V. Network Security and Connectivity Preservation

Future networks become more decentralized and ad-hoc in nature which are vulnerable to various attacks such as Denial-of-Service (DoS) and cyber-physical attack. Recently, the DQL has been used as an effective solution to avoid and prevent the attacks. In this section, we review the applications of DQL in addressing the following security issues.

- *Jamming attack:* In the jamming attack, attackers as jammers transmit Radio Frequency (RF) jamming signals with high power to cause interference to the legitimate communication channels, thus reducing the SINR at legitimate receivers. Anti-jamming techniques such as the frequency hopping [118] and user mobility, i.e., moving out from the heavy jamming area, have been commonly used. However, without being aware of the radio channel model and the jamming methods, it is challenging for the users to choose an appropriate frequency channel as well as to determine how to leave and avoid the attack. DQL enables the users to learn an optimal policy based on their past observations, and thus DQL can be used to address the above challenge.
- *Cyber-physical attack:* The cyber-physical attack is an integrity attack in which an attacker manipulates data to alter control signals in the system. This attack often happens in autonomous systems such as Intelligent Transportation Systems (ITSs) and increases the risk of accidents to Autonomous Vehicles (AVs). The DQL allows the AVs to learn optimal actions based on their time-varying observations of the attacker' activities. Thus, the DQL can be used to achieve robust and dynamic control of the AV to the attacks.
- *Connectivity preserving:* This refers to maintaining the connectivity among the robots, e.g., UAVs, to support the communication and exchange of information among them. The system and network environment is generally dynamic and complex, and thus the DQL which allows each robot to make dynamic decisions based on its state can be effectively used to preserve the connectivity in the system.

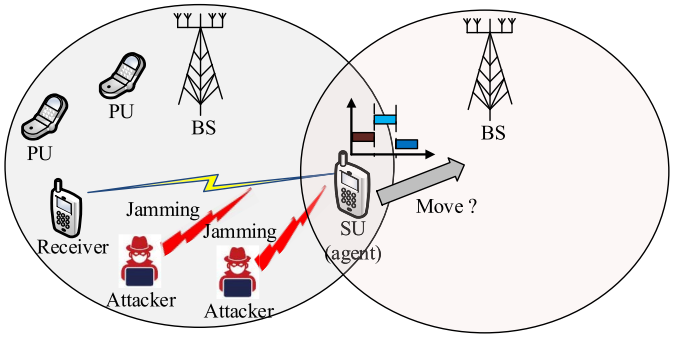



Fig. 17. Jamming attack in cognitive radio network [119].

### A. Network Security

This section discusses the applications of DQL to address the jamming attack and the cyber-physical attack.

*1) Jamming Attack:* A pioneer work using the DQL for the anti-jamming is [119]. The network model is a Cognitive Radio Network (CRN) as shown in Fig. 17 which consists of one Secondary User (SU), multiple Primary Users (PUs), and multiple jammers. The network has a set of frequency channels for hopping. At each time slot, each jammer can arbitrarily select one of the channels to send its jamming signal, and the SU, i.e., the agent, needs to choose a proper action based on the SU's current state. The action is (i) selecting one of the channels to send its signals or (ii) leaving the area to connect to another BS. The jammers are assumed to avoid causing interference to the PUs, and thus the SU's current state consists of the number of PUs and the discretized SINR of the SU signal at the last time slot. The objective of the SU is to maximize its expected discounted utility over time slots. Note that when the SU chooses to leave the area to connect to another BS, it spends a mobility cost. Thus, the utility is defined as a function of the SINR of the SU signal and the mobility cost. Since the number of frequency channels may be large that results in a large action set, the CNN is used for the DQL to quickly learn the optimal policy. As shown in the simulation results, the proposed DQL has a faster convergence speed than that of the Q-learning algorithm. In particular, the utility of the SU increases from 2.73 at the beginning to 3.39 at time slot 1000 that is 8.3% higher than that of the Q-learning

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:35:20 UTC from IEEE Xplore. Restrictions apply.

TABLE V
A SUMMARY OF APPROACHES USING DQL FOR CACHING AND OFFLOADING

| ISSUES | REF. | MODEL | LEARNING ALGORITHMS | AGENT | STATES | ACTIONS | REWARDS | NETWORKS |
|---|---|---|---|---|---|---|---|---|
| Wireless proactive caching | [77] | MDP | DQN using actor-critic, DDPG | Base station | Cached contents and requested content | Replace selected content or not | Cache hit rate (score 1 or 0) | CRN |
| | [91] | MDP | DQN using FNN | Base station | Channel states and computational capabilities | User association, computational unit, content delivery | Energy consumption | CRN |
| | [81] | MDP | DQN using NAFs | Cloud database | Encoding of a query, query cache miss rate | Cache expiration times | Cache hit rates, CDN utilization | Cloud database |
| | [84] [85] | MDP | DQN using FNN | Central scheduler | Channel coefficients, cache state | Active users and resource allocation | Network throughput | MU MIMO system |
| | [86] | MDP | DQN using CNN | Central scheduler | Channel coefficients, cache state | Active users and resource allocation | Network throughput | MU MIMO system |
| | [87] | MDP | DDQN | Service provider | Conditions of cache nodes, transmission rates of content chunks | The content chunks to cache and to remove | Network cost, QoE | Content centric IoT |
| | [89] | MDP | DQN using LSM and ESN | Base station | Historical content request | User association, cached contents and formats | Reliability | Cellular system |
| | [93] [96] | MDP | DQN using CNN | Service provider | Available BS, MEC, and cache | User association, caching, and offloading | Composite revenue | Vehicular ad hoc network |
| | [92] | MDP | DQN using FNN | Service provider | Available BS, MEC, and cache | User association, caching, and offloading | Composite revenue | Vehicular ad hoc network |
| | [94] | MDP | DDQN and dueling DQN | Service provider | Available BS, MEC, and cache | User association, caching, and offloading | Composite revenue | Vehicular ad hoc network |
| | [98] | MDP | DQN using CNN | Base station | Channel state, computational capability, content/version indicator, and the trust value | User association, caching, and offloading | Revenue | Mobile social network |
| Data and computation offloading | [101] | MDP | DQN using CNN | Mobile user | User's location and remaining file size | Idle, transmit via WLAN or cellular network | Total data rate | Cellular system |
| | [102] | MDP | DQN using FNN | Base station | Sum of cost and computational capacity of the MEC server | Offloading decision and resource allocation | Sum of cost of delay and energy consumption | Cellular system |
| | [103] | MDP | DQN using FNN | Mobile user | Channel qualities, states of energy and task queues | Offloading and resource allocation | Long term cost function | Cellular system |
| | [104] | MDP | DDQN, SARSA | Mobile user | Channel qualities, states of energy and task queues | Offloading decision and computational resource allocation | Long term cost function | Cellular system |
| | [108] | Game | DQN using CNN, hotbooting Q-learning | Mobile user | Channel states, size of App traces | Offloading rate | Utility related to detection accuracy, response speed, and the transmission cost | Cellular system |
| | [117] | MDP | DDQN | Fog node | Delay, container's location and resource allocation | Container's next location | Composite utility related to delay, power consumption, and migration cost | Fog computing |

algorithm. Moreover, considering the scenario with two jammers, the proposed DQL outperforms the frequency-hopping method in terms of the SINR and the mobility cost.

The model in [119] is constrained to two jammers. As the number of jammers in the network increases, the proposed scheme may not be effective. The reason is that it becomes hard for the SU to find good actions when the number of jammed channels increases. An appropriate solution, as proposed in [120], allows the receiver of the SU to leave its current location. Since the leaving incurs the mobility cost, the receiver, i.e., the agent, needs an optimal policy, i.e., staying at or leaving the current location, to maximize its utility. In this scenario, the DQL based on CNN can be used for the receiver to find the optimal action to maximize its expected utility. Here, the utility and state of the receiver are essentially defined similarly to that of the agent in [119]. In particular, the state includes the discretized SINR of the signal measured by the receiver at the last time slot. Simulation results show that the proposed DQL converges to SINR and utility values that are higher than those obtained by the Q-learning and random schemes. In particular, the SINR value obtained by the proposed DQL is 3.4, while those obtained by the Q-learning and random schemes are 3.3 and 2.8, respectively.

The above approaches, i.e., in [119] and [120], define states of the agents based on raw SINR values of the signals. In practical wireless environments, the number of SINR values may be large and even infinite. Moreover, the raw SINR can be inaccurate and noisy. To cope with the challenge of the infinite

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:35:20 UTC from IEEE Xplore. Restrictions apply.

number of states, the DQL can use a recursive Convolutional Neural Network (RCNN) as proposed in [121]. By using the pre-processing layer and recursive convolution layers, the RCNN is able to remove noise from the network environment and extract useful features of the SINR, i.e., discrete spectrum sample values greater than a noise threshold, thus reducing the computational complexity. The network model and the problem formulation considered in [121] are similar to those in [119]. However, instead of directly using the raw SINR, the state of the SU is the extracted features of the SINR. Also, the action of the SU includes only frequency-hopping decision. The simulation results show that the proposed DQL based on the RCNN can converge in both fixed and dynamic jamming scenarios while the Q-learning cannot converge in the dynamic jamming one. Furthermore, the proposed DQL can achieve the average throughput close to that of the optimal scheme, i.e., an anti-jamming scheme with completely known jamming actions.

Instead of finding the frequency-hopping decisions, the authors in [122] propose the use of DQL to find an optimal power control policy for the anti-jamming. The model is an IoT network including IoT devices and one jammer. The jammer can observe the communications of the transmitter and chooses a jamming strategy to reduce the SINR at the receiver. Thus, the transmitter chooses an action, i.e., transmit power level, to maximize its utility. Here, the utility is the difference between the SINR and the energy consumption cost due to the transmission. Note that choosing the transmit power impacts the future jamming strategy, and thus the interaction between the transmitter and the jammer can be formulated as an MDP. The transmitter is the agent, and the state is SINR measured at its receiver at the last time slot. The DQN using the CNN is then adopted to find an optimal power control policy for the transmitter to maximize its expected accumulated discounted reward, i.e., the utility, over time slots. The simulation results show that the proposed DQL can improve the utility of the transmitter up to 17.7% compared with the Q-learning. Also, the proposed DQL reduces the utility of the jammer around 18.1% compared with the Q-learning. Moreover, the proposed DQL has faster convergence speed than that of the Q-learning. Specifically, the proposed DQL converges at time slot 210, while the Q-learning converges at time slot 240.

To prevent the jammer's observations of communications, the transmitter can change its communication strategy, e.g., by using relays that are far from the jamming area. The relays can be UAVs as proposed in [123]. The model consists of one UAV, i.e., a relay, one jammer, one mobile user and its serving BS (see Fig. 18). The mobile user transmits messages to its server via the serving BS. In the case that the serving BS is heavily jammed, the UAV helps the mobile user to relay the messages to the server through a backup BS. In particular, depending on the SINR and Bit Error Rate (BER) values sent from the serving BS, the UAV as an agent decides the relay power level to maximize its utility, i.e., the difference between the SINR and the relay cost. The relay power level can be considered to be the UAV's actions, and the SINR and BER are its states. As such, the next state observed by the UAV is independent of all the past states and actions. The problem is formulated as an MDP. To quickly achieve the optimal relay policy for the UAV, the DQL based on CNN is then adopted. The simulation results in [123] show that the proposed DQL scheme takes only 200 time slots to converge to the optimal policy, which is 83.3% less than that of the relay scheme based on Q-learning [124]. Moreover, the proposed DQL scheme reduces the BER of the user by 46.6% compared with the hill climbing-based UAV relay scheme [125].

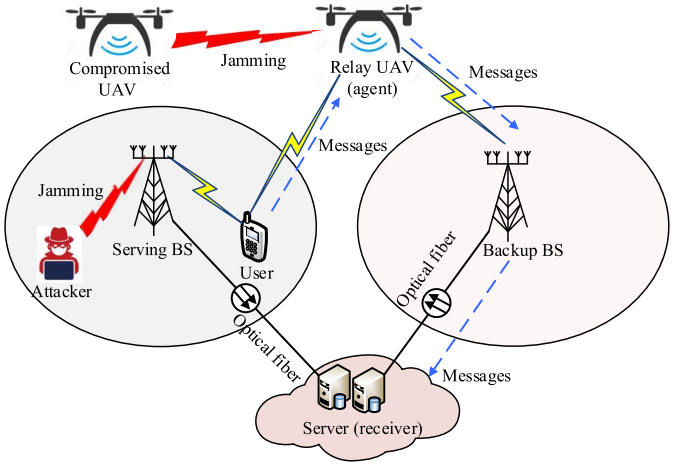



Fig. 18. Anti-jamming scheme based on UAV [123].

The scheme proposed in [123] assumes that the relay UAV is sufficiently far from the jamming area. However, as illustrated in Fig. 18, the attacker can use a compromised UAV close to the relay UAV to launch the jamming attack to the relay UAV. In such a scenario, the authors in [126] show that the DQL can still be used to address the attack. The system model is based on physical layer security and consists of one UAV and one attacker. The attacker is assumed to be "smarter" than that in the model in [123]. This means that the attacker can observe channels that the UAV uses to communicate with the BS in the past time slots and then chooses jamming power levels on the target channels. Therefore, the UAV needs to find a power allocation policy, i.e., transmit power levels on the channels, to maximize the secrecy capacity of the UAV-BS communication. Similar to [123], the DQL based on CNN is used which enables the UAV to choose its actions, i.e., transmit power levels on the channels, based on its state, i.e., the attacker's jamming power level in the last time slot. The reward is the difference between the secrecy capacity of the UAV and BS and the energy consumption cost.

The simulation results in [126] show that the proposed DQL can improve the UAV's utility up to 13% compared with the baseline scheme [127] which uses the Win or Learn Faster-Policy Hill Climbing (WoLF-PHC) to prevent the attack. Also, the safe rate of the UAV, i.e., the probability that the UAV is attacked, obtained by the proposed DQL is 7% higher than that of the baseline. However, the proposed DQL has higher computational complexity and takes longer time to make a decision in each time epoch compared with the WoLH-PHC. Thus, the proposed DQL is applied only to a single-UAV system. For the future work, scenarios with multiple UAVs need to be considered. In such a scenario, more computational

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:35:20 UTC from IEEE Xplore. Restrictions apply.

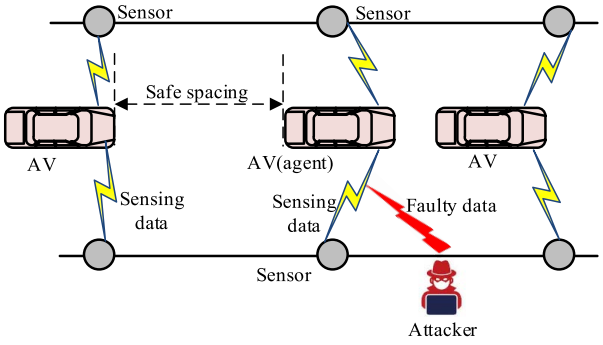



Fig. 19. Car-following model with cyber-physical attack.

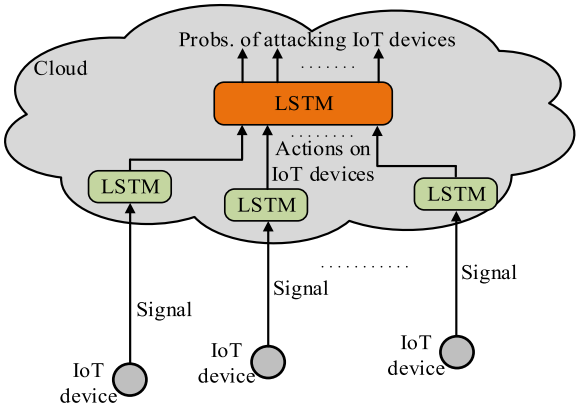



Fig. 20. Cyber-physical detection in IoT systems using DQL.

overhead is expected and multi-agent DQL algorithms can be applied.

*2) Cyber-Physical Attack:* In autonomous systems such as ITSs, the attacker can seek to inject faulty data to information transmitted from the sensors to the AVs. The AVs which receive the injected information may inaccurately estimate the safe spacing among them. This increases the risk of AV accidents. Vehicular communication security algorithms, e.g., [128], can be used to minimize the spacing deviation. However, the attacker's actions in these algorithms are assumed to be stable which may not be applicable in practical systems. The DQL that enables the AVs to learn optimal actions based on their time-varying observations of the attacker' actions can be thus used.

The first work using the DQL for the cyber-physical attack in an ITS can be found in [129]. The system is a *car-following* model [130] of the General Motors as shown in Fig. 19. In the model, each AV updates its speed based on measurement information received from the closest road smart sensors. The attacker attempts to inject faulty data to the measurement information. However, the attacker cannot inject the measurements of different sensors equally due to its resource constraint. Thus, the AV can choose less-faulty measurements by selecting a vector of measurement weights. The objective of the attacker is to maximize the deviation, i.e., the utility, from the safe spacing between the AV and its nearby AV while that of the AV is to minimize the deviation. The interaction between the attacker and the AV can be modeled as a zero-sum game. The authors in [129] show that the DQL can be used to find the equilibrium strategies. In particular, the action of the AV is to choose a weight vector. Its state includes the past actions, i.e., the weight vectors, and the past deviation values. Since the actions and deviations have continuous values, the state space is infinite. Thus, LSTM units that are able to extract useful features are adopted for the DQL to reduce the state space. The simulation results show that by using the past actions and deviations for learning the attacker's action, the proposed DQL scheme can guarantee a lower steady-state deviation than the Kalmar filter-based scheme [128]. Moreover, by using the LSTM units, the results show that the proposed DQL scheme can converge much faster than the baseline scheme.

Another work that uses the LSTM to extract useful features from the measurement information to detect the cyber-physical attack is proposed in [131]. The model is an IoT system including a cloud and a set of IoT devices. The IoT devices generate signals and transmit the signals to the cloud (see Fig. 20). The cloud uses the received signals for estimation and control of the IoT devices' operation. An attacker can launch the cyber-physical attack by manipulating the IoT devices' output signals that causes control errors at the cloud and degrades the performance of the IoT system. To detect the attack, the cloud uses LSTM units to extract stochastic features or fingerprints such as flatness, skewness, and kurtosis, of the IoT devices' signals. The cloud sends the fingerprints back to the IoT devices, and the IoT devices embed, i.e., watermark, the fingerprints inside the signals. The cloud uses the fingerprints to authenticate the IoT devices' signals to detect the attack. The computational complexity of the proposed signal authentication method is $O(df_s^i)$, where $d$ is the delay that the cloud authenticates any IoT signal, and $f_s^i$ is the sampling rate of IoT device $i$.

The algorithm proposed in [131] is also called *dynamic watermarking* [132] which is able to detect the cyber-physical attack and to prevent eavesdropping attacks. However, the algorithm requires large computational resources at the cloud for the IoT device signal authentication. Consequently, the cloud can only authenticate a limited number of vulnerable IoT devices. The cloud can choose the vulnerable IoT devices by observing their security status. However, this can be impractical since the IoT devices may not report their security status. Thus, the authors in [133] propose to use the DQL that enables the cloud to decide which IoT devices to authenticate with the incomplete information. Since IoT devices with more valuable data are likely to be attacked, the reward is defined as a function of data values of IoT devices. The cloud's state includes attack actions of the attacker on the IoT devices in the past time slots. The actions of the attacker on the IoT devices can be obtained by using the dynamic watermarking algorithm in [131] (see Fig. 20). The DQL then uses an LSTM unit to find the optimal policy. The input of the LSTM unit is the state of the cloud, and the output includes probabilities of attacking the IoT devices. By using a real dataset from the accelerometers, the simulation results show that the proposed DQL can improve the cloud's utility up to 30% compared with the case in which the cloud chooses the IoT devices with equal probability.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:35:20 UTC from IEEE Xplore. Restrictions apply.

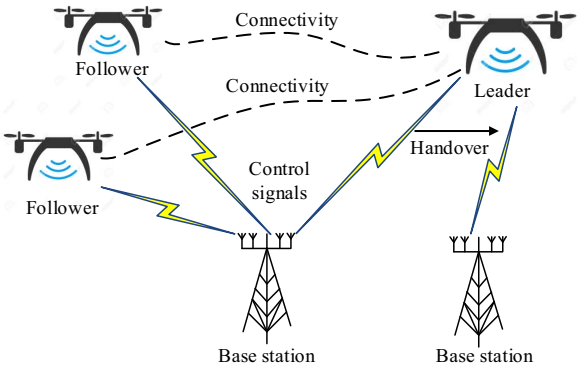



Fig. 21. Connectivity preservation of a multi-UAV network.

### B. Connectivity Preservation

Multi-robot systems such as multi-UAV cooperative networks have been widely applied in many fields such as military, e.g., enemy detecting. In the cooperative multi-robot system, the connectivity among the robots, e.g., UAVs in Fig 21, is required to enable the communication and exchange of information. To tackle the connectivity preservation problem, the Artificial Potential Field (APF) algorithm [134] is used. However, the algorithm cannot be directly adopted when the robots are undertaking missions in dynamic and complex environments. The DQL which allows each robot to make dynamic decisions based on its own state can be effectively applied to preserve the connectivity in the multirobot system. Such an approach is proposed in [135].

The model in [135] consists of two robots or UAVs, i.e., one leader robot and one follower robot. In the model, a central control, i.e., a ground BS, adjusts the velocity of the follower such that the follower stays in the communication range of the leader at all time (see Fig 21). The connectivity preservation problem can be thus formulated as an MDP. The agent is the BS, and the states are the relative position and the velocity of the leader with respect to the follower. The action space consists of possible velocity values of the follower. Taking an action returns a reward which is $+1$ if the follower is in the range of the leader, and $-1$ otherwise. A DQN using FNN is used which enables the BS to learn an optimal policy to maximize the expected discounted cumulative reward. The input of the DQN includes the states of the two robots, and the output is the action space of the follower. The simulation results show that for different locations of the leader and the follower, the score obtained by the proposed scheme is always 100, while the score of the APF method can be occasionally less than 100. This means that the proposed scheme achieves better connectivity between the two robots than that of the APF method. However, a general scenario with more than one leader and one follower needs to be investigated.

Considering the general scenario, the authors in [136] address the connectivity preservation between multiple leaders and multiple followers. The robot system is definitely connected if any two robots are connected via a direct link or multi-hop link. To express the connectivity in such a robot system, the authors introduce the concept of *algebraic connectivity* [137] which is the second smallest eigenvalue of a Laplacian matrix. The robot system is connected if the algebraic connectivity of the system is positive. Thus, the problem is to adjust the velocity of the followers such that the algebraic connectivity is positive over time slots. This problem can be formulated as an MDP in which the agent is the ground BS, the state is a combination of the states of all robots, the action is a set of possible velocity values for the followers. The reward is $+1$ if the algebraic connectivity of the system increases or holds, and becomes a penalty of $-1$ if the algebraic connectivity decreases. Similar to [135], a DQN is adopted. Due to the large action space of the followers, the actor-critic neural network [31] is used. The simulation results show that the followers always follow the motion of the leaders even if the leaders' trajectory dynamically changes. This demonstrates the capability of DQN to tackle the connectivity preservation problem for multi-robot systems. However, the proposed DQN requires more time to converge than that in [135] because of the presence of more followers.

The proposed schemes in [135] and [136] do not consider a minimum distance between the leaders and followers. The leaders and followers can collide with each other if the distance between them is too short. Thus, the BS needs to guarantee the minimum distance between them. One solution is to have the minimum distance in the reward as proposed in [138]. In particular, if the leader is too close to its follower, the reward of the system is penalized regarding the minimum distance. The DQL algorithm proposed in [136] is then used such that the BS learns proper actions, e.g., turning left and right, to maximize the cumulative reward.

When BSs are densely deployed, the UAVs or mobile users need to trigger a frequent handover to preserve the connectivity. The frequent handover increases communication overhead and energy consumption of the mobile users, and interrupts data flows. Thus, it is essential to maintain an appropriate handover rate. The authors in [32] address the handover decision problem in an ultra-density network. The network model consists of multiple mobile users, SBSs, and one central controller. At each time slot, the user needs to decide its serving SBS. The handover decision process can be modeled as an MDP, and the DQL is adopted to find an optimal handover policy for each user to minimize the number of handover occurrences while ensuring certain throughput. The state of the user, i.e., the agent, includes reference signal quality received from candidate SBSs and the last action of the user. The reward is defined as the difference between the data rate of the user and its energy consumption for the handover process. Given a high density of users, the DQL using A3C and LSTM is adopted to find the optimal policy in short training time. The simulation results show that the proposed DQL can achieve higher throughput and lower handover rate than those of the upper confidence bandit algorithm [139] with similar training time. Specifically, the throughput and the handover rate of the DQL are 0.7 bit/s/Hz and 0.0003, respectively, while those of the upper confidence bandit algorithm are 0.67 bit/s/Hz and 0.00049, respectively.

To enhance the reliability of the communication between the SBSs and the mobile users, the SBSs should be able to handle network faults and failure automatically as self-healing.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:35:20 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
A SUMMARY OF APPROACHES USING DQL FOR NETWORK SECURITY AND CONNECTIVITY PRESERVATION

| ISSUES | REF. | MODEL | LEARNING ALGORITHMS | AGENT | STATES | ACTIONS | REWARDS | NETWORKS |
|---|---|---|---|---|---|---|---|---|
| Network security | [119] | Game | DQN using CNN | Secondary user | Number of PUs and signal SINR | Channel selection and leaving decision | SINR and mobility cost | CRN |
| | [120] | Game | DQN using CNN | Receiving transducer | Signal SINR | Staying and leaving decisions | SINR and mobility cost | Underwater acoustic network |
| | [121] | MDP | DQN using RCNN | SU | Signal SINR | Channel selection | SINR and mobility cost | CRN |
| | [122] | MDP | DQN using CNN | Transmit IoT device | Signal SINR | Channel selection | SINR and energy consumption cost | IoT |
| | [123] | MDP | DQN using CNN | Relay UAV | Signal SINR and BER | Relay power | SINR and relay cost | UAV |
| | [126] | MDP | DQN using CNN | Transmit UAV | Jamming power | Transmit power | Secrecy capacity and energy consumption cost | UAV |
| | [129] | Game | DQN using LSTM units | Autonomous vehicle | Deviation values | Measurement weight selection | Safe spacing deviation | ITS |
| | [133] | Game | DQN using LSTM units | Cloud | Attack actions on IoT devices | IoT device set selection | IoT devices' data values | IoT |
| Connectivity preservation | [135] | MDP | DQN using FNN | Ground base station | Relative positions and the velocity of robots | Velocity decision | Sore +1 and -1 | Robot system |
| | [136] | MDP | DQN using A3C | Ground base station | Relative positions and the velocity of robots | Velocity decision | Sore +1 and -1 | Robot system |
| | [138] | POMDP | DQN using A3C | Ground base station | Information of distances among robots | Turning left and turning right decisions | Sore +1 and -1 | Robot system |
| | [32] | MDP | DQN using A3C and LSTM | Mobile users | Reference signal received quality and the last action | Serving SBS selection | Data rate and energy consumption | Ultra-dense network |
| | [140] | MDP | DQN using CNN | MBS | The number of active alarms | Enabling transmit diversity and changing antenna azimuth | Score -1, 0, +1, and +5 | Self-organization network |

The DQL can be applied as proposed in [140] to make optimal parameter adjustments based on the observation of the network performance. The model is the 5G network including one MBS. The MBS as an agent needs to handle network faults such as transmit diversity faults and antenna azimuth change, e.g., because of wind. These faults are represented as the MBS's state that is the number of active alarms. Based on the alarms, the MBS can take actions including (i) enabling the transmit diversity and (ii) setting the antenna azimuth to default value. The reward that the MBS receives is the scores, e.g., $-1$, 0, and $+1$, depending on the number of faults happening. The DQL is used to learn the optimal policy. The simulation results show that the proposed DQL can achieve network throughput close to that of the oracle-based self-healing, i.e., the upper-performance bound, but incurs less fault message passing overhead. In particular, the message passing complexity of the proposed DQL is $O(N)$, and that of the oracle-based self-healing is $O(N^2)$, where $N$ is the number of SBSs in the network.

*Summary:* This section reviews applications of DQL for the network security and connectivity preservation. The reviewed approaches are summarized along with the references in Table VI. We observe that the CNN is mostly used for the DQL to enhance the network security. Moreover, DQL approaches for the anonymous system such as robot systems and ITS receive more attentions than other networks. However, the applications of DQL for the cyber-physical security are relatively few and need to be investigated.

## VI. MISCELLANEOUS ISSUES

In previous sections, we have reviewed and analyzed the applications of DRL framework in different aspects (with different technological focuses) of communications and networking, i.e., network access and rate control, caching and offloading, security and connectivity preservation. However, the optimization of communications and networking actually involves cross-layer co-design and interactions among different network entities. For example, the traffic engineering problem may require joint optimization of routing, transmit power control, channel access control, resource allocation and so on. The problem can become more complicated for the emerging communications systems, e.g., UAV and vehicular networks. In this section, we review the other uses of DRL in communications and networking. These issues include (i) traffic engineering and routing, (ii) resource sharing and scheduling, and (iii) crowdsensing and social networking. Due to its model-free nature, DRL provides a flexible tool for dynamic and diversified applications, typically involving high-dimensional, cross-layer, and multi-agent interactions. All these imply a huge space of state transitions and actions.

### A. Traffic Engineering and Routing

Traffic Engineering (TE) in communication networks refers to Network Utility Maximization (NUM) by optimizing a path to forward the data traffic, given a set of network flows from source to destination nodes. Traditional NUM problems are

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:35:20 UTC from IEEE Xplore. Restrictions apply.

mostly model-based. However, with the advances of wireless communication technologies, the network environment becomes more complicated and dynamic, which makes it hard to model, predict, and control. The recent development of DQL methods provides a feasible and efficient way to design experience-driven and model-free schemes that can learn and adapt to the dynamic wireless network from past observations.

Routing optimization is one of the major control problems in traffic engineering. The authors in [141] present the first attempt to use the DQL for the routing optimization. Through the interaction with the network environment, the DQL agent at the network controller determines the paths for all source-destination pairs. The system state is represented by the bandwidth request between each source-destination pair, and the reward is a function of the mean network delay. The DQL agent leverages the actor-critic method for solving the routing problem that minimizes the network delay, by adapting routing configurations automatically to current traffic conditions. The DQL agent is trained using the traffic information generated by a gravity model [142]. The routing solution is then evaluated by OMNet+ discrete event simulator [143]. Simulation results show that the well-trained DQL agent can produce a near-optimal routing configuration in a single step and thus the agent is agile for real-time network control. The proposed approach is attractive as the traditional optimization-based techniques require a large number of steps to produce a new configuration. The authors in [144] consider a similar network model with multiple end-to-end communication sessions. Each source-destination pair has a set of candidate paths that can transport the traffic load. Experimental results show that the conventional DDPG method does not work well for the continuous control problem in [144]. One possible explanation is that DDPG utilizes uniform sampling for experience replay, which ignores different significance of the transition samples.

The authors in [144] also combine two new techniques to optimize DDPG particularly for traffic engineering problems, i.e., TE-aware exploration and actor-critic-based PER methods. The TE-aware exploration leverages the shortest path algorithm and NUM-based solution as the baseline during exploration. The PER method is conventionally used in DQL, e.g., [87] and [117], while the authors in [144] integrate the PER method with the actor-critic framework for the first time. The proposed scheme assigns different priorities to transitions in the experience replay. Based on the priority, the proposed scheme samples the transitions in each epoch. The system state consists of throughput and delay performance of each communication session. The action specifies the amount of traffic load going through each of the paths. By learning the dynamics of network environment, the DQL agent aims to maximize the total utility of all the communication sessions, which is defined based on end-to-end throughput and delay [145]. Packet-level simulations using NS-3 [146], tested on well-known network topologies as well as random topologies generated by BRITE [147], reveal that the proposed DQL scheme significantly reduces the end-to-end delay and improves the network utility, compared with the baseline schemes including DDPG and the NUM-based solutions.

The networking and routing optimization become more complicated in the UAV-based wireless communications. The authors in [138] model autonomous navigation of one single UAV in a large-scale unknown complex environment as a POMDP, which can be solved by actor-critic-based DRL method. The system state includes its distances and orientation angles to nearby obstacles, the distance and angle between its present position and the destination. The UAV's action is to turn left or right or keep ahead. The reward is composed of four parts: an exponential penalty term if it is too close to any obstacles, a linear penalty term to encourage minimum time delay, the transition and direction rewards if the UAV is getting close to the target position in a proper direction. Instead of using conventional DDPG for continuous control, the Recurrent Deterministic Policy Gradient (RDPG) is proposed for the POMDP by approximating the actor and critic using RNNs. Considering that RDPG is not suitable for learning using memory replay, the authors in [138] propose the fast-RDPG method by utilizing the actor-critic framework with function approximation [148]. The proposed method derives policy update for POMDP by directly maximizing the expected long-term accumulated discounted reward.

Path planning for multiple UAVs connected via cellular systems is studied in [149] and [150]. Each UAV aims to achieve a tradeoff between maximizing energy efficiency and minimizing both latency and interference caused to the ground network along its path. The network state observable by each UAV includes its distances and orientation angles to cellular BSs, the orientation angle to its destination, and the horizontal coordinates of all UAVs. The action of each UAV includes an optimal path, transmit power, and cell association along its path. The interaction among UAVs is cast as a dynamic game and solved by a multi-agent DRL framework. The use of ESN in the DRL framework allows each UAV to retain previous memory states and make a decision for unseen network states, based on the reward obtained from previous states. ESN is a new type of RNNs with feedback connections, consisting of the input, recurrent, and output weight matrices. ESN training is typically quick and computationally efficient compared with other RNNs. Deep ESNs can exploit the advantages of a hierarchical temporal feature representation at different levels of abstraction, hence disentangling the difficulties in modeling complex tasks. The analysis shows that the computational complexity of the proposed DRL is $O(A^3)$, where $A$ is the number of discretized areas. Simulation results show that the proposed scheme improves the tradeoff between energy efficiency, wireless latency, and the interference caused to the ground network. Results also show that each UAV's altitude is a function of the ground network density and the UAV's objective function is an important factor in achieving the UAV's target.

Besides networked UAVs, vehicle-to-infrastructure also constitutes an important part and provides rich application implications in 5G ecosystem. The authors in [151] adopt the DQL to achieve an optimal control policy in communication-based train control system, which is supported by bidirectional train-ground communications. The control problem aims to optimize the handoff decision and train control policy, i.e., accelerate

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:35:20 UTC from IEEE Xplore. Restrictions apply.

or decelerate, based on the states of stochastic channel conditions and real-time information including train position, speed, measured SNR from APs, and handoff indicator. The objective of the DQL agent is to minimize a weighted combination of operation profile tracking error and energy consumption.

### *B. Resource Sharing and Scheduling*

System capacity is one of the most important performance metrics in wireless communication networks. System capacity enhancements can be based on the optimization of resource sharing and scheduling among multiple wireless nodes. The integration of DRL into 5G systems would revolutionize the resource sharing and scheduling schemes from model-based to model-free approaches and meet various application demands by learning from the network environment.

The authors in [152] study the user scheduling in a multi-user massive MIMO system. User scheduling is responsible for allocating resource blocks to BSs and mobile users, taking into account the channel conditions and QoS requirements. Based on this user scheduling strategy, a DRL-based coverage and capacity optimization is proposed to obtain dynamically the scheduling parameters and a unified threshold of QoS metric. The performance indicators are calculated as the average spectrum efficiency of all the users. The system state is an indicator of the average spectrum efficiency. The action of the scheduler is a set of scheduling parameters to maximize the reward as a function of the average spectrum efficiency. The DRL scheme uses policy gradient method to learn a policy function (instead of a Q-function) directly from trajectories generated by the current policy. The policy network is trained with a variant of the REINFORCE algorithm [148]. The simulation results in [152] show that compared with the optimization-based algorithms that suffer from incomplete network information, the policy gradient method achieves much better performance in terms of network coverage and capacity.

In [153], the authors focus on dynamic resource allocation in a cloud radio access network and present a DQL-based framework to minimize the total power consumption while fulfilling mobile users' QoS requirements. The system model contains multiple Remote Radio Heads (RRHs) connected to a cloud BaseBand Unit (BBU). The information of RRHs can be shared in a centralized manner. The system state contains information about the mobile users' demands and the RRHs' working states, e.g., active or sleep. According to the system state and the result of last execution, the DQL agent at the BBU decides whether to turn on or off certain RRH(s), and how to allocate beamforming weight for each active RRH. The objective is to minimize the total expected power consumption. The authors propose a two-step decision framework to reduce the size of action space. In the first step, the DQL agent determines the set of active RRHs by Q-learning and DNNs. In the second step, the BBU derives the optimal resource allocation for the active RRHs by solving a convex optimization problem. Through the combination of DQL and optimization techniques, the proposed framework results in a relatively small action space and low online computational complexity. Simulation results show that the framework achieves significant power savings while satisfying user demands and is robust in highly dynamic network environment. The aforementioned works mostly focus on simulations and numerical comparisons. With one step further, the authors in [154] implement a multi-objective DQL framework as the radio-resource-allocation controller for space communications. The implementation uses modular software architecture to encourage re-use and easy modification for different algorithms, which is integrated into the real space-ground system developed by NASA Glenn Research Center.

In emerging and future wireless networks, BSs are deployed with a high density that introduces a number of challenges to resource management such as the power allocation and interference management. Traditional power allocation approaches, e.g., the iterative algorithm based on closed-form factional programming [155] can be used. However, the approaches always require full CSI that may be not available as the networks become larger and more complicated. Learning algorithms such as DRL can be used for the power allocation. The authors in [156] propose to use a DQL scheme which allows the BSs to learn their optimal power control policy. In the proposed scheme, each BS is an agent, the action is choosing power levels, and the state includes interference that the BS caused to its neighbors in the last time slot. The objective is to maximize the BS's data rate. The DQN using FNN is then adopted to implement the DQL algorithm. The DQN using FNN is then adopted to implement the DQL algorithm. In the simulation, the authors model the channel variations in the Jake's model. With random CSI and delay, the DQL algorithm is shown to achieve near-optimal power allocation in real time. In certain scenarios, it even outperforms the centralized algorithms. This verifies that the DQL algorithm can be fast and competitive, especially for the scenarios with inaccurate CSI and non-negligible delay. For the future work, a joint power control and channel selection can be considered.

Network slicing [157] and NFV [97] are two emerging concepts for resource allocation in the 5G ecosystem to provide cost-effective services with better performance. The network infrastructure, e.g., cache, computation, and radio resources, is comparatively static while the upper-layer Virtualized Network Functions (VNFs) are dynamic to support time-varying application-specific service requests. The concept of network slicing is to divide the network resources into multi-layer slices, managed by different service renderers independently with minimal conflicts. The concept of Service Function Chaining (SFC) is to orchestrate different VNFs to provide required functionalities and QoS provisioning.

The authors in [158] propose a DQL scheme for QoS/QoE-aware SFC in NFV-enabled 5G systems. Typical QoS metrics are bandwidth, delay, throughput, etc. The evaluation of QoE normally involves the end-user's participation in rating the service based on direct user perception. The authors quantify QoE by measurable QoS metrics without end-user involvements, according to the Weber-Fechner Law (WFL) [159] and exponential interdependency of QoE and QoS hypothesis [160]. These two principles actually define nonlinear relationship between QoE and QoS. The system state represents the network environment including network topology,

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:35:20 UTC from IEEE Xplore. Restrictions apply.

QoS/QoE status of the VNF instances, and the QoS requirements of the SFC request. The DQL agent selects a certain direct successive VNF instance as an action. The reward is a composite function of the QoE gain, the QoS constraint penalty, and the OPEX penalty. A DQL based on CNNs is implemented to approximate the action-value function. The authors in [161] review the application of a DQL framework in two typical resource management scenarios using network slicing. For radio resource slicing, the authors simulate a scenario containing one single BS with different types of services. Due to limited spectrum resource, the radio resource slicing aims to optimize the allocation of resource blocks to each slice. The reward can be defined as a weighted sum of spectrum efficiency and QoE. For priority-based core network slicing, the authors simulate a scenario with 3 SFCs demanding different computational resources and waiting time. The aim is to minimize the scheduling delay by optimizing the common or dedicated VNFs. The reward can be defined as a weighted sum of spectrum efficiency and QoE. For priority-based core network slicing, the authors simulate a scenario with 3 SFCs demanding different computational resources and waiting time. The reward is the sum of waiting time in different SFCs. Simulation results in both scenarios show that the DQL framework could exploit more implicit relationship between user activities and resource allocation in resource constrained scenarios, and enhance the effectiveness and agility for network slicing.

Mobile data traffic is anticipated to significantly increase that may cause congestion at SBSs in 5G wireless networks. The conventional technique adjusts the uplink-downlink ratio based on the current network traffic condition and lacks knowledge of future traffic patterns. The conventional technique is thus prone to repeated congestion. The deep learning technique based on LSTM can be used as proposed in [162] that allows the SBS to make local prediction of its traffic load based on the past and current traffic load. The input of the LSTM is the traffic information of the SBS in the past and current time slots, and the output is the traffic load predictions in the next time slots. Based on the prediction, the SBS proactively adjusts the uplink-downlink ratio to avoid the congestion. The simulation results show that the proposed scheme outperforms the conventional methods in terms of network throughput and packet loss rate.

Resource allocation and scheduling problems are also important for computer clusters or database systems. This usually leads to an online decision-making problem depending on the information of workload and environment. The authors in [163] propose a DRL-based solution, DeepRM, by employing policy gradient methods [148] to manage resources in computer systems directly from experience. The same policy gradient method is also used in [152] for user scheduling and resource management in wireless systems. DeepRM is a multi-resource cluster scheduler that learns to optimize various objectives such as minimizing average job slowdown or completion time. The system state is the current allocation of cluster resources and the resource profiles of jobs in the queue. The action of the scheduler is to decide how to schedule the pending jobs. By simulations with synthetic dataset, DeepRM is shown to perform comparably or better than state-of-the-art heuristics, e.g., Shortest-Job-First (SJF). It adapts to different conditions and converges quickly, without any prior knowledge of system behavior. In [164], the authors use the actor-critic method to address the scheduling problem in a general-purpose distributed data stream processing systems, which deal with processing of continuous data flow in real time or near-real-time. The system model contains multiple threads, processes, and machines. The system state consists of the current scheduling decision and the workload of each data source. The scheduling problem is to assign each thread to a process of a machine. The agent at the scheduler determines the assignment of each thread, with the objective of minimizing the average processing time. The DRL framework includes three components, i.e., an actor network, an optimizer producing a K-NN set of the actor network's output action, and the critic network predicting the Q-value for each action in the set. The action is selected from the K-NN set with the maximum Q-value. The use of optimizer may avoid unstable learning and divergence problems in conventional actor-critic methods [165]. Simulation results show that the proposed scheme can reduce the average processing time by 25.1% compared to the default scheduler and 9% compared to the model-based method.

### C. Power Control and Data Collection

With the prevalence of IoT and smart mobile devices, mobile crowdsensing becomes a cost-effective solution for network information collection to support more intelligent operations of wireless systems. The authors in [166] consider spectrum sensing and power control in non-cooperative cognitive radio networks. There is no information exchange between PUs and SUs. As such, the SU outsources the sensing task to a set of spatially distributed sensing devices to collect information about the PU's power control strategy. The SU's power control can be formulated as an MDP. The system state is determined by the Received Signal Strength (RSS) at individual sensing devices. The SU chooses its transmit power from the set of pre-specified power levels based on the current state. A reward is obtained if both primary and SUs can fulfill their SNR requirements. Considering the randomness in RSS measurements, the authors propose a DQL scheme for the SU to learn and adjust its transmit power. The DQL is then implemented by a DQN by using FNN. The simulation results show that the proposed DQL scheme is able to converge to a close-to-optimal solution.

In massive MIMO networks, each base station is able to serve a number of users simultaneously. Thus, one critical problem is to allocate power to the users. To address the problem, the authors in [167] propose to use the deep learning. The model consists of multiple base stations that serve multiple users. The problem is to determine a power allocation vector for the users to maximize the global energy efficiency of the network. The global energy efficiency is generally proportional to the data rates of the users and inversely proportionally to their total power consumption. The FNN is trained in which the input includes the signals received at the users, and the

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:35:20 UTC from IEEE Xplore. Restrictions apply.

output includes the predicted power allocation vector. The loss function is formulated from the predicted power allocation vector and the optimal power allocation vector. Here, the optimal power allocation vector is obtained off-line by using the Sequential Fractional Programming (SFP) algorithm [168]. The simulation results show that the proposed deep learning scheme can achieve the energy efficiency close to that of the SFP algorithm while using significantly less CPU time.

Max-min fairness is also a performance metric in the massive MIMO network to enhance the channel capacity of the users. The authors in [169] adopt a deep learning model for the power allocation to achieve the objective. The model consists of multiple cells, and each cell consists of a base station that serves the base station's users. The problem of the base station is to determine the power allocation vector for its serving users to achieve the max-min fairness. Here, the max-min fairness is a max-min optimization problem in which the objective function is the ergodic channel capacity of each user. The FNN is trained in which the input includes geographical positions of the users in the whole network and the output includes the predicted power allocation vector of the users served by the base station. The loss function is a function of the optimal power allocation vector and the predicted power allocation vector. In particular, the optimal power allocation vector is obtained by offline solving the base station's optimization problem based on the traditional bisection approach [170]. The simulation results show that the proposed deep learning scheme matches well the Minimum Mean Square Error (MMSE) scheme [171] that is optimal but has high computational complexity.

The authors in [172] leverage the DQL framework for sensing and control problems in a Wireless Sensor and Actor Network (WSAN), which is a group of wireless devices with the ability to sense events and to perform actions based on the sensed data shared by all sensors. The system state includes processing power, mobility abilities, and functionalities of the actors and sensors. The mobile actor can choose its moving direction, networking, sensing and actuation policies to maximize the number of connected actor nodes and the number of sensing events.

The authors in [173] focus on mobile crowdsensing paradigm, where data inference is incorporated to reduce sensing costs while maintaining the quality of sensing. The target sensing area is split into a set of cells. The objective of a sensing task is to collect data, e.g., temperature and air quality, in all the cells. Here, the sensed data from different cells of the target sensing area may has different quality. The problem for each sensor in the mobile crowdsensing paradigm is how to choose cells to collect data to minimize the sensing cost while guaranteeing the sensing data quality. The optimal cell selection algorithms, e.g., [174], need to know the true data of each cell in advance that is usually impossible in practice. DQL is able to provide optimal decisions without a complete knowledge of the true data. Thus, it can be adopted for the cell selection of the mobile sensors to decide which cell is better to perform sensing tasks. The system state includes the selection matrices for a few past decision epochs. The reward function is determined by the sensing quality and cost in the chosen cells. To extract temporal correlations in learning, the authors propose the DRQN that uses LSTM layers in DQL to capture the hidden patterns in state transitions. Considering inter-data correlations, the authors use the transfer learning method to reduce the amount of data in training. That is, the cell selection strategy learned for one task can benefit another correlated task. Hence, the parameters of DRQN can be initialized by another DRQN with rich training data. Simulations are conducted based on two real-life datasets collected from sensor networks, i.e., the Sensor-Scope dataset [175] in the EPFL campus and the U-Air dataset of air quality readings in Beijing [176]. The experiments verify that DRQN reduces up to 15% of the sensed cells with the same inference quality guarantee. The authors in [177] combine UAV and unmanned vehicle in mobile crowdsensing for smart city applications. The UAV cruises in the above of the target region for city-level data collection. Meanwhile, the unmanned vehicle carrying mobile charging stations moves on the ground and can charge the UAV at a preset charging point.

The target region is divided into multiple subregions and each subregion has a different sample priority. The authors in [177] propose a DQL-based control framework for the unmanned vehicle to schedule its data collection, constrained by limited energy supply. The system state includes information about the sample priority of each subregion, the location of charging point, and the moving trace of the UAV and unmanned vehicle. The UAV and unmanned vehicle can choose the moving direction. The DQL framework utilizes CNNs for extracting the correlation of adjacent subregions, which can increase the convergence speed in training. The DQL algorithm can be enhanced by using a feasible control solution as the baseline during exploration. The PER method is also used in DQL to assign higher priorities to important transitions so that the DQL agent can learn from samples more efficiently. The proposed scheme is evaluated by using real dataset of taxi traces in Rome [178]. Simulation results reveal that the proposed DQL algorithm can obtain the highest data collection rate compared with the MDP and other heuristic baselines.

Mobile crowdsensing is vulnerable to faked sensing attacks, as selfish users may report faked sensing results to save their sensing costs and avoid compromising their privacy. The authors in [179] formulate the interactions between the server and a number of crowdsensing users as a Stackelberg game. The server is the leader that sets and broadcasts its payment policy for different sensing accuracy. In particular, the higher payment is set for more sensing accuracy. Based on the server's sensing policy, each user as a follower then chooses its sensing effort and thus the sensing accuracy to receive the payment. The payment motivates the users to put in sensing efforts, and thus the payment decision process can be modeled as an MDP. The server can apply Q-learning to optimize payment policy without requiring the sensing model. However, in the presence of a large state space, Q-learning has a slow learning rate. Therefore, the server can use the DQL to obtain the optimal payment to maximize its utility, based on the system state consisting of the past sensing quality and the payment policy. The DQL uses a deep CNN to accelerate the learning process and improve the crowdsensing performance

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:35:20 UTC from IEEE Xplore. Restrictions apply.

against selfish users. Simulation results show that the DQL-based scheme produces a higher sensing quality, lower attack rate, and higher utility of the server, exceeding those of both the Q-learning and the random payment strategies. Moreover, the DQL-based scheme can converge at 200 time slots that is 225% faster than that of the Q-learning.

Social networking is an important component of smart city applications. The authors in [180] aim to extract useful information by observing and analyzing the users' behaviors in social networking. One of the main difficulties is that the social behaviors are usually fuzzy and divergent. The authors model pervasive social networking as a monopolistically competitive market, which contains different users as data providers selling information at a certain price. Given the market model, the DQL can be used to estimate the users' behavior patterns and find the market equilibrium. Considering the costly deep learning structure, the authors in [180] propose a Decentralized DRL (DDRL) framework that decomposes the costly deep component from the RL algorithms at individual users. The deep component can be a feature extractor integrated with the network infrastructure and provide mutual knowledge for all individuals. Multiple RL agents can purchase the most desirable data from the mutual knowledge. The authors combine well-known RL algorithms, i.e., Q-learning and learning automata, to estimate users' patterns which are described by vectors of probabilities representing the users' preferences or altitudes to different information. In social networking and smart city applications with human involvement, there can be both labeled and unlabeled data and hence a semi-supervised DRL framework can be designed, by combining the strengths of DNNs and statistical modeling to improve the performance and accuracy in learning. Then, the authors in [181] introduce the semi-supervised DRL framework that utilizes variational auto-encoders [182] as an inference engine to infer the classification of unlabeled data. As a case study, the proposed DRL framework is customized to provide indoor localization based on the RSS from Bluetooth devices. The positioning environment contains a set of positions. Each position is associated with the set of RSS values from the set of anchor devices with known positions. The system state includes a vector of RSS values, the current location, and the distance to the target. The DQL agent, i.e., the positioning algorithm itself, chooses a moving direction to minimize the error distance to the target point. Simulations tested on real-world dataset show an improvement of 23% in terms of the error distance to the target compared with the supervised DRL scheme.

### D. Direction-of-Arrival (DoA) Estimation

Massive MIMO will be deployed for 5G to achieve high-speed communications at Gbps. For this, the Direction-of-Arrival (DoA) estimation is the prerequisite for realizing massive MIMO. However, the required DoA estimation is very challenging since it is difficult to exploit the characteristics of the channel and sparsity of the massive MIMO systems. Deep learning is able to learn useful features, and it can be used for the DoA estimation as proposed in [183]. In particular, the DNN is trained based on a training dataset including DoAs and received signals. The DoAs are randomly generated, and the received signals are obtained by using different wireless channel models. The input of the DNN includes the DoAs and the received signals, and the output includes the estimated DoAs. The simulation results show that the high accuracy of the DoA estimation can be achieved with a mean square error of $10^{-3}$.

### E. Signal Detection

Apart from improving the accuracy of the DoA estimation, enhancing the accuracy of the symbol detection needs to be considered to reduce Bit Error Ratio (BER). The authors in [184] propose to use the deep learning for the symbol detection in OFDM systems. Specifically, the DNN is used in which the input includes each pair of pilot symbol and the corresponding received signal. The output of the DNN includes the predicted pilot symbol. The simulation results show that the proposed scheme outperforms the least square technique in terms of BER. These results imply that the DNN can learn the characteristics of the wireless channel accurately.

### F. User Association and Load Balancing

In the complicated future networks such as HetNets and massive MIMO, user association is one of critical problems to assign users to base stations to maximize the total data rate. The existing approaches generally incur a considerable complexity overhead that impairs the real-time user association. Thus, deep learning can be used to perform the user association in real-time in realistic massive MIMO networks as proposed in [185]. The model is similar to that in [169]. The problem is to determine the user-association vector to maximize the total data rate of all users in the network. The FNN is trained to output the user association vector given the user positions. The training set consists of multiple pairs of user positions and optimal user association vector. Here, the optimal association vector is obtained offline by using the branch-and-cut algorithm [186]. Simulation results show that the total data rate obtained by the deep learning scheme is close to that obtained by the optimal user association scheme from [171]. This implies that the structure of the optimal association can be learned by the deep learning.

### G. User Localization

Channel State Information (CSI) has been recently used for the user localization since the CSI values contain information related to locations of the users. However, in the complicated wireless networks such as massive MIMO, measuring the wireless channel characteristics, i.e., the CSI values, is challenging. Deep learning that is able to learn useful features from complicated environments can be used for the user localization as proposed in [187]. The deep learning algorithm consists of two phases, i.e., the offline training phase and online localization phase. In the offline training phase, a mobile device transmits packets from known locations to a base station. At each location, the base station measures the corresponding CSI values. The measured CSI values as a fingerprint of the location are used to train the FNN. The input

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:35:20 UTC from IEEE Xplore. Restrictions apply.

of the FNN includes the measured CSI values, and the output includes the reconstructed CSI values. The gready learning algorithm [188] is used to train the FNN that minimizes the error between the measured CSI values and the reconstructed CSI values. In the online localization phase, the reconstructed CSI values are used to estimate the location of the mobile device based on the probabilistic methods. Simulation results show that with the proposed deep learning algorithm, 60% of the positions in the test have errors under 1m, while the baseline algorithm [189] only ensures that 25% of the tested positions have errors under 1m.

The FNN can work well as the number of CSI values is small. However, it may be computationally intractable with a large number of CSI values. To address the issue, CNNs can be used to learn the features of the wireless channels for the user localization as proposed in [190]. However, CNN is suitable to process inputs with grid-like structures, e.g., an image. Therefore, the CSI values must be first transformed into so-called channel snapshots. Then, the CNN is trained to output the user locations given the channel snapshots. The analysis shows that the overall complexity of the proposed CNN algorithm is $O(K^2MLN_FS_1S_2)$, where $K$ is the number of convolution Kernels, $L$ is the number of convolutional-activation-pooling layers, $M$ is the number of the base station's antennas, $N_F$ is the number of frequency points, and $S_1$ and $S_2$ are the sizes of the Kernels, respectively. As such, the complexity does not depend on the training set size that is one main advantage of using the CNN for the user localization.

### H. Access Device Detection

Massive Machine-Type Communication (mMTC) is expected to be one of key technologies in 5G. mMTC enables tens of billions of machine-type devices to communicate with each other with high availability, low latency, and high reliability. However, the mMTC systems only allow the devices to transmit small packets with minimum signaling overhead. Thus, it is challenging for the base stations to detect and identify active devices, i.e., devices to access the base stations. The traditional detection algorithms such as iterative algorithm [191] can be used. However, they fail to consider the time constraints. Deep learning that is able to provide online decisions can be an alternative solution as proposed in [192]. The system model consists of a base station and multiple machine-type devices. Each device is assigned with a unique pilot sequence. The signal received at the base station is the sum of signals transmitted from active users. Given the received signal, the base station needs to determine which users are active. The base station uses the FNN in which the input is the received signal, and the output contains the user activity matrix. The FNN is trained in which the training data is randomly generated and the loss function is formulated from the received signal, the user activity matrix and the channel estimations of the users, and the pilot matrix of the users. Simulation results show that the proposed deep learning scheme outperforms the traditional iterative algorithm from [191] in terms of active user detection success rate.

*Summary:* In this section, we review miscellaneous uses of DRL in wireless and networked systems. DRL provides a flexible tool in rich and diversified applications, conventionally involving dynamic system modeling and multi-agent interactions. All these imply a huge space of state transitions and actions. These approaches are summarized along with the references in Table VII. We observe that the NUM problems in 5G ecosystem for traffic engineering and resource allocation face very diversified control variables, including discrete indicators, e.g., for BS (de)activation, user/cell association, and path selection, as well as continuous variables such as bandwidth allocation, transmit power, and beamforming optimization. Hence, both DQL and policy gradient methods are used extensively for discrete and continuous control problems, respectively.

## VII. Challenges, Open Issues, and Future Research Directions

Different approaches reviewed in this survey evidently show that DRL can effectively address various emerging issues in communications and networking. There are existing challenges, open issues, and new research directions which are discussed as follows.

### A. Challenges

*1) State Determination in Density Networks:* The DRL approaches, e.g., [28], allow the users to find an optimal access policy without having complete and/or accurate network information. However, the DRL approaches often require the users to report their local states at every time slot. To observe the local state, the user needs to monitor Received Signal Strength Indicators (RSSIs) from its neighboring BSs, and then it temporarily connects to the BS with the maximum RSSI. However, the future networks will deploy a high density of the BSs, and the RSSIs from different BSs may not be different. Thus, it is challenging for the users to determine the temporary BS [193].

*2) Knowledge of Jammers' Channel Information:* The DRL approach for wireless security as proposed in [126] enables the UAV to find optimal transmit power levels to maximize the security capacity of the UAV and the BS. However, to formulate the reward of the UAV, a perfect knowledge of channel information of the jammers is required. This is challenging and even impossible in practice.

*3) Multi-Agent DRL in Dynamic HetNets:* Most of the existing works focus on the customizations of DRL framework for individual network entities, based on locally observed or exchanged network information. Hopefully, the network environment is relatively static to ensure convergent learning results and stable policies. This requirement may be challenged in a dynamic heterogenous 5G network, which consists of hierarchically nested IoT devices/networks with fast changing service requirements and networking conditions. In such a situation, the DQL agents for individual entities have to be light-weighted and agile to the change of network conditions. This implies a reduce to the state and action spaces in learning, which however may compromise the performance of the

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:35:20 UTC from IEEE Xplore. Restrictions apply.

TABLE VII
A SUMMARY OF APPLICATIONS OF DQL FOR TRAFFIC ENGINEERING, RESOURCE SCHEDULING, AND DATA COLLECTION

| ISSUES | REF. | MODEL | LEARNING ALGORITHMS | AGENT | STATES | ACTIONS | REWARDS | SCENARIOS |
|---|---|---|---|---|---|---|---|---|
| Traffic engineering and routing | [141] | MDP | DQN using actor-critic networks | Network controller | Bandwidth request of each node pair | Traffic load split on different paths | Mean network delay | 5G network |
| | [144] | NUM | DQN using actor-critic networks | Network controller | Throughput and delay performance | Traffic load split on different paths | $\alpha$-fairness utility | 5G network |
| | [138] | POMDP | DQN using actor-critic networks | UAV | Local sensory information, e.g., distances and angles | Turn left or right | Composite reward | UAV navigation |
| | [149] [150] | Game | DQN using ESN | UAV | Coordinates, distances, and orientation angles | Path, transmit power, and cell association | Weighted sum of energy efficiency, latency, and interference | Cellular-connected UAVs |
| | [151] | MDP | DQN using FNN | Train scheduler | Channel conditions, train position, speed, SNR, and handoff indicator | Making handoff of connection, or accelerate or decelerate the train | Tracking error and energy consumption | Vehicle-to-infrastructure system |
| Resource sharing and scheduling | [153] | MDP | DQN using FNN | Cloud baseband unit | MUs' demands and the RRHs' working states | Turn on or off certain RRH(s), and beamforming allocation | Expected power consumption | Cloud RAN |
| | [158] | MDP | DQN with CNN | Network controller | Network topology, QoS/QoE status, and the QoS requirements | Successive VNF instance | Composite function of QoE gain, QoS constraints penalty, and OPEX penalty | Cellular system |
| | [161] | MDP | DQN using FNN | Network controller | The number of arrived packets/the priority and time-stamp of flows | Bandwidth/SFC allocation | Weighted sum of spectrum efficiency and QoE/waiting time in SFCs | 5G network |
| | [164] | MDP | DQN using actor-critic networks | Central scheduler | Current scheduling decision and the workload | Assignment of each thread | Average processing time | Distributed stream data processing |
| Data collection | [166] | MDP | DQN using FNN | Secondary user | Received signal strength at individual sensors | Transmit power | Fixed reward if QoS satisfied | CRN |
| | [173] | MDP | DRQN, LSTM, transfer learning | Mobile sensors | Cell selection matrices | Next cell for sensing | A function of the sensing quality and cost | WSN |
| | [177] | MDP | DQN using CNN | UAV and unmanned vehicle | Subregions' sample priority, charging point's location, and trace of the UAV and unmanned vehicle | Moving direction of the UAV and unmanned vehicle | Fixed reward related to subregions' sample priority | UAV and vehicle |
| | [179] | Game | DQN using CNN | Crowdsensing server | Previous sensing quality and payment policy | Current payment policy | Utility | Mobile crowdsensing |
| | [180] | Game | DDQN | Mobile users | Current preferences | Positive or negative altitude | – | Mobile social network |

convergent policy. The interactions among multiple agents also complicate the network environment and cause a considerable increase to the state space, which inevitably slows down the learning algorithms.

*4) Training and Performance Evaluation of DRL Framework:* The DRL framework requires large amounts of data for both training and performance evaluation. In wireless systems, such data is not easily accessible as we rarely have referential data pools as other deep learning scenarios, e.g., computer vision. Most of the existing works rely on simulated dataset, which undermines the confidence of the DRL framework in practical system. The simulated data set is usually generated by a specific stochastic model, which is a simplification of the real system and may overlook the hidden patterns. Hence, a more effective way for generating simulation data is required to ensure that the training and performance evaluation of the DRL framework are more consistent with practical system.

## B. Open Issues

*1) Distributed DRL Framework in Wireless Networks:* The DRL framework requires large amounts of training for DNNs. This may be implemented at a centralized network controller, which has sufficient computational capacity and the capability for information collection. However, for massive end users with limited capabilities, it becomes a meaningful task to design distributed implementation for the DRL framework that decomposes resource-demanding basic functionalities, e.g., information collection, sharing, and DNN training, from reinforcement learning algorithms at individual devices. The basic functionalities can be integrated with the network controller. It remains an open issue for the design of network infrastructure that supports these common functionalities for distributed DRL. The overhead of information exchange between end users and network controller also has to be well controlled.

*2) Balance Between Information Quality and Learning Performance:* The majority of the existing works consider the

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:35:20 UTC from IEEE Xplore. Restrictions apply.

orchestration of networking, transmission control, offloading, and caching decisions in one DRL framework to derive the optimal policy, e.g., [91]–[96], [98], [99]. However, from a practical viewpoint, the network system will have to pay substantially increasing cost for information gathering. The cost is incurred from large delay, pre-processing of asynchronous information, excessive energy consumption, reduced learning speed, etc. Hence, an open issue is to find the optimal balance between information quality and learning performance so that the DQL agent does not consume too much resources only to achieve insignificantly marginal increase in the learning performance.

### C. Future Research Directions

*1) DRL for Channel Estimation in Wireless Systems:* We expect that the combination of Wireless Power Transfer (WPT) and Mobile Crowd Sensing (MCS), namely Wireless-Powered Crowd Sensing (WPCS) will be a promising technique for the emerging IoT services. To this end, a higher power transfer efficiency of WPT is very critical to enable the deployment of WPCS in low-power wide area network. A "large-scale array antenna based WPT" will achieve this goal of higher WPT efficiency, but the channel estimation should be performed with minimal power consumption at a sensor node. This is because of that the sensor must operate with self-powering via WPT from the dedicated energy source, e.g., power beacon, Wi-Fi or small-cell access point, and/or ambient RF sources, e.g., TV tower, Wi-Fi AP and cellular BS. In this regard, the channel estimation based on the receive power measurements at the sensor node is one viable solution, because the receive power can be measured by the passive-circuit power meter with negligible power consumption. DRL can be used for the time-varying wireless channels with temporal correlations over time by taking the receive power measurements from the sensor node as the input for DRL, which will enable the channel estimation for WPT efficiently.

*2) DRL for Crowdsensing Service Optimization:* In MCS, mobile users contribute sensing data to a crowdsensing service provider and receive an incentive in return. However, due to limited resources, e.g., bandwidth and energy, the mobile user has to decide on whether and how much data to be uploaded to the provider. Likewise, the provider aiming to maximize its profit has to determine the amount of incentive to be given. The provider's decision depends on the actions of the mobile users. For example, with many mobile users contributing data to the crowdsensing service provider, the provider can lower the incentive. Due to a large state space of a large number of users and dynamic environment, DRL can be applied to obtain an optimal crowdsensing policy similar to [194].

*3) DRL for Cryptocurrency Management in Wireless Networks:* Pricing and economic models have been widely applied to wireless networks [195], [196]. For example, wireless users pay money to access radio resources or mobile services. Alternatively, the users can receive money if they contribute to the networks, e.g., offering a relay or cache function. However, using real money and cash in such scenarios faces many issues related to accounting, security, and privacy. Recently, the concept of cryptocurrency based on the blockchain technology has been introduced and adopted in wireless networks, e.g., [197], which has been shown to be a secure and efficient solution. However, the value of cryptocurrency, i.e., token or coin, can be highly dynamic depending on many market factors. The wireless users possessing the tokens can decide to keep or spend the tokens, e.g., for radio resource access and service usage or exchange into real money. In the random cryptocurrency market environment, DRL can be applied to achieve the maximum long-term reward of the cryptocurrency management for wireless users as in [198].

*4) DRL for Auction:* An auction has been effectively used for radio resource management, e.g., spectrum allocation [199]. However, obtaining the solution of the auction, e.g., a winner determination problem, can be complicated and intractable when the number of participants, i.e., bidders and sellers, become very large. Such a scenario is typical in next-generation wireless networks such as 5G highly-dense heterogeneous networks. DRL appears to be an efficient approach for solving different types of auctions such as in [200].

## VIII. Conclusions

This paper has presented a comprehensive survey of the applications of deep reinforcement learning to communications and networking. First, we have presented an overview of reinforcement learning, deep learning, and deep reinforcement learning. Then, we have introduced various deep reinforcement learning techniques and their extensions. Afterwards, we have provided detailed reviews, analyses, and comparisons of the deep reinforcement learning to solve different issues in communications and networking. The issues include dynamic network access, data rate control, wireless caching, data offloading, network security, connectivity preservation, traffic routing, and data collection. Finally, we have outlined important challenges, open issues as well as future research directions.

## References

[1] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. Cambridge, U.K.: MIT Press, 1998.
[2] I. Goodfellow, Y. Bengio, A. Courville, and Y. Bengio, *Deep Learning*. Cambridge, U.K.: MIT Press, 2016.
[3] *Google Achieves Ai "Breakthrough" by Beating Go Champion*, BBC, London, U.K., Jan. 2016. [Online]. Available: https://www.bbc.com/news/technology-35420579
[4] M. L. Puterman, *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. New York, NY, USA: Wiley, 2014.
[5] D. P. Bertsekas, *Dynamic Programming and Optimal Control*, vol. 1. Belmont, MA, USA: Athena Sci., 2005.
[6] R. Bellman, *Dynamic Programming*. Mineola, NY, USA: Courier Corporat., 2013.
[7] Y. Li, "Deep reinforcement learning: An overview," *arXiv preprint arXiv:1701.07274*, pp. 1–85, Nov. 2018.
[8] K. Arulkumaran, M. P. Deisenroth, M. Brundage, and A. A. Barath, "A brief survey of deep reinforcement learning," *IEEE Signal Process. Mag.*, vol. 34, no. 6, pp. 26–38, Nov. 2017.
[9] Y. Xin *et al.*, "Machine learning and deep learning methods for cybersecurity," *IEEE Access*, vol. 6, pp. 35365–35381, 2018.
[10] Z. M. Fadlullah *et al.*, "State-of-the-art deep learning: Evolving machine intelligence toward tomorrow's intelligent network traffic control systems," *IEEE Commun. Surveys Tuts.*, vol. 19, no. 4, pp. 2432–2455, 4th Quart., 2017.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:35:20 UTC from IEEE Xplore. Restrictions apply.

[11] Q. Mao, F. Hu, and Q. Hao, "Deep learning for intelligent wireless networks: A comprehensive survey," *IEEE Commun. Surveys Tuts.*, vol. 20, no. 4, pp. 2595–2621, 4th Quart., 2018.

[12] M. Chen, U. Challita, W. Saad, C. Yin, and M. Debbah, "Machine learning for wireless networks with artificial intelligence: A tutorial on neural networks," *arXiv preprint arXiv:1710.02913*, pp. 1–98, Oct. 2017.

[13] W. Wang, A. Kwasinski, D. Niyato, and Z. Han, "A survey on applications of model-free strategy learning in cognitive wireless networks," *IEEE Commun. Surveys Tuts.*, vol. 18, no. 3, pp. 1717–1757, 3rd Quart., 2016.

[14] R. Lowe, Y. Wu, A. Tamar, J. Harb, O. P. Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 6379–6390.

[15] G. E. Monahan, "State of the art—A survey of partially observable Markov decision processes: Theory, models, and algorithms," *Manag. Sci.*, vol. 28, no. 1, pp. 1–16, 1982.

[16] L. S. Shapley, "Stochastic games," *Proc. Nat. Acad. Sci. USA*, vol. 39, no. 10, pp. 1095–1100, 1953.

[17] J. Hu and M. P. Wellman, "Nash Q-learning for general-sum stochastic games," *J. Mach. Learn. Res.*, vol. 4, pp. 1039–1069, Nov. 2003.

[18] W. C. Dabney, "Adaptive step-sizes for reinforcement learning," Ph.D. dissertation, Dept. Comput. Sci., Univ. Massachusetts Amherst, Amherst, MA, USA, 2014.

[19] C. J. Watkins and P. Dayan, "Q-learning," *Mach. Learn.*, vol. 8, nos. 3–4, pp. 279–292, 1992.

[20] J. Donahue *et al.*, "Long-term recurrent convolutional networks for visual recognition and description," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Boston, MA, USA, 2015, pp. 2625–2634.

[21] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[22] Y. Lin, X. Dai, L. Li, and F.-Y. Wang, "An efficient deep reinforcement learning model for urban traffic control," *arXiv preprint arXiv:1808.01876*, pp. 1–10, Aug. 2018.

[23] S. Gu, E. Holly, T. Lillicrap, and S. Levine, "Deep reinforcement learning for robotic manipulation with asynchronous off-policy updates," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, Singapore, 2017, pp. 3389–3396.

[24] S. Thrun and A. Schwartz, "Issues in using function approximation for reinforcement learning," in *Proc. Connectionist Models Summer*, Hillsdale, NJ, USA, 1993, pp. 1–9.

[25] H. V. Hasselt, "Double Q-learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2010, pp. 2613–2621.

[26] H. Van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. AAAI*, vol. 2. Phoenix, AZ, USA, Feb. 2016, pp. 2094–2100.

[27] O. Naparstek and K. Cohen, "Deep multi-user reinforcement learning for dynamic spectrum access in multichannel wireless networks," in *Proc. IEEE GLOBECOM*, Singapore, Dec. 2017, pp. 1–7.

[28] N. Zhao, Y.-C. Liang, D. Niyato, Y. Pei, M. Wu, and Y. Jiang, "Deep reinforcement learning for user association and resource allocation in heterogeneous networks," in *Proc. IEEE GLOBECOM*, Abu Dhabi, UAE, Dec. 2018, pp. 1–6.

[29] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized experience replay," *arXiv preprint arXiv:1511.05952*, pp. 1–21, Feb. 2016.

[30] Z. Wang, T. Schaul, M. Hessel, H. Van Hasselt, M. Lanctot, and N. De Freitas, "Dueling network architectures for deep reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, New York, NY, USA, Jun. 2016, pp. 1–9.

[31] V. Mnih *et al.*, "Asynchronous methods for deep reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, New York, NY, USA, Jun. 2016, pp. 1928–1937.

[32] Z. Wang, Y. Xu, L. Li, H. Tian, and S. Cui, "Handover control optimization via asynchronous multi-user deep reinforcement learning," in *Proc. IEEE ICC*, Kansas City, MO, USA, May 2018, pp. 1–6.

[33] M. G. Bellemare, W. Dabney, and R. Munos, "A distributional perspective on reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, Sydney, NSW, Australia, Aug. 2017, pp. 449–458.

[34] M. Fortunato *et al.*, "Noisy networks for exploration," in *Proc. Int. Conf. Learn. Represent.*, 2018, pp. 1–18.

[35] M. Hessel *et al.*, "Rainbow: Combining improvements in deep reinforcement learning," in *Proc. 32nd AAAI Conf. Artif. Intell.*, 2018, pp. 3215–3222.

[36] T. P. Lillicrap *et al.*, "Continuous control with deep reinforcement learning," in *Proc. ICRL*, May 2016, pp. 1–14.

[37] D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, and M. Riedmiller, "Deterministic policy gradient algorithms," in *Proc. ICML*, 2014, pp. 387–395.

[38] M. Hausknecht and P. Stone, "Deep recurrent Q-learning for partially observable MDPS," *CoRR*, vol. abs/1507.06527, no. 1, pp. 1–7, 2015.

[39] D. Zhao, H. Wang, K. Shao, and Y. Zhu, "Deep reinforcement learning with experience replay based on SARSA," in *Proc. IEEE Symp. Comput. Intell. (SSCI)*, Athens, Greece, 2016, pp. 1–6.

[40] W. Wang, J. Hao, Y. Wang, and M. Taylor, "Towards cooperation in sequential prisoner's dilemmas: A deep multiagent reinforcement learning approach," *arXiv preprint arXiv:1803.00162*, pp. 1–13, Mar. 2018.

[41] J. Heinrich and D. Silver, "Deep reinforcement learning from self-play in imperfect-information games," *arXiv preprint arXiv:1603.01121*, pp. 1–10, Jun. 2016.

[42] D. Fooladivanda and C. Rosenberg, "Joint resource allocation and user association for heterogeneous wireless cellular networks," *IEEE Trans. Wireless Commun.*, vol. 12, no. 1, pp. 248–257, Jan. 2013.

[43] Y. Lin, W. Bao, W. Yu, and B. Liang, "Optimizing user association and spectrum allocation in HetNets: A utility perspective," *IEEE J. Sel. Areas Commun.*, vol. 33, no. 6, pp. 1025–1039, Jun. 2015.

[44] S. Wang, H. Liu, P. H. Gomes, and B. Krishnamachari, "Deep reinforcement learning for dynamic multichannel access," in *Proc. Int. Conf. Comput. Netw. Commun. (ICNC)*, 2017, pp. 599–603.

[45] Q. Zhao, B. Krishnamachari, and K. Liu, "On myopic sensing for multi-channel opportunistic access: Structure, optimality, and performance," *IEEE Trans. Wireless Commun.*, vol. 7, no. 12, pp. 5431–5440, Dec. 2008.

[46] V. Mnih *et al.*, "Playing Atari with deep reinforcement learning," *CoRR*, vol. abs/1312.5602, pp. 1–9, Dec. 2013.

[47] R. Govindan. *TutorNet: A Low Power Wireless IoT Testbed*. [Online]. Available: http://anrg.usc.edu/www/tutornet/

[48] S. Wang, H. Liu, P. H. Gomes, and B. Krishnamachari, "Deep reinforcement learning for dynamic multichannel access in wireless networks," *IEEE Trans. Cogn. Commun. Netw.*, vol. 4, no. 2, pp. 257–265, Jun. 2018.

[49] J. Zhu, Y. Song, D. Jiang, and H. Song, "A new deep Q-learning-based transmission scheduling mechanism for the cognitive Internet of Things," *IEEE Internet Things J.*, vol. 5, no. 4, pp. 2375–2385, Aug. 2017.

[50] J. Fearnley, "Strategy iteration algorithms for games and Markov decision processes," Ph.D. dissertation, Dept. Comput. Sci., Univ. Warwick, Coventry, U.K., 2010.

[51] M. Chu, H. Li, X. Liao, and S. Cui, "Reinforcement learning based multi-access control and battery prediction with energy harvesting in IoT systems," *IEEE Internet Things J.*, vol. 6, no. 2, Apr. 2019.

[52] X. Di, K. Xiong, P. Fan, H.-C. Yang, and K. B. Letaief, "Optimal resource allocation in wireless powered communication networks with user cooperation," *IEEE Trans. Wireless Commun.*, vol. 16, no. 12, pp. 7936–7949, Dec. 2017.

[53] H. Ye and G. Y. Li, "Deep reinforcement learning for resource allocation in V2V communications," in *Proc. IEEE ICC*, Kansas City, MO, USA, May 2018, pp. 1–6.

[54] U. Challita, L. Dong, and W. Saad, "Proactive resource management in LTE-U systems: A deep learning perspective," *arXiv preprint arXiv:1702.07031*, pp. 1–29, Dec. 2017.

[55] M. Balazinska and P. Castro. (2003). *IBM Watson Research Center*. [Online]. Available: https://crawdad.org/ibm/watson/20030219

[56] H. Li, "Multiagent Q-learning for Aloha-like spectrum access in cognitive radio systems," *EURASIP J. Wireless Commun. Netw.*, vol. 2010, no. 1, pp. 1–15, May 2010.

[57] S. Liu, X. Hu, and W. Wang, "Deep reinforcement learning based dynamic channel allocation algorithm in multibeam satellite systems," *IEEE Access*, vol. 6, pp. 15733–15742, 2018.

[58] A. R. Elsherif, W.-P. Chen, A. Ito, and Z. Ding, "Resource allocation and inter-cell interference management for dual-access small cells," *IEEE J. Sel. Areas Commun.*, vol. 33, no. 6, pp. 1082–1096, Jun. 2015.

[59] M. Chen, W. Saad, and C. Yin, "Liquid state machine learning for resource allocation in a network of cache-enabled LTE-U UAVs," in *Proc. IEEE GLOBECOM*, Singapore, 2017, pp. 1–6.

[60] W. Maass, "Liquid state machines: Motivation, theory, and applications," in *Computability in Context: Computation and Logic in the Real World*. London, U.K.: World Sci., 2011, pp. 275–296.

[61] M. Chen, M. Mozaffari, W. Saad, C. Yin, M. Debbah, and C. S. Hong, "Caching in the sky: Proactive deployment of cache-enabled unmanned aerial vehicles for optimized quality-of-experience," *IEEE J. Sel. Areas Commun.*, vol. 35, no. 5, pp. 1046–1061, May 2017.


Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:35:20 UTC from IEEE Xplore. Restrictions apply.

[62] I. Szita, V. Gyenes, and A. Lõrincz, "Reinforcement learning with echo state networks," in *Proc. Int. Conf. Artif. Neural Netw.*, 2006, pp. 830–839.

[63] *Tyouku of China Network Video Index*. [Online]. Available: http://index.youku.com/

[64] T. Stockhammer, "Dynamic adaptive streaming over HTTP: Standards and design principles," in *Proc. 2nd Annu. ACM Conf. Multimedia Syst.*, San Jose, CA, USA, 2011, pp. 133–144.

[65] M. Gadaleta, F. Chiariotti, M. Rossi, and A. Zanella, "D-DASH: A deep Q-learning framework for DASH video streaming," *IEEE Trans. Cogn. Commun. Netw.*, vol. 3, no. 4, pp. 703–718, Dec. 2017.

[66] J. Klaue, B. Rathke, and A. Wolisz, "EvalVid—A framework for video transmission and quality evaluation," in *Proc. Int. Conf. Model. Techn. Tools Comput. Perform. Eval.*, 2003, pp. 255–272.

[67] H. Mao, R. Netravali, and M. Alizadeh, "Neural adaptive video streaming with pensieve," in *Proc. Conf. ACM Special Interest Group Data Commun.*, 2017, pp. 197–210.

[68] H. Riiser, P. Vigmostad, C. Griwodz, and P. Halvorsen, "Commute path bandwidth traces from 3G networks: Analysis and applications," in *Proc. 4th ACM Multimedia Syst. Conf.*, 2013, pp. 114–118.

[69] X. Yin, A. Jindal, V. Sekar, and B. Sinopoli, "A control-theoretic approach for dynamic adaptive video streaming over HTTP," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 45, no. 4, pp. 325–338, 2015.

[70] T. Huang, R.-X. Zhang, C. Zhou, and L. Sun, "QARC: Video quality aware rate control for real-time video streaming based on deep reinforcement learning," in *Proc. ACM Int. Conf. Multimedia*, Seoul, South Korea, Oct. 2018, pp. 1208–1216.

[71] (2016). *Measuring Fixed Broadband Report*. [Online]. Available: https://www.fcc.gov/reports-research/reports/measuring-broadband-america/raw-data-measuring-broadband-america-2016

[72] S. Chinchali *et al.*, "Cellular network traffic scheduling with deep reinforcement learning," in *Proc. Nat. Conf. Artif. Intell. (AAAI)*, 2018, pp. 1–9.

[73] Z. Zhang, Y. Zheng, M. Hua, Y. Huang, and L. Yang, "Cache-enabled adaptive bit rate streaming via deep self-transfer reinforcement learning," in *Proc. Int. Conf. Wireless Commun. Signal Process.*, Hangzhou, China, Oct. 2018, pp. 1–6.

[74] P. V. R. Ferreir *et al.*, "Multiobjective reinforcement learning for cognitive satellite communications using deep neural network ensembles," *IEEE J. Sel. Areas Commun.*, vol. 36, no. 5, pp. 1030–1041, May 2018.

[75] D. Tarchi, G. E. Corazza, and A. Vanelli-Coralli, "Adaptive coding and modulation techniques for next generation hand-held mobile satellite communications," in *Proc. IEE ICC*, Budapest, Hungary, 2013, pp. 4504–4508.

[76] M. T. Hagan and M. B. Menhaj, "Training feedforward networks with the Marquardt algorithm," *IEEE Trans. Neural Netw.*, vol. 5, no. 6, pp. 989–993, Nov. 1994.

[77] C. Zhong, M. C. Gursoy, and S. Velipasalar, "A deep reinforcement learning-based framework for content caching," in *Proc. Annu. Conf. Inf. Sci. Syst.*, Princeton, NJ, USA, Mar. 2018, pp. 1–6.

[78] T. P. Lillicrap *et al.*, "Continuous control with deep reinforcement learning," *CoRR*, vol. abs/1509.02971, pp. 1–14, Feb. 2016. [Online]. Available: http://arxiv.org/abs/1509.02971

[79] G. Dulac-Arnold, R. Evans, P. Sunehag, and B. Coppin, "Reinforcement learning in large discrete action spaces," *CoRR*, vol. abs/1512.07679, pp. 1–11, Apr. 2016. [Online]. Available: http://arxiv.org/abs/1512.07679

[80] L. Lei, L. You, G. Dai, T. X. Vu, D. Yuan, and S. Chatzinotas, "A deep learning approach for optimizing content delivering in cache-enabled HetNet," in *Proc. Int. Symp. Wireless Commun. Syst. (ISWCS)*, Bologna, Italy, Aug. 2017, pp. 449–453.

[81] M. Schaarschmidt, F. Gessert, V. Dalibard, and E. Yoneki, "Learning runtime parameters in computer systems with delayed experience injection," *arXiv preprint arXiv:1610.09903*, pp. 1–10, Oct. 2016.

[82] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, "Benchmarking cloud serving systems with YCSB," in *Proc. 1st ACM Symp. Cloud Comput.*, Indianapolis, IN, USA, 2010, pp. 143–154.

[83] M. Deghel, E. Bastug, M. Assaad, and M. Debbah, "On the benefits of edge caching for MIMO interference alignment," in *Proc. IEEE Int. Workshop Signal Process. Adv. Wireless Commun. (SPAWC)*, Stockholm, Sweden, 2015, pp. 655–659.

[84] Y. He and S. Hu, "Cache-enabled wireless networks with opportunistic interference alignment," *arXiv preprint arXiv:1706.09024*, pp. 1–6, Jun. 2017.

[85] Y. He, C. Liang, F. R. Yu, N. Zhao, and H. Yin, "Optimization of cache-enabled opportunistic interference alignment wireless networks: A big data deep reinforcement learning approach," in *Proc. IEEE ICC*, Paris, France, 2017, pp. 1–6.

[86] Y. He *et al.*, "Deep-reinforcement-learning-based optimization for cache-enabled opportunistic interference alignment wireless networks," *IEEE Trans. Veh. Technol.*, vol. 66, no. 11, pp. 10433–10445, Nov. 2017.

[87] X. He, K. Wang, H. Huang, T. Miyazaki, Y. Wang, and S. Guo, "Green resource allocation based on deep reinforcement learning in content-centric IoT," *IEEE Trans. Emerg. Topics Comput.*, to be published.

[88] Q. Wu, Z. Li, and G. Xie, "CodingCache: Multipath-aware CCN cache with network coding," in *Proc. ACM SIGCOMM Workshop Inf. Centric Netw.*, Hong Kong, 2013, pp. 41–42.

[89] M. Chen, W. Saad, and C. Yin, "Echo-liquid state deep learning for content transmission and caching in wireless VR networks with cellular-connected UAVs," *arXiv preprint arXiv:1804.03284*, pp. 1–10, Apr. 2018.

[90] M. Chen, U. Challita, W. Saad, C. Yin, and M. Debbah, "Machine learning for wireless networks with artificial intelligence: A tutorial on neural networks," *CoRR*, vol. abs/1710.02913, pp. 1–98, Oct. 2017. [Online]. Available: http://arxiv.org/abs/1710.02913

[91] Y. He, Z. Zhang, and Y. Zhang, "A big data deep reinforcement learning approach to next generation green wireless networks," in *Proc. IEEE GLOBECOM*, 2017, pp. 1–6.

[92] Y. He *et al.*, "Resource allocation in software-defined and information-centric vehicular networks with mobile edge computing," in *Proc. IEEE 86th Veh. Technol. Conf. (VTC-Fall)*, 2017, pp. 1–5.

[93] Y. He, F. R. Yu, N. Zhao, H. Yin, and A. Boukerche, "Deep reinforcement learning (DRL)-based resource management in software-defined and virtualized vehicular ad hoc networks," in *Proc. 6th ACM Symp. Develop. Anal. Intell. Veh. Netw. Appl.*, Miami, FL, USA, 2017, pp. 47–54.

[94] Y. He, N. Zhao, and H. Yin, "Integrated networking, caching, and computing for connected vehicles: A deep reinforcement learning approach," *IEEE Trans. Veh. Technol.*, vol. 67, no. 1, pp. 44–55, Jan. 2018.

[95] L. T. Tan and R. Q. Hu, "Mobility-aware edge caching and computing framework in vehicle networks: A deep reinforcement learning," *IEEE Trans. Veh. Technol.*, vol. 67, no. 11, pp. 10190–10203, Nov. 2018.

[96] Y. He, F. R. Yu, N. Zhao, V. C. Leung, and H. Yin, "Software-defined networks with mobile edge computing and caching for smart cities: A big data deep reinforcement learning approach," *IEEE Commun. Mag.*, vol. 55, no. 12, pp. 31–37, Dec. 2017.

[97] B. Han, V. Gopalakrishnan, L. Ji, and S. Lee, "Network function virtualization: Challenges and opportunities for innovations," *IEEE Commun. Mag.*, vol. 53, no. 2, pp. 90–97, Feb. 2015.

[98] Y. He, F. R. Yu, N. Zhao, and H. Yin, "Secure social networks in 5G systems with mobile edge computing, caching and device-to-device (D2D) communications," *IEEE Wireless Commun.*, vol. 25, no. 3, pp. 103–109, Jun. 2018.

[99] Y. He, C. Liang, F. R. Yu, and Z. Han, "Trust-based social networks with computing, caching and communications: A deep reinforcement learning approach," *IEEE Trans. Netw. Sci. Eng.*, to be published.

[100] C. Zhang, B. Gu, Z. Liu, K. Yamori, and Y. Tanaka, "Cost-and energy-aware multi-flow mobile data offloading using Markov decision process," *IEICE Trans. Commun.*, vol. E101-B, no. 3, pp. 657–666, 2017.

[101] C. Zhang, Z. Liu, B. Gu, K. Yamori, and Y. Tanaka, "A deep reinforcement learning based approach for cost-and energy-aware multi-flow mobile data offloading," *IEICE Trans. Commun.*, vol. 101-B, no. 7, pp. 1625–1634, 2018.

[102] L. Ji, G. Hui, L. Tiejun, and L. Yueming, "Deep reinforcement learning based computation offloading and resource allocation for MEC," in *Proc. IEEE WCNC*, Barcelona, Spain, 2018, pp. 1–5.

[103] X. Chen, H. Zhang, C. Wu, S. Mao, Y. Ji, and M. Bennis, "Performance optimization in mobile-edge computing via deep reinforcement learning," in *Proc. IEEE VTC-Fall*, Chicago, IL, USA, Aug. 2018, pp. 1–6.

[104] X. Chen, H. Zhang, C. Wu, S. Mao, Y. Ji, and M. Bennis, "Optimized computation offloading performance in virtual edge computing systems via deep reinforcement learning," *IEEE Internet Things J.*, to be published.

[105] J. Ye and Y.-J. A. Zhang, "DRAG: Deep reinforcement learning based base station activation in heterogeneous networks," *arXiv:1809.02159*, pp. 1–12, Sep. 2018.


Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:35:20 UTC from IEEE Xplore. Restrictions apply.

[106] H. Li, H. Gao, T. Lv, and Y. Lu, "Deep Q-learning based dynamic resource allocation for self-powered ultra-dense networks," in *Proc. IEEE ICC (ICC Workshops)*, Kansas City, MO, USA, 2018, pp. 1–6.

[107] J. Liu, B. Krishnamachari, S. Zhou, and Z. Niu, "DeepNap: Data-driven base station sleeping operations through deep reinforcement learning," *IEEE Internet Things J.*, vol. 5, no. 6, pp. 4273–4282, Dec. 2018.

[108] X. Wan, G. Sheng, Y. Li, L. Xiao, and X. Du, "Reinforcement learning based mobile offloading for cloud-based malware detection," in *Proc. IEEE GLOBECOM*, Singapore, 2017, pp. 1–6.

[109] L. Xiao, X. Wan, C. Dai, X. Du, X. Chen, and M. Guizani, "Security in mobile edge caching with reinforcement learning," *IEEE Wireless Commun.*, vol. 25, no. 3, pp. 116–122, Jun. 2018.

[110] A. S. Shamili, C. Bauckhage, and T. Alpcan, "Malware detection on mobile devices using distributed machine learning," in *Proc. Int. Conf. Pattern Recognit.*, Istanbul, Turkey, Aug. 2010, pp. 4348–4351.

[111] Y. Li, J. Liu, Q. Li, and L. Xiao, "Mobile cloud offloading for malware detections with learning," in *Proc. IEEE INFOCOM Workshops*, Hong Kong, Apr. 2015, pp. 197–201.

[112] M. Min, D. Xu, L. Xiao, Y. Tang, and D. Wu, "Learning-based computation offloading for IoT devices with energy harvesting," *IEEE Trans. Veh. Technol.*, vol. 68, no. 2, pp. 1930–1941, Feb. 2019.

[113] L. Quan, Z. Wang, and F. Ren, "A novel two-layered reinforcement learning for task offloading with tradeoff between physical machine utilization rate and delay," *Future Internet*, vol. 10, no. 7, p. 60, 2018.

[114] D. V. Le and C.-K. Tham, "Quality of service aware computation offloading in an ad-hoc mobile cloud," *IEEE Trans. Veh. Technol.*, vol. 67, no. 9, pp. 8890–8904, Sep. 2018.

[115] D. V. Le and C.-K. Tham, "A deep reinforcement learning based offloading scheme in ad-hoc mobile clouds," in *Proc. IEEE INFOCOM IECCO Workshop*, Honolulu, HI, USA, Apr. 2018, pp. 760–765.

[116] S. Yu, X. Wang, and R. Langar, "Computation offloading for mobile edge computing: A deep learning approach," in *Proc. IEEE PIMRC*, Montreal, QC, Canada, Oct. 2017, pp. 1–6.

[117] Z. Tang, X. Zhou, F. Zhang, W. Jia, and W. Zhao, "Migration modeling and learning algorithms for containers in fog computing," *IEEE Trans. Services Comput.*, to be published.

[118] P. Popovski, H. Yomo, and R. Prasad, "Strategies for adaptive frequency hopping in the unlicensed bands," *IEEE Wireless Commun.*, vol. 13, no. 6, pp. 60–67, Dec. 2006.

[119] G. Han, L. Xiao, and H. V. Poor, "Two-dimensional anti-jamming communication based on deep reinforcement learning," in *Proc. 42nd IEEE Int. Conf. Acoust. Speech Signal Process.*, 2017, pp. 2087–2091.

[120] L. Xiao, D. Jiang, X. Wan, W. Su, and Y. Tang, "Anti-jamming underwater transmission with mobility and learning," *IEEE Commun. Lett.*, vol. 22, no. 3, pp. 542–545, Mar. 2018.

[121] X. Liu, Y. Xu, L. Jia, Q. Wu, and A. Anpalagan, "Anti-jamming communications using spectrum waterfall: A deep reinforcement learning approach," *IEEE Commun. Lett.*, vol. 22, no. 5, pp. 998–1001, May 2018.

[122] Y. Chen, Y. Li, D. Xu, and L. Xiao, "DQN-based power control for IoT transmission against jamming," in *Proc. IEEE 87th Veh. Technol. Conf. (VTC Spring)*, 2018, pp. 1–5.

[123] X. Lu, L. Xiao, and C. Dai, "UAV-aided cellular communications with deep reinforcement learning against jamming," *arXiv preprint arXiv:1805.06628*, pp. 1–6, Apr. 2019.

[124] L. Xiao, X. Lu, D. Xu, Y. Tang, L. Wang, and W. Zhuang, "UAV relay in VANETs against smart jamming with reinforcement learning," *IEEE Trans. Veh. Technol.*, vol. 67, no. 5, pp. 4087–4097, May 2018.

[125] S. Lv, L. Xiao, Q. Hu, X. Wang, C. Hu, and L. Sun, "Anti-jamming power control game in unmanned aerial vehicle networks," in *Proc. IEEE GLOBECOM*, 2017, pp. 1–6.

[126] L. Xiao, C. Xie, M. Min, and W. Zhuang, "User-centric view of unmanned aerial vehicle transmission against smart attacks," *IEEE Trans. Veh. Technol.*, vol. 67, no. 4, pp. 3420–3430, Apr. 2018.

[127] M. Bowling and M. Veloso, "Multiagent learning using a variable learning rate," *Artif. Intell.*, vol. 136, no. 2, pp. 215–250, 2002.

[128] Y. Chen, S. Kar, and J. M. F. Moura, "Cyber-physical attacks with control objectives," *IEEE Trans. Autom. Control*, vol. 63, no. 5, pp. 1418–1425, May 2018.

[129] A. Ferdowsi, U. Challita, W. Saad, and N. B. Mandayam, "Robust deep reinforcement learning for security and safety in autonomous vehicle systems," in *Proc. Int. Conf. Intell. Transp. Syst.*, Maui, HI, USA, Nov. 2018, pp. 307–312.

[130] M. Brackstone and M. McDonald, "Car-following: A historical review," *Transport. Res. F Traffic Psychol. Behav.*, vol. 2, no. 4, pp. 181–196, 1999.

[131] A. Ferdowsi and W. Saad, "Deep learning-based dynamic watermarking for secure signal authentication in the Internet of Things," in *Proc. IEEE ICC*, Kansas City, MO, USA, 2018, pp. 1–6.

[132] B. Satchidanandan and P. R. Kumar, "Dynamic watermarking: Active defense of networked cyber–physical systems," *Proc. IEEE*, vol. 105, no. 2, pp. 219–240, Feb. 2017.

[133] A. Ferdowsi and W. Saad, "Deep learning for signal authentication and security in massive Internet-of-Things systems," *IEEE Trans. Commun.*, vol. 67, no. 2, pp. 1371–1387, Feb. 2019.

[134] P. Vadakkepat, K. C. Tan, and W. Ming-Liang, "Evolutionary artificial potential fields and their application in real time robot path planning," in *Proc. Congr. Evol. Comput.*, vol. 1, 2000, pp. 256–263.

[135] W. Huang, Y. Wang, and X. Yi, "Deep Q-learning to preserve connectivity in multi-robot systems," in *Proc. 9th Int. Conf. Signal Process. Syst.*, 2017, pp. 45–50.

[136] W. Huang, Y. Wang, and X. Yi, "A deep reinforcement learning approach to preserve connectivity for multi-robot systems," in *Proc. Int. Congr. Image Signal Process. BioMed. Eng. Informat. (CISP-BMEI)*, Shanghai, China, 2017, pp. 1–7.

[137] H. A. Poonawala, A. C. Satici, H. Eckert, and M. W. Spong, "Collision-free formation control with decentralized connectivity preservation for nonholonomic-wheeled mobile robots," *IEEE Trans. Control Netw. Syst.*, vol. 2, no. 2, pp. 122–130, Jun. 2015.

[138] C. Wang, J. Wang, X. Zhang, and X. Zhang, "Autonomous navigation of UAV in large-scale unknown complex environment with deep reinforcement learning," in *Proc. IEEE GlobalSIP*, 2017, pp. 858–862.

[139] C. Shen, C. Tekin, and M. van der Schaar, "A non-stochastic learning approach to energy efficient mobility management," *IEEE J. Sel. Areas Commun.*, vol. 34, no. 12, pp. 3854–3868, Dec. 2016.

[140] M. Faris and E. Brian. (2018). *Deep Q-Learning for Self-Organizing Networks Fault Management and Radio Performance Improvement*. [Online]. Available: https://arxiv.org/abs/1707.02329

[141] G. Stampa, M. Arias, D. Sanchez-Charles, V. Muntes-Mulero, and A. Cabellos, "A deep-reinforcement learning approach for software-defined networking routing optimization," *arXiv preprint arXiv:1709.07080*, pp. 1–3, Sep. 2017.

[142] M. Roughan, "Simplifying the synthesis of Internet traffic matrices," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 35, no. 5, pp. 93–96, 2015. [Online]. Available: http://arxiv.org/abs/1710.02913

[143] A. Varga and R. Hornig, "An overview of the OMNeT++ simulation environment," in *Proc. Int. Conf. Simulat. Tools Techn. Commun. Netw. Syst. Workshops*, 2008, Art. no. 60.

[144] Z. Xu *et al.*, "Experience-driven networking: A deep reinforcement learning based approach," *arXiv preprint arXiv:1801.05757*, 2018.

[145] K. Winstein and H. Balakrishnan, "TCP ex Machina: Computer-generated congestion control," in *Proc. ACM SIGCOMM*, 2013, pp. 123–134.

[146] K. Wehrle, M. Günes, and J. Gross, "Modeling and tools for network simulation," in *The NS-3 Network Simulator*. Heidelberg, Germany: Springer, 2010.

[147] A. Medina, A. Lakhina, I. Matta, and J. Byers, "BRITE: An approach to universal topology generation," in *Proc. IEEE MASCOTS*, Aug. 2001, pp. 346–353.

[148] R. S. Sutton, D. McAllester, S. Singh, and Y. Mansour, "Policy gradient methods for reinforcement learning with function approximation," in *Proc. 12th Int. Conf. Neural Inf. Process. Syst.*, 1999, pp. 1057–1063.

[149] U. Challita, W. Saad, and C. Bettstetter, "Deep reinforcement learning for interference-aware path planning of cellular connected UAVs," in *Proc. IEEE ICC*, Kansas City, MO, USA, May 2018, pp. 1–6.

[150] U. Challita, W. Saad, and C. Bettstetter, "Cellular-connected UAVs over 5G: Deep reinforcement learning for interference management," *arXiv preprint arXiv:1801.05500*, pp. 1–30, Jan. 2018.

[151] L. Zhu, Y. He, F. R. Yu, B. Ning, T. Tang, and N. Zhao, "Communication-based train control system performance optimization using deep reinforcement learning," *IEEE Trans. Veh. Technol.*, vol. 66, no. 12, pp. 10705–10717, Dec. 2017.

[152] Y. Yang *et al.*, "DECCO: Deep-learning enabled coverage and capacity optimization for massive MIMO systems," *IEEE Access*, to be published.

[153] Z. Xu, Y. Wang, J. Tang, J. Wang, and M. C. Gursoy, "A deep reinforcement learning based framework for power-efficient resource allocation in cloud RANs," in *Proc. IEEE ICC*, 2017, pp. 1–6.

[154] T. M. Hackett, S. G. Bilén, P. V. R. Ferreira, A. M. Wyglinski, and R. C. Reinhart, "Implementation of a space communications cognitive engine," in *Proc. Cogn. Commun. Aeros. Appl. Workshop (CCAA)*, 2017, pp. 1–7.


Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:35:20 UTC from IEEE Xplore. Restrictions apply.

[155] K. Shen and W. Yu, "Fractional programming for communication systems—Part I: Power control and beamforming," *IEEE Trans. Signal Process.*, vol. 66, no. 10, pp. 2616–2630, May 2018.

[156] Y. S. Nasir and D. Guo, "Deep reinforcement learning for distributed dynamic power allocation in wireless networks," *arXiv preprint arXiv:1808.00490*, pp. 1–12, Apr. 2019.

[157] X. Foukas, G. Patounas, A. Elmokashfi, and M. K. Marina, "Network slicing in 5G: Survey and challenges," *IEEE Commun. Mag.*, vol. 55, no. 5, pp. 94–100, May 2017.

[158] X. Chen *et al.*, "Reinforcement learning based QoS/QoEaware service function chaining in software-driven 5G slices," *Trans. Emerg. Telecommun. Technol.*, vol. 29, no. 11, Nov. 2018.

[159] P. Reichl, S. Egger, R. Schatz, and A. D'Alconzo, "The logarithmic nature of QoE and the role of the Weber–Fechner law in QoE assessment," in *Proc. IEEE ICC*, Cape Town, South Africa, May 2010, pp. 1–5.

[160] M. Fiedler, T. Hossfeld, and P. Tran-Gia, "A generic quantitative relationship between quality of experience and quality of service," *IEEE Netw.*, vol. 24, no. 2, pp. 36–41, Mar./Apr. 2014.

[161] Z. Zhao *et al.*, "Deep reinforcement learning for network slicing," *arXiv preprint arXiv:1805.06591*, pp. 1–12, Nov. 2018.

[162] Y. Zhou, Z. M. Fadlullah, B. Mao, and N. Kato, "A deep-learning-based radio resource assignment technique for 5G ultra dense networks," *IEEE Netw.*, vol. 32, no. 6, pp. 28–34, Nov./Dec. 2018.

[163] H. Mao, M. Alizadeh, I. Menache, and S. Kandula, "Resource management with deep reinforcement learning," in *Proc. 15th ACM Workshop Hot Topics Netw.*, 2016, pp. 50–56.

[164] T. Li, Z. Xu, J. Tang, and Y. Wang, "Model-free control for distributed stream data processing using deep reinforcement learning," *Proc. VLDB Endowment*, vol. 11, no. 6, pp. 705–718, 2018.

[165] G. D. Arnold *et al.*, "Deep reinforcement learning in large discrete action spaces," *arXiv: 1512.07679*, pp. 1–11, Apr. 2016.

[166] X. Li, J. Fang, W. Cheng, H. Duan, Z. Chen, and H. Li, "Intelligent power control for spectrum sharing in cognitive radios: A deep reinforcement learning approach," *IEEE Access*, vol. 6, pp. 25463–25473, 2018.

[167] A. Zappone, M. Debbah, and Z. Altman, "Online energy-efficient power control in wireless networks by deep neural networks," in *Proc. IEEE 19th Int. Workshop Signal Process. Adv. Wireless Commun. (SPAWC)*, 2018, pp. 1–5.

[168] A. Zappone, E. Björnson, L. Sanguinetti, and E. Jorswieck, "Globally optimal energy-efficient power control and receiver design in wireless networks," *IEEE Trans. Signal Process.*, vol. 65, no. 11, pp. 2844–2859, Jun. 2017.

[169] L. Sanguinetti, A. Zappone, and M. Debbah, "Deep learning power allocation in massive MIMO," in *Proc. Asilomar Conf. Signals Syst. Comput.*, Pacific Grove, CA, USA, Oct. 2018, pp. 1257–1261.

[170] S. Boyd, V. Balakrishnan, and P. Kabamba, "A bisection method for computing the $H_\infty$ norm of a transfer matrix and related problems," *Math. Control Signals Syst.*, vol. 2, no. 3, pp. 207–219, 1989.

[171] E. Björnson, J. Hoydis, and L. Sanguinetti, "Massive MIMO has unlimited capacity," *IEEE Trans. Wireless Commun.*, vol. 17, no. 1, pp. 574–590, Jan. 2018.

[172] T. Oda, R. Obukata, M. Ikeda, L. Barolli, and M. Takizawa, "Design and implementation of a simulation system based on deep Q-network for mobile actor node control in wireless sensor and actor networks," in *Proc. Int. Conf. Adv. Inf. Netw. Appl. Workshops (WAINA)*, 2017, pp. 195–200.

[173] L. Wang, W. Liu, D. Zhang, Y. Wang, E. Wang, and Y. Yang, "Cell selection with deep reinforcement learning in sparse mobile crowdsensing," in *Proc. IEEE Int. Conf. Distrib. Comput. Syst.*, Vienna, Austria, Jul. 2018, pp. 1543–1546.

[174] L. Wang *et al.*, "CCS-TA: Quality-guaranteed online task allocation in compressive crowdsensing," in *Proc. ACM Int. Joint Conf. Pervasive Ubiquitous Comput.*, 2015, pp. 683–694.

[175] F. Ingelrest, G. Barrenetxea, G. Schaefer, M. Vetterli, O. Couach, and M. Parlange, "SensorScope: Application-specific sensor network for environmental monitoring," *ACM Trans. Sensor Netw.*, vol. 6, no. 2, pp. 1–32, 2010.

[176] Y. Zheng, F. Liu, and H. P. Hsieh, "U-Air: When urban air quality inference meets big data," in *Proc. ACM SIGKDD Int. Conf. Knowl. Disc. Data Min.*, 2013, pp. 1436–1444.

[177] B. Zhang, C. H. Liu, J. Tang, Z. Xu, J. Ma, and W. Wang, "Learning-based energy-efficient data collection by unmanned vehicles in smart cities," *IEEE Trans. Ind. Informat.*, vol. 14, no. 4, pp. 1666–1676, Apr. 2018.

[178] L. Bracciale, M. Bonola, P. Loreti, G. Bianchi, R. Amici, and A. Rabuffi. (Jul. 2014). *CRAWDAD Dataset ROMA/Taxi (v. 2014-07-17)*. [Online]. Available: http://crawdad.org/roma/taxi/20140717

[179] L. Xiao, Y. Li, G. Han, H. Dai, and H. V. Poor, "A secure mobile crowdsensing game with deep reinforcement learning," *IEEE Trans. Inf. Forensics Security*, vol. 13, no. 1, pp. 35–47, Jan. 2018.

[180] Y. Zhang, B. Song, and P. Zhang, "Social behavior study under pervasive social networking based on decentralized deep reinforcement learning," *J. Netw. Comput. Appl.*, vol. 86, pp. 72–81, May 2017.

[181] M. Mohammadi, A. Al-Fuqaha, M. Guizani, and J.-S. Oh, "Semisupervised deep reinforcement learning in support of IoT and smart city services," *IEEE Internet Things J.*, vol. 5, no. 2, pp. 624–635, Apr. 2018.

[182] D. P. Kingma, S. Mohamed, D. J. Rezende, and M. Welling, "Semi-supervised learning with deep generative models," in *Proc. Adv. Neural Inf. Process. Syst.*, 2014, pp. 3581–3589.

[183] H. Huang, J. Yang, H. Huang, Y. Song, and G. Gui, "Deep learning for super-resolution channel estimation and DOA estimation based massive MIMO system," *IEEE Trans. Veh. Technol.*, vol. 67, no. 9, pp. 8549–8560, Sep. 2018.

[184] H. Ye, G. Y. Li, and B.-H. Juang, "Power of deep learning for channel estimation and signal detection in OFDM systems," *IEEE Wireless Commun. Lett.*, vol. 7, no. 1, pp. 114–117, Feb. 2018.

[185] A. Zappone, L. Sanguinetti, and M. Debbah, "User association and load balancing for massive MIMO through deep learning," in *Proc. Asilomar Conf. Signals Syst. Comput.*, Pacific Grove, CA, USA, Oct. 2018, pp. 1262–1266.

[186] J. Desrosiers and M. E. Lübbecke, "Branch-price-and-cut algorithms," in *Encyclopedia of Operations Research and Management Science*. Chichester, U.K.: Wiley, 2011, pp. 109–131.

[187] X. Wang, L. Gao, S. Mao, and S. Pandey, "CSI-based fingerprinting for indoor localization: A deep learning approach," *IEEE Trans. Veh. Technol.*, vol. 66, no. 1, pp. 763–776, Jan. 2017.

[188] Y. Bengio, P. Lamblin, D. Popovici, and H. Larochelle, "Greedy layer-wise training of deep networks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2007, pp. 153–160.

[189] J. Xiao, K. Wu, Y. Yi, and L. M. Ni, "FIFS: Fine-grained indoor fingerprinting system," in *Proc. ICCCN*, Munich, Germany, 2012, pp. 1–7.

[190] J. Vieira, E. Leitinger, M. Sarajlic, X. Li, and F. Tufvesson, "Deep convolutional neural networks for massive MIMO fingerprint-based positioning," in *Proc. Annu. Int. Symp. Pers. Indoor Mobile Radio Commun.*, 2017, pp. 1–6.

[191] D. L. Donoho, Y. Tsaig, I. Drori, and J.-L. Starck, "Sparse solution of underdetermined systems of linear equations by stagewise orthogonal matching pursuit," *IEEE Trans. Inf. Theory*, vol. 58, no. 2, pp. 1094–1121, Feb. 2012.

[192] Y. Bai, B. Ai, and W. Chen, "Deep learning based fast multiuser detection for massive machine-type communication," *arXiv preprint arXiv:1807.00967*, pp. 1–5, Jul. 2018.

[193] G. Cao, Z. Lu, X. Wen, T. Lei, and Z. Hu, "AIF: An artificial intelligence framework for smart wireless network management," *IEEE Commun. Lett.*, vol. 22, no. 2, pp. 400–403, Feb. 2018.

[194] Y. Zhan, Y. Xia, J. Zhang, T. Li, and Y. Wang, "Crowdsensing game with demand uncertainties: A deep reinforcement learning approach."

[195] N. C. Luong, P. Wang, D. Niyato, Y. Wen, and Z. Han, "Resource management in cloud networking using economic analysis and pricing models: A survey," *IEEE Commun. Surveys Tuts.*, vol. 19, no. 2, pp. 954–1001, 2nd Quart., 2017.

[196] N. C. Luong, D. T. Hoang, P. Wang, D. Niyato, D. I. Kim, and Z. Han, "Data collection and wireless communication in Internet of Things (IoT) using economic analysis and pricing models: A survey," *IEEE Commun. Surveys Tuts.*, vol. 18, no. 4, pp. 2546–2590, 4th Quart., 2016.

[197] F. Shi, Z. Qin, and J. A. McCann, "OPPay: Design and implementation of a payment system for opportunistic data services," in *Proc. IEEE Int. Conf. Distrib. Comput. Syst.*, Atlanta, GA, USA, Jul. 2017, pp. 1618–1628.

[198] Z. Jiang and J. Liang, "Cryptocurrency portfolio management with deep reinforcement learning," in *Proc. Intell. Syst. Conf. (IntelliSys)*, 2017, pp. 905–913.

[199] N. C. Luong, P. Wang, D. Niyato, Y.-C. Liang, F. Hou, and Z. Han, "Applications of economic and pricing models for resource management in 5G wireless networks: A survey," *IEEE Commun. Surveys Tuts.*, to be published.


Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:35:20 UTC from IEEE Xplore. Restrictions apply.

[200] J. Zhao, G. Qiu, Z. Guan, W. Zhao, and X. He, "Deep reinforcement learning for sponsored search real-time bidding," in *Proc. ACM SIGKDD Int. Conf. Knowl. Disc. Data Min.*, London, U.K., Aug. 2018, pp. 1021–1030.



**Nguyen Cong Luong** received the B.E. degree in electronic and telecommunication engineering from the Hanoi University of Science and Technology. His research interest includes Internet of Things.



**Dinh Thai Hoang** (M'16) received the Ph.D. degree in computer science and engineering from Nanyang Technological University, Singapore, in 2016. He is currently a Faculty Member with the School of Electrical and Data Engineering, University of Technology Sydney, Australia. His research interests include emerging topics in wireless communications and networking, such as ambient backscatter communications, vehicular communications, cybersecurity, IoT, and 5G networks. He is an Exemplary Reviewer of the IEEE TRANSACTIONS ON COMMUNICATIONS in 2018 and the IEEE TRANSACTIONS ON WIRELESS COMMUNICATIONS in 2017 and 2018. He is currently an Editor of the IEEE WIRELESS COMMUNICATIONS LETTERS and the IEEE TRANSACTIONS ON COGNITIVE COMMUNICATIONS AND NETWORKING.



**Shimin Gong** (M'15) received the B.E. and M.E. degrees in electrical engineering from the Huazhong University of Science and Technology, Wuhan, China, in 2008 and 2012, respectively, and the Ph.D. degree in computer engineering from Nanyang Technological University, Singapore, in 2014. He is currently an Associate Professor with the School of Intelligent Systems Engineering, Sun Yat-sen University, Shenzhen, China. His research interests include IoT, mobile edge computing, wireless communications, and networking. He was a Guest Lead Editor for a special issue in the IEEE TRANSACTIONS ON COGNITIVE COMMUNICATIONS AND NETWORKING. He was a recipient of the MAC and Cross-Layer Design Best Paper Award in IEEE WCNC 2019.



**Dusit Niyato** (M'09–SM'15–F'17) received the B.Eng. degree from the King Mongkut's Institute of Technology Ladkrabang, Thailand, in 1999 and the Ph.D. degree in electrical and computer engineering from the University of Manitoba, Canada, in 2008. He is currently a Professor with the School of Computer Science and Engineering, Nanyang Technological University, Singapore. His research interests are in the area of Internet of Things and network resource pricing.



**Ping Wang** (M'08–SM'15) received the Ph.D. degree in electrical engineering from the University of Waterloo, Canada, in 2008. She was with Nanyang Technological University, Singapore. She is currently an Associate Professor with the Department of Electrical Engineering and Computer Science, York University, Canada. Her current research interests include resource allocation in multimedia wireless networks, cloud computing, and smart grid. She was a co-recipient of the Best Paper Awards from the IEEE International Conference on Communications in 2007, from the IEEE Wireless Communications and Networking Conference in 2012, and from the IEEE Communications Society Green Communications and Computing Technical Committee in 2018. She has been serving as an Associate Editor for several journals, including the IEEE TRANSACTIONS ON WIRELESS COMMUNICATIONS, the *EURASIP Journal on Wireless Communications and Networking*, and the *International Journal of Ultra Wideband Communications and Systems*.



**Ying-Chang Liang** (F'11) is currently a Professor with the University of Electronic Science and Technology of China, China, where he leads the Center for Intelligent Networking and Communications and serves as the Deputy Director of the Artificial Intelligence Research Institute. He was a Professor with the University of Sydney, Australia, a Principal Scientist and a Technical Advisor with the Institute for Infocomm Research, Singapore, and a Visiting Scholar with Stanford University, USA. His research interests include wireless networking and communications, cognitive radio, symbiotic radio, dynamic spectrum access, Internet-of-Things, artificial intelligence, and machine learning techniques.

Dr. Liang was a recipient of the Prestigious Engineering Achievement Award from the Institution of Engineers, Singapore, in 2007, the Outstanding Contribution Appreciation Award from the IEEE Standards Association in 2011, the Recognition Award from the IEEE Communications Society Technical Committee on Cognitive Networks in 2018, and numerous paper awards, including the IEEE Jack Neubauer Memorial Award in 2014 and the IEEE Communications Society APB Outstanding Paper Award in 2012. He has been recognized by Thomson Reuters (currently, Clarivate Analytics) as a Highly Cited Researcher since 2014. He is the Founding Editor-in-Chief of the IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS: Cognitive Radio Series, and the Key Founder and currently the Editor-in-Chief of the IEEE TRANSACTIONS ON COGNITIVE COMMUNICATIONS AND NETWORKING. He is also serving as an Associate Editor-in-Chief for *China Communications*. He served as a Guest/Associate Editor of the IEEE TRANSACTIONS ON WIRELESS COMMUNICATIONS, the IEEE JOURNAL OF SELECTED AREAS IN COMMUNICATIONS, the *IEEE Signal Processing Magazine*, the IEEE TRANSACTIONS ON VEHICULAR TECHNOLOGY, and the IEEE TRANSACTIONS ON SIGNAL AND INFORMATION PROCESSING OVER NETWORK. He was also an Associate Editor-in-Chief of *Random Matrices: Theory and Applications* (World Scientific). He was a Distinguished Lecturer of the IEEE Communications Society and the IEEE Vehicular Technology Society. He was the Chair of the IEEE Communications Society Technical Committee on Cognitive Networks, and served as the TPC Chair and the Executive Co-Chair of the IEEE Globecom'17. He was elected as a fellow of the IEEE for contributions to cognitive radio communications.



**Dong In Kim** (S'89–M'91–SM'02–F'19) received the Ph.D. degree in electrical engineering from the University of Southern California, Los Angeles, CA, USA, in 1990. He was a Tenured Professor with the School of Engineering Science, Simon Fraser University, Burnaby, BC, Canada. Since 2007, he has been with Sungkyunkwan University, Suwon, South Korea, where he is currently a Professor with the College of Information and Communication Engineering. He was a recipient of the NRF of Korea Engineering Research Center in Wireless Communications for RF Energy Harvesting from 2014 to 2021. From 2001 to 2019, he served as an Editor of Spread Spectrum Transmission and Access and an Editor-at-Large of Wireless Communication I for the IEEE TRANSACTIONS ON COMMUNICATIONS. From 2002 to 2011, he also served as an Editor and a Founding Area Editor of Cross-Layer Design and Optimization for the IEEE TRANSACTIONS ON WIRELESS COMMUNICATIONS. From 2008 to 2011, he served as the Co-Editor-in-Chief for the IEEE/KICS JOURNAL OF COMMUNICATIONS AND NETWORKS. He served as the Founding Editor-in-Chief for the IEEE WIRELESS COMMUNICATIONS LETTERS from 2012 to 2015. He has been elevated to the grade of fellow of the IEEE for contributions to cross-layer design of wireless communications systems. He is also a fellow of the Korean Academy of Science and Technology and a member of the National Academy of Engineering of Korea.

Authorized licensed use limited to: INDIAN INSTITUTE OF INFORMATION TECHNOLOGY. Downloaded on October 01,2026 at 11:35:20 UTC from IEEE Xplore. Restrictions apply.